LAPORAN PROYEK AKHIR
Cyber Optik

NRP	: 1. 3223600046
	2. 3223600055
	3. 3223600058
Nama	: 1. Hari Tammim Mu'Izha
	2. Arimbi Kusuma Jati
	3. Ihsanta Zaki Sanjaya
Kelas	: 2 D4 Teknik Komputer B
Matakuliah	: Maretha Ruswiansari S.ST., M.T
Tanggal	: Praktikum Bahasa Pemrograman
Dosen	: Jumat, 6 Desember 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2024

Cyber Optik

I. Latar Belakang

Dalam era digital yang semakin maju, koneksi internet menjadi kebutuhan esensial bagi berbagai aktivitas, baik dalam skala personal maupun profesional. Stabilitas dan kecepatan koneksi internet menjadi aspek penting yang mendukung kelancaran komunikasi, transaksi, dan transfer data. Namun, sering kali kualitas koneksi internet yang digunakan tidak selalu sesuai dengan ekspektasi, terutama di beberapa wilayah dengan infrastruktur jaringan yang kurang optimal.

Pengguna internet memerlukan alat yang dapat memantau dan mengevaluasi kualitas koneksi mereka secara real-time. Selain itu, kebutuhan akan keamanan jaringan dan perlindungan privasi juga semakin meningkat, mengingat ancaman siber yang semakin canggih. Penggunaan jaringan virtual pribadi (VPN) dan kemampuan untuk memantau lalu lintas jaringan secara mendetail sangat penting untuk memastikan keamanan data yang dikirim dan diterima.

Cyber Optik diharapkan mampu melakukan pengujian kecepatan internet (*speedtest*), memantau jaringan (*sniffing*), serta menyediakan akses jaringan aman (VPN). Aplikasi ini dirancang sebagai solusi multifungsi untuk memberikan pengguna kontrol penuh atas koneksi jaringan mereka, memantau kualitas internet, serta meminimalkan risiko ancaman siber. Dengan aplikasi ini, pengguna dapat meningkatkan pemahaman mereka terhadap kualitas jaringan dan mengambil langkah proaktif untuk mengoptimalkannya

II. Tujuan dan Manfaat Aplikasi

Pengembangan aplikasi ini bertujuan untuk:

1. Mengukur Kualitas Koneksi Jaringan:
   Menyediakan fitur pengujian kecepatan (speedtest) yang mendetail, mencakup kecepatan unduh, unggah, ping, jitter, dan paket yang hilang, sehingga pengguna dapat memantau performa jaringan mereka dengan mudah.

2. Memantau Lalu Lintas Jaringan:
   Menyediakan fitur sniffing yang memungkinkan pengguna untuk menangkap paket data pada jaringan yang melewati aplikasi. Dikarenakan Batasan sniffing, hanya bisa menangkap packet yang masuk atau keluar dari jaringan aplikasi. Kemudian menambahkan perhitungan packet yang berhasil diterima, maupun di blokir, sehingga memudahkan pengguna dalam melihat packet jaringan yang ada.

3. Menyediakan Koneksi Aman:
   Memberikan koneksi terenkripsi melalui VPN untuk menjangkau internet dari luar region dari , terutama saat mengakses jaringan publik atau terbuka. Terdapat beberapa server luar, yang dapat di akses nantinya.

4. Memfasilitasi Transfer Data Aman:
   Menyediakan fitur transfer file menggunakan protokol SFTP untuk memudahkan pengiriman dan penerimaan file secara aman antar perangkat.

   Aplikasi ini memberikan berbagai manfaat kepada pengguna, di antaranya:

1. Monitoring Koneksi:
   Pengguna dapat memantau kualitas koneksi internet mereka secara real-time, termasuk mendapatkan informasi tentang kecepatan, latensi, dan stabilitas jaringan. Informasi ini membantu pengguna dalam mengevaluasi kualitas layanan dari penyedia internet mereka.

2. Keamanan dan Privasi Terjamin:
   Dengan adanya fitur VPN, pengguna dapat melindungi data pribadi mereka saat berselancar di internet, terutama ketika menggunakan jaringan publik. Selain itu, fitur sniffing membantu pengguna dalam mendeteksi aktivitas jaringan yang mencurigakan.

3. Kemudahan Transfer File:
   Fitur SFTP memungkinkan transfer file yang aman antar perangkat, memudahkan pengguna untuk mengelola data mereka tanpa khawatir dengan ancaman keamanan

4. Peningkatan Efisiensi dan Produktivitas:
   Dengan adanya pengukuran kecepatan jaringan dan keamanan yang lebih baik, aplikasi ini membantu pengguna dalam meningkatkan efisiensi aktivitas online, baik dalam bekerja maupun berkomunikasi.

III. Fitur-fitur dalam Aplikasi

Aplikasi memiliki berbagai fitur yang berhubungan dengan jaringan, berikut merupakan rincian dari fitur yang ada:

1. Speedtest
   Mengukur kecepatan jaringan dan kualitas oneksi. Terdapat data yang mendetail serta dapat menyimpan dan membagikan hasilnya. Berikut elemen yang tersedia:
   - Kecepatan unduh (Mbps): Tampilan hasil unduhan (download).
   - Kecepatan unggah (Mbps): Tampilan hasil unggahan (upload).
   - Keterangan: Tampilan hasil pengujian, apakah internetnya cepat atau tidak.

   Interaksi

   - Start: Melakukan pengujian speedtest

2. Sniffing
   Menangkap dan menampilkan paket yang melewati jaringan. Fitur ini menyertakan informasi lengkap tentang paket yang ditangkap. Berikut elemen yang tersedia:
   - No: urutan atau nomor ID paket yang ditangkap.
   - Time: Watku Ketika paket ditangkap.
   - Source: Alamat IP sumber (IPv4/IPv6) atau alamat MAC.

- Path: Lokasi dari https
- Protocol: Jenis protocol yang digunakan (TCP, UDP, ICMP, dll).
- Length: Ukuran paket dalam byte
- Info: Informasi tambahan terkait paket, seperti flags dan payload jika relevan.

Interaksi

- Save: Menyimpan packet yang ditangkap ke file log.
- Start: Menjalankan sniffing.
- Stop: Menghentikan proses sniffing.
- Share: Membagikan hasil sniffing ke aplikasi lain.

3. Terminal
   Menyediakan akses ke anatarmua basis perintah (CLI) untuk melakukan remote secara SSH. Berikut elemen yang tersedia:
   - Command Input Field: Area di mana pengguna dapat mengetik perintah.
   - Output Display: Tampilan hasil dari perintah yang dijalankan
   - History: Menyimpan riwayat perintah yang sudah dijalankan.

   Interaksi:
   - Connect/Disconect SSH: Koneksi ke server remote dengan protocol SSH.

4. VPN
   Menyediakan koneksi melalui jaringan terinkripsi. Dengan DNS over HTTPS (DoH) atau DNS over TLS (DoT). Berikut elemen yang tersedia:
   - DNS Protocol Status: Tampilan apakah DNS menggunakan protocol tertentu aktif.
   - Colocation Center: Tampilan lokasi server VPN yang terhubung.
   - Connection Type: Informasi apakah VPN menggunakan Wi-Fi, data seluluer, maupun jaringan lainnya.
   - Public IP: Tampilan alamat IP public pengguna saat VPN diaktifkan.
   - Device Info: Tampilan informasi tentang perangkat pengguna.

   Interaksi:

   - Togle VPN (On/Off): Mengaktifkan atau menonaktifkan VPN.

5. Files Transfer (SFTP)
   Mengirim dan menerima file antara perangkat Android dan perangkat lain. Berikut elemen yang tersedia:
   - File List: Daftar file local yang bisa dipilih untuk dikirim.
   - Connection Selector: Memilih perangkat lain yang terhubung melalui SFTP.

   Interaksi:

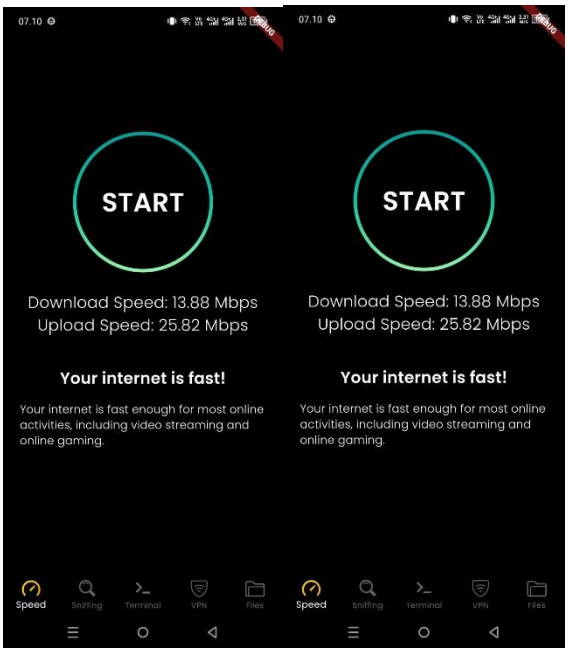
   - Read : Melihat daftar file dan folder, dan dapat melihat isi file txt
   - Upload : Melakukan upload file ke server tujuan

- Create : Membuat directory baru

IV. Panduan User dalam Penggunaan Aplikasi

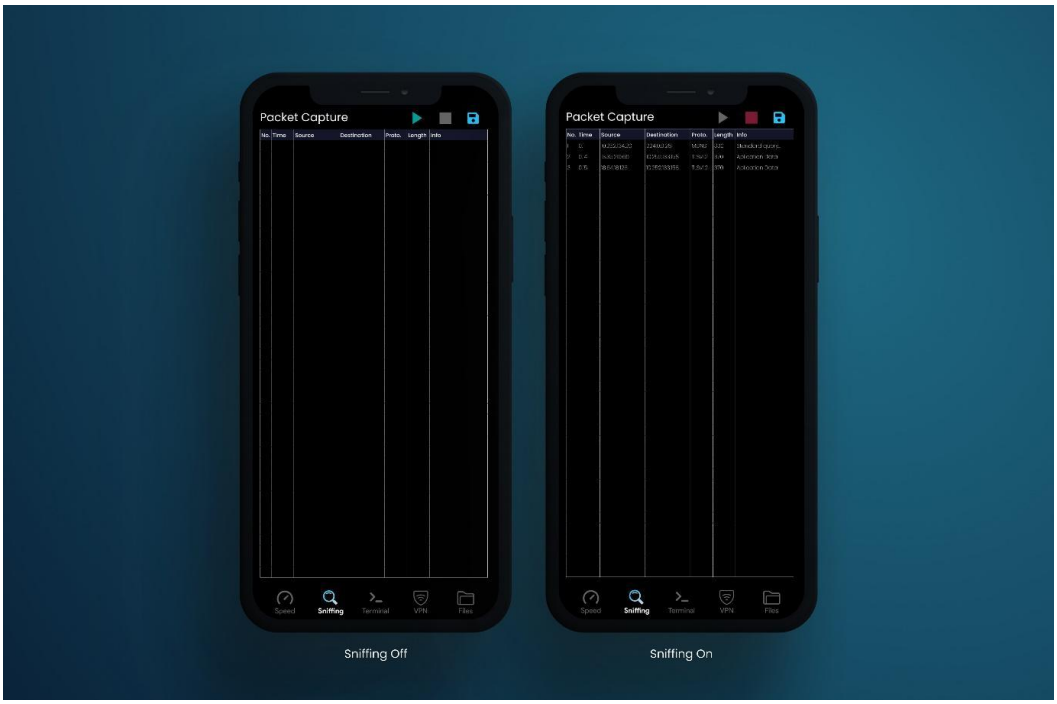
1. Setelah aplikasi terpasang anda dapat membuka aplikasi dan tampilan seperti berikut akan tampil.

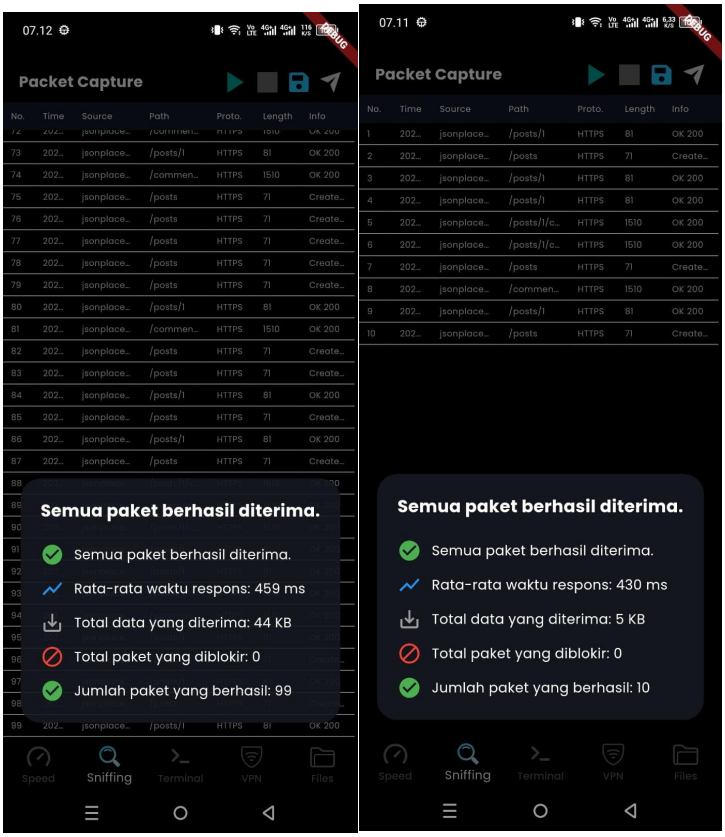
2. Kemudian akan tampil menu Speedtest seperti tampilan berikut.



Menu Speedtest menyediakan informasi kecepatan unduhan, kecepatan unggahan, dan kecepatan koneksi jaringan. Saat tulisan “START” ditekan maka dapat menampilkan persentase proses kecepatan koneksi internet. Kemudian menampilkan informasi koneksi internet.

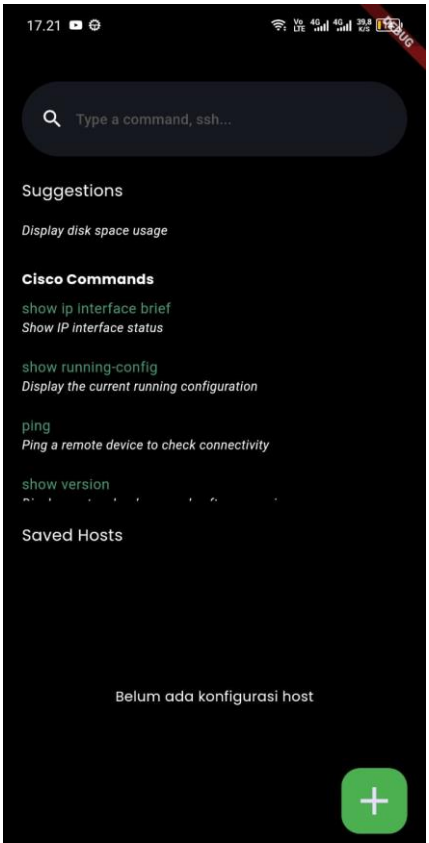
3. Anda dapat memantau lalu lintas jaringan anda pada fitur Sniffing,



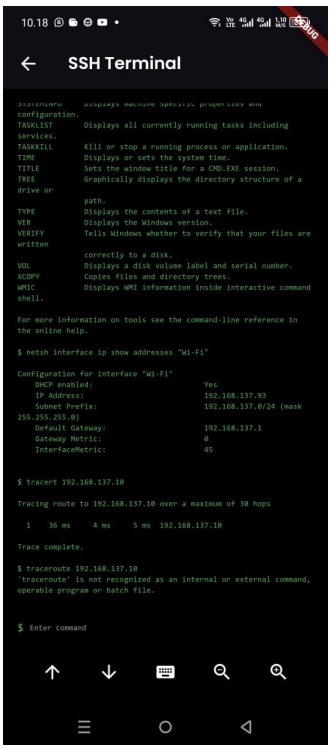


Pada fitur ini anda dapat menangkap paket yang berisi informasi tentang data jaringan komputer. Anda juga dapat membagikan paket yang telah di-capture dengan menekan tombol pesawat kertas di pojok kanan atas tampilan aplikasi. Apabila proses capturing package dihentikan, tekan tombol kotak stop di pojok kanan atas, maka aplikasi akan menampilkan pop up keterangan paket yang tercapture.

4. Jika anda ingin melakukan remote device lain, anda dapat menggunakan fitur Terminal milik Cyber Optik. Pada tab terminal akan tampil seperti berikut,

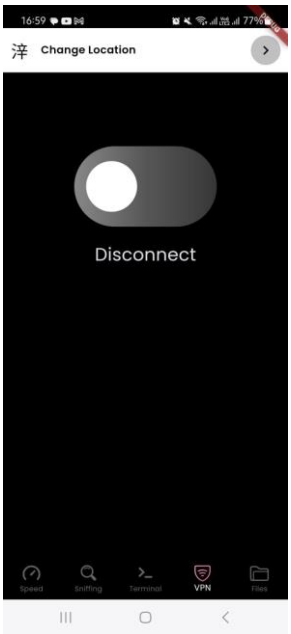


Untuk menyambungkan device secara remote anda perlu mengkonfigurasi host. Ketuk tombol “+” di sisi kanan bawah halaman dan anda akan diperintahkan untuk memasukkan IP address dari device yang ingin diremote. Device akan berperan sebagai host dan aplikasi ini akan berperan sebagai client. Setelah device terkonfigurasi maka anda dapat mengetikkan “ssh” pada search bar. Setelah itu anda akan melihat tampilan seperti di bawah ini,

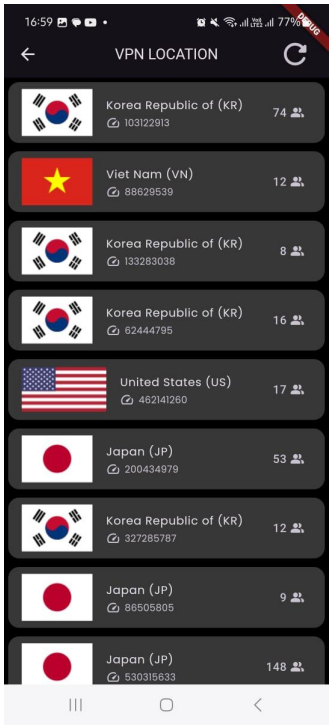


Anda telah terhubung dengan device host dan dapat mengetikkan command lain seperti ipconfig, ping, tracert, untuk mengambil data atau informasi lainnya yang terdapat pada device host. Pada bilah bawah halaman terdapat beberapa elemen yang dapat anda gunakan. Seperti elemen panah ke atas atau ke bawah untuk menampilkan command sebelum dan setelah command terbaru yang anda masukkan. Elemen keyboard dapat digunakan untuk memunculkan bilah keyboard. Kemudian anda dapat melakukan zoom in dan zoom out halaman dengan menekan elemen lup atau kaca pembesar di sisi kanan halaman.

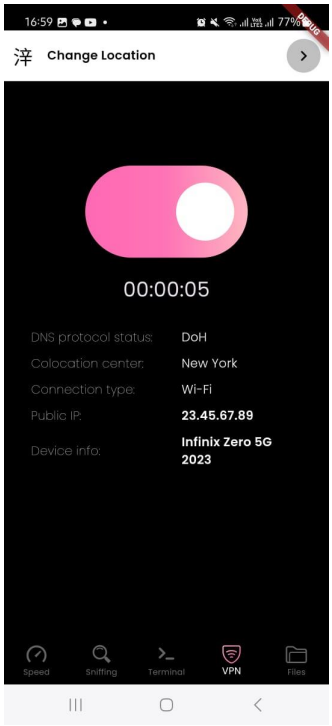
5. Anda dapat mengamankan alamat IP dari pihak lain anda dapat menggunakan fitur VPN pada aplikasi Cyber Optik ini. Anda akan dibawa ke tampilan awal seperti ini,



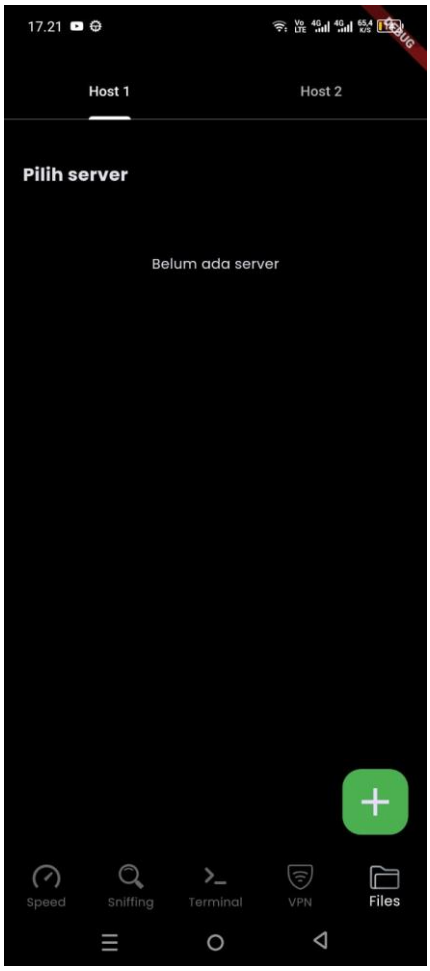
Bila anda ingin terhubung silahkan ketuk lingkaran putih untuk menghubungkan dengan server yang tersedia. Apabila anda ingin mengganti lokasi server, anda dapat mengetuk tanda panah di pojok kanan atas halaman aplikasi hingga muncul tampilan seperti berikut,



Anda dapat memperbarui halaman dengan menekan ikon refresh di pojok kanan atas halaman. Apabila sudah terhubung maka aplikasi akan menampilkan timer penggunaan VPN dan informasi seperti tampilan di bawah ini,



6. Pada halaman paling kanan anda akan menemukan fitur Files. Fitur ini dapat anda gunakan untuk mengunduh atau mengunggah file ke perangkat lain. Anda dapat menyinkronkan aplikasi dengan perangkat host dengan menekan tombol “+”. Bila device sudah terhubung maka selamat, anda dapat mengakses file dari perangkat host cukup dengan aplikasi Cyber Optik.



V. Program Lengkap

Aplikasi Cyber Optik dikembangkan dengan struktur proyek yang terorganisir untuk memisahkan logika frontend, backend, dan elemen lainnya secara jelas. Struktur ini memungkinkan pengelolaan kode yang efisien dan kolaborasi tim yang lebih baik. Program dapat dilihat dan di download melalui link berikut

https://github.com/FoldExz/network-app

Berikut adalah penjelasan singkat terkait folder utama dan file penting dalam program:

- Main.dart

File utama aplikasi yang menjadi entry point. File ini mengatur navigasi antar halaman dan inisialisasi state.

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';
import 'package:flutter1/pages/speed_test_page.dart';
import 'package:flutter1/pages/sniffing_page.dart';
import 'package:flutter1/pages/terminal_page.dart';
import 'package:flutter1/pages/vpn_page.dart';
import 'package:flutter1/pages/files_page.dart';
import 'package:flutter1/pages/details_screen.dart'; // Import DetailsScreen
import 'package:permission_handler/permission_handler.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData(
        brightness: Brightness.dark,
        scaffoldBackgroundColor: Colors.black,
        primarySwatch: Colors.blue,
        textTheme: const TextTheme(
          bodyLarge: TextStyle(fontFamily: 'Poppins'),
          bodyMedium: TextStyle(fontFamily: 'Poppins'),
        ),
        splashColor: Colors.transparent, // Hilangkan efek ripple
        highlightColor: Colors.transparent, // Hilangkan highlight
      ),
      home: const HomePage(),
      routes: {
        '/details': (context) => const DetailsScreen(),
      },
    );
  }
}

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  _HomePageState createState() => _HomePageState();
```

```
}

class _HomePageState extends State<HomePage> {
  int _selectedIndex = 0;

  // Remove `const` from the `_pages` list because it contains non-constant widget
instances.
  static final List<Widget> _pages = <Widget>[
    const SpeedTestPage(),
    const SniffingPage(),
    const TerminalPage(),
    const VPNPage(),
    const FileTransferPage(),
  ];

  @override
  void initState() {
    super.initState();
    _requestStoragePermission(); // Memanggil fungsi di sini
  }

  Future<void> _requestStoragePermission() async {
    if (await Permission.storage.request().isGranted) {
      // Izin diberikan, Anda dapat melanjutkan dengan logika penyimpanan
    } else {
      // Tampilkan pesan atau tangani kasus di mana izin tidak diberikan
    }
  }

  void _onItemTapped(int index) {
    setState(() {
      _selectedIndex = index;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: _pages[_selectedIndex],
      bottomNavigationBar: BottomNavigationBar(
        items: <BottomNavigationBarItem>[
          BottomNavigationBarItem(
            icon: _buildIcon('assets/icons/speedtest.svg', 0),
            label: 'Speed',
          ),
          BottomNavigationBarItem(
            icon: _buildIcon('assets/icons/sniffing.svg', 1),
            label: 'Sniffing',
          ),
          BottomNavigationBarItem(
            icon: _buildIcon('assets/icons/terminal.svg', 2),
            label: 'Terminal',
          ),
          BottomNavigationBarItem(
            icon: _buildIcon('assets/icons/vpn.svg', 3),
            label: 'VPN',
          ),
          BottomNavigationBarItem(
            icon: _buildIcon('assets/icons/files.svg', 4),
            label: 'Files',
          ),
        ],
        currentIndex: _selectedIndex,
        selectedItemColor: Colors.white,
        unselectedItemColor: Colors.white.withOpacity(0.4),
        backgroundColor: Colors.black,
```

```
        type: BottomNavigationBarType.fixed,
        enableFeedback: false, // Nonaktifkan efek getaran/sentuhan
        selectedIconTheme: const IconThemeData(
          color: Colors.white, // Warna ikon saat dipilih
          opacity: 1.0,
        ),
        unselectedIconTheme: const IconThemeData(
          color: Colors.white70, // Warna ikon saat tidak dipilih
          opacity: 0.4,
        ),
        onTap: _onItemTapped,
      ),
    );
  }

  Widget _buildIcon(String assetPath, int index) {
    return Opacity(
      opacity: _selectedIndex == index ? 1.0 : 0.4,
      child: SvgPicture.asset(
        assetPath,
        width: 24,
        height: 24,
        colorFilter: _selectedIndex == index
            ? null
            : const ColorFilter.mode(Colors.white, BlendMode.srcIn),
      ),
    );
  }
}
```

- Folder pages

  Folder pages/ berisi berbagai file yang mengatur tampilan dan logika antarmuka pengguna (UI) dari aplikasi Cyber Optik. Setiap file memiliki peran spesifik yang mendukung fungsi utama aplikasi. Berikut adalah jabaran kode untuk tiap file:

➢ Speed_test_page.dart

File yang berisi tampilan utama, logika, serta fungsi dari fitur speedtest.

```
import 'package:flutter/material.dart';
import 'dart:async';
import 'dart:math';
import 'package:http/http.dart' as http;
import '../utils/network_check.dart';

class SpeedTestPage extends StatefulWidget {
  const SpeedTestPage({super.key});

  @override
  _SpeedTestPageState createState() => _SpeedTestPageState();
}

class _SpeedTestPageState extends State<SpeedTestPage> {
  bool isConnected = true;
  double? downloadSpeed;
  double? uploadSpeed;
  bool isTesting = false;
  Timer? _progressTimer;
  StreamSubscription<bool>? _internetSubscription;
  int progressPercent = 0; // Untuk menyimpan persen progres

  final String downloadUrl =
      // 'http://192.168.100.11:3000/download/largeFile.bin';
```

```
      // 'http://hil.icmp.hetzner.com/100MB.bin';
      // 'http://ash.icmp.hetzner.com/100MB.bin';
      // 'https://fsn1-speed.hetzner.com/100MB.bin';
      'https://hel1-speed.hetzner.com/100MB.bin';
  final String uploadUrl = 'http://192.168.137.8:3000/upload';

  @override
  void initState() {
    super.initState();

    // Mendengarkan status koneksi internet

    _internetSubscription = NetworkCheck.internetStatusStream.listen((status) {
      setState(() {
        isConnected = status;
      });
    });
  }

  @override
  void dispose() {
    _progressTimer?.cancel(); // Batalkan timer jika ada

    _internetSubscription
        ?.cancel(); // Membatalkan subscription ketika widget dibuang

    super.dispose();
  }

  Future<void> _startSpeedTest() async {
    print("Speed test started"); // Log saat pengujian dimulai

    setState(() {
      isTesting = true;
      downloadSpeed = null;
      uploadSpeed = null;
      progressPercent = 0; // Reset persen progres
    });

    // Mulai animasi progres
    _startProgressAnimation();

    // Menjalankan pengujian download dan upload secara paralel
    var downloadFuture = _testDownloadSpeed();
    var uploadFuture = _testUploadSpeed();

    var results = await Future.wait([downloadFuture, uploadFuture]);

    // Mendapatkan hasil pengujian download dan upload
    double? download = results[0];
    double? upload = results[1];

    print(
        "Speed test finished. Download: $download Mbps, Upload: $upload Mbps"); //
Log saat pengujian selesai

    if (mounted) {
      setState(() {
        downloadSpeed = download;
        uploadSpeed = upload ?? -1.0; // Menandakan upload gagal jika null
        isTesting = false;
      });
    }

    // Setelah tes selesai, lanjutkan progres ke 100%
    if (mounted) {
```

```
      setState(() {
        progressPercent = 100; // Progres akan jadi 100% jika tes selesai
      });
    }

    _progressTimer?.cancel(); // Menghentikan animasi saat tes selesai
  }

  // Fungsi untuk memulai animasi progres
  void _startProgressAnimation() {
    _progressTimer = Timer.periodic(const Duration(milliseconds: 100), (timer) {
      if (isTesting && progressPercent < 98) {
        setState(() {
          progressPercent += 2; // Tambah progres 2% setiap 100ms hingga 89%
        });
      } else if (!isTesting && progressPercent < 100) {
        setState(() {
          progressPercent += 1; // Lanjutkan progres ke 100 setelah tes selesai
        });
      } else {
        _progressTimer?.cancel(); // Berhenti jika sudah 100% atau tes selesai
      }
    });
  }

  // Fungsi untuk mengonversi ukuran byte ke format yang lebih mudah dibaca (B, KB,
MB, GB)
  String convertSize(int sizeBytes) {
    if (sizeBytes == 0) {
      return "0B";
    }
    List<String> sizeName = [
      "B",
      "KB",
      "MB",
      "GB",
      "TB",
      "PB",
      "EB",
      "ZB",
      "YB"
    ];
    int i = (log(sizeBytes) / log(1024)).floor();
    num p = pow(1024, i);
    double s = (sizeBytes / p).roundToDouble();
    return "${s.toString()}${sizeName[i]}";
  }

  Future<double?> _testDownloadSpeed() async {
    print("Starting download test..."); // Log mulai pengujian download
    try {
      final stopwatch = Stopwatch()..start();
      final response = await http.get(Uri.parse(downloadUrl));

      if (response.statusCode == 200) {
        stopwatch.stop();
        int bytes = response.contentLength ?? response.bodyBytes.length;
        double seconds = stopwatch.elapsedMilliseconds / 1000;
        double speedMbps = (bytes / seconds) / (1024 * 1024) * 8;

        String fileSize = convertSize(
            bytes); // Mengonversi ukuran file ke format yang lebih mudah dibaca
        print(
            "Download speed test successful: $speedMbps Mbps, File Size: $fileSize");
// Log hasil pengujian download
```

```
        return speedMbps;
      } else {
        print("Failed to download. Status Code: ${response.statusCode}");
      }
    } catch (e) {
      print("Download speed test failed: $e");
    }
    return null;
  }

  Future<double?> _testUploadSpeed() async {
    print("Starting upload test..."); // Log mulai pengujian upload
    try {
      final stopwatch = Stopwatch()..start();

      var request = http.MultipartRequest('POST', Uri.parse(uploadUrl));
      var file = http.MultipartFile.fromBytes(
        'file',
        List<int>.generate(104857600, (index) => index % 256),
        filename: 'filedummy.txt',
      );
      request.files.add(file);

      var response = await request.send();
      stopwatch.stop();

      int bytesSent = file.length;
      double seconds = stopwatch.elapsedMilliseconds / 1000;
      double speedMbps = (bytesSent / seconds) / (1024 * 1024) * 8;

      if (response.statusCode == 200) {
        print(
             "Upload speed test successful: $speedMbps Mbps, Data Size: 100MB"); //
Log hasil pengujian upload
        return speedMbps;
      } else {
        print("Failed to upload data. Status Code: ${response.statusCode}");
      }
    } catch (e) {
      print("Upload speed test failed: $e");
    }
    return null;
  }

   //  Fungsi  untuk  menentukan  kategori  kecepatan  internet  dan  pesan  tambahan
berdasarkan rata-rata
  String getSpeedCategory(double downloadSpeed, double uploadSpeed) {
    // Hitung rata-rata kecepatan unduhan dan unggahan
    double averageSpeed = (downloadSpeed + uploadSpeed) / 2;

    // Menentukan kategori kecepatan berdasarkan rata-rata
    if (averageSpeed >= 50) {
      return "Your internet is very fast!";
    } else if (averageSpeed >= 10) {
      return "Your internet is fast!";
    } else {
      return "Your internet is slow!";
    }
  }

// Fungsi untuk mendapatkan penjelasan tambahan
  String getSpeedExplanation(double speed) {
    if (speed >= 50) {
       return "Your internet should be able to handle multiple devices streaming HD
videos, video conferencing, and gaming simultaneously.";
    } else if (speed >= 10) {
```

```
      return "Your internet is fast enough for most online activities, including video
streaming and online gaming.";
    } else {
      return "Your internet may struggle with HD video streaming.";
    }
  }

  @override
  Widget build(BuildContext context) {
    var mediaQuery = MediaQuery.of(context);
    double screenWidth = mediaQuery.size.width;
    double screenHeight = mediaQuery.size.height;
    double circleSize =
        (screenWidth < screenHeight ? screenWidth : screenHeight) * 0.5;
    double borderWidth = circleSize * 0.025;
    double startTextSize = circleSize * 0.22;
    double statusTextSize = (screenWidth * 0.05).clamp(24, 50);

    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          GestureDetector(
            onTap: isTesting ? null : _startSpeedTest,
            child: Container(
              width: circleSize,
              height: circleSize,
              decoration: BoxDecoration(
                gradient: isConnected
                    ? const LinearGradient(
                        colors: [
                          Color(0xFF00A19C),
                          Color(0xFF03CAA4),
                          Color(0xFF96F5A9),
                        ],
                        begin: Alignment.topCenter,
                        end: Alignment.bottomCenter,
                      )
                    : const LinearGradient(
                        colors: [
                          Color(0xFF7A1B34),
                          Color(0xFFFF2D6A),
                          Color(0xFFFFA376),
                        ],
                        begin: Alignment.topCenter,
                        end: Alignment.bottomCenter,
                      ),
                shape: BoxShape.circle,
                border: Border.all(
                  width: borderWidth,
                  color: Colors.transparent,
                ),
              ),
              child: Center(
                child: Container(
                  width: circleSize - (2 * borderWidth),
                  height: circleSize - (2 * borderWidth),
                  decoration: const BoxDecoration(
                    shape: BoxShape.circle,
                    color: Colors.black,
                  ),
                  child: Center(
                    child: Text(
                      isTesting ? '$progressPercent%' : 'START',
                      style: TextStyle(
                        fontFamily: 'Poppins',
```

```
                        fontWeight: FontWeight.w600,
                        fontSize: startTextSize,
                        color: Colors.white,
                      ),
                    ),
                  ),
                ),
              ),
            ),
          ),
          SizedBox(height: screenHeight * 0.03),
          Text(
            isTesting
                ? 'Testing in progress...'
                : downloadSpeed != null
                          ?  'Download  Speed:  ${downloadSpeed!.toStringAsFixed(2)}
Mbps\nUpload     Speed:     ${uploadSpeed     ==     null     ?     'failed'     :
uploadSpeed!.toStringAsFixed(2)} Mbps'
                    : isConnected
                        ? 'Connected to the internet'
                        : 'Check connection, \ninternet not connected',
            textAlign: TextAlign.center,
            style: TextStyle(
              fontFamily: 'Poppins',
              fontWeight: FontWeight.w300,
              fontSize: statusTextSize,
              color: Colors.white,
            ),
          ),
          // Tambahkan dua baris jarak
          SizedBox(height: screenHeight * 0.05),
          // Tampilkan kategori kecepatan internet berdasarkan hasil download speed
          if (downloadSpeed != null && uploadSpeed != null)
            Column(
              children: [
                  // Menampilkan kategori kecepatan internet berdasarkan rata-rata
download dan upload
                Text(
                  getSpeedCategory(downloadSpeed!, uploadSpeed!),
                  textAlign: TextAlign.center,
                  style: TextStyle(
                    fontFamily: 'Poppins',
                    fontWeight: FontWeight.w600,
                    fontSize: statusTextSize,
                    color: Colors.white,
                  ),
                ),
                SizedBox(height: screenHeight * 0.02), // Memberikan jarak
                   // Menampilkan penjelasan tambahan dengan huruf lebih kecil dan
perataan kiri
                Padding(
                  padding: EdgeInsets.only(
                      left: screenWidth * 0.06), // Memberikan jarak di kanan
                  child: Text(
                    getSpeedExplanation(
                            downloadSpeed!), // Menampilkan penjelasan berdasarkan
downloadSpeed
                    textAlign: TextAlign.left, // Perataan kiri
                    style: TextStyle(
                      fontFamily: 'Poppins',
                      fontWeight: FontWeight.w300,
                      fontSize:
                          statusTextSize * 0.7, // Ukuran huruf lebih kecil
                      color: Colors.white,
                    ),
                  ),
```

```
                ),
              ],
            ),
          ],
        ),
      );
    }
  }
```

- Sniffing_page.dart

  File yang berisi tampilan utama, logika, serta fungsi dari fitur sniffing.

```
import 'dart:async';
import 'dart:math';
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';
import 'package:flutter_network_inspector/models/inspector_result.dart';
import 'package:http/http.dart' as http;
import 'package:path_provider/path_provider.dart';
import 'package:share_plus/share_plus.dart';
import '../utils/sniffing_helper.dart';
import '../utils/network_check.dart';

class NetworkStatusPopup extends StatelessWidget {
  final double successRate;
  final int avgResponseTime;
  final int totalDataReceived;
  final int blockedPackets;
  final int successfulPackets;
  final bool isProcessing;

  const NetworkStatusPopup({
    super.key,
    required this.successRate,
    required this.avgResponseTime,
    required this.totalDataReceived,
    required this.blockedPackets,
    required this.successfulPackets,
    this.isProcessing = false,
  });

  @override
  Widget build(BuildContext context) {
    String networkStatusText;
    if (isProcessing) {
      networkStatusText = 'Memproses data, harap tunggu...';
    } else if (blockedPackets > 0) {
      networkStatusText = 'Terdapat paket yang diblokir: $blockedPackets';
    } else if (successfulPackets == 0) {
      networkStatusText = 'Tidak ada koneksi internet.';
    } else if (successfulPackets > 0 && avgResponseTime == 0) {
      networkStatusText = 'Kesalahan koneksi (Connection Error).';
    } else {
      networkStatusText = 'Semua paket berhasil diterima.';
    }

    final screenWidth = MediaQuery.of(context).size.width; // Ambil lebar layar

    return Align(
      alignment: Alignment.bottomCenter,
      child: Padding(
```

```
padding: const EdgeInsets.only(bottom: 70.0, top: 20.0),
child: Container(
  width: screenWidth * 0.9, // Gunakan nilai yang benar
  decoration: BoxDecoration(
    color: const Color(0xFF15181F).withOpacity(0.9),
    borderRadius: BorderRadius.circular(20),
  ),
  padding: const EdgeInsets.all(20),
  child: Column(
    mainAxisSize: MainAxisSize.min,
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        networkStatusText,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 20,
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 20),
      Row(
        children: [
          Icon(
            blockedPackets > 0
                ? Icons.error
                : successfulPackets == 0
                    ? Icons.wifi_off
                    : avgResponseTime == 0
                        ? Icons.warning
                        : Icons.check_circle,
            color: blockedPackets > 0
                ? Colors.red
                : successfulPackets == 0
                    ? Colors.orange
                    : avgResponseTime == 0
                        ? Colors.yellow
                        : Colors.green,
          ),
          const SizedBox(width: 10),
          Expanded(
            child: Text(
              networkStatusText,
              style: const TextStyle(color: Colors.white, fontSize: 16),
            ),
          ),
        ],
      ),
      const SizedBox(height: 10),
      Row(
        children: [
          const Icon(Icons.show_chart, color: Colors.blue),
          const SizedBox(width: 10),
          Expanded(
            child: Text(
              'Rata-rata waktu respons: $avgResponseTime ms',
              style: const TextStyle(color: Colors.white, fontSize: 16),
            ),
          ),
        ],
      ),
      const SizedBox(height: 10),
      Row(
        children: [
          const Icon(Icons.save_alt, color: Colors.grey),
          const SizedBox(width: 10),
```

```
                Expanded(
                  child: Text(
                    'Total data yang diterima: $totalDataReceived KB',
                    style: const TextStyle(color: Colors.white, fontSize: 16),
                  ),
                ),
              ],
            ),
            const SizedBox(height: 10),
            Row(
              children: [
                const Icon(Icons.block, color: Colors.red),
                const SizedBox(width: 10),
                Expanded(
                  child: Text(
                    'Total paket yang diblokir: $blockedPackets',
                    style: const TextStyle(color: Colors.white, fontSize: 16),
                  ),
                ),
              ],
            ),
            const SizedBox(height: 10),
            Row(
              children: [
                const Icon(Icons.check_circle, color: Colors.green),
                const SizedBox(width: 10),
                Expanded(
                  child: Text(
                    'Jumlah paket yang berhasil: $successfulPackets',
                    style: const TextStyle(color: Colors.white, fontSize: 16),
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    ),
  );
}
}

class SniffingPage extends StatefulWidget {
  const SniffingPage({super.key});

  @override
  _SniffingPageState createState() => _SniffingPageState();
}

class _SniffingPageState extends State<SniffingPage> {
  bool isStarted = false;
  final FNICLient _client = FNICLient();
  final List<InspectorResult> _sniffedData = [];
  Timer? _timer; // Menambahkan Timer

  @override
  void initState() {
    super.initState();
    FNICLient.inspectorNotifierList.addListener(_updateSniffedData);
  }

  @override
  void dispose() {
    FNICLient.inspectorNotifierList.removeListener(_updateSniffedData);
    _timer?.cancel(); // Hentikan timer saat dispose
    _client.close();
```

```
  super.dispose();
}

void _updateSniffedData() {
  final sniffedData = FNICLient.inspectorNotifierList.value;

  setState(() {
    _sniffedData.clear(); // Kosongkan data sebelumnya
    _sniffedData.addAll(sniffedData); // Menambahkan data yang baru
  });
}

void _calculateNetworkMetrics() {
  if (_sniffedData.isEmpty) return;

  // Hitung permintaan yang berhasil
  int successfulRequests = _sniffedData
      .where((result) =>
          result.statusCode != null &&
          result.statusCode! >= 200 &&
          result.statusCode! < 300)
      .length;

  // Hitung permintaan yang "diblokir" atau gagal (kode 400 dan 500)
  int blockedPackets = _sniffedData
      .where((result) =>
          result.statusCode != null &&
          (result.statusCode! >= 400 && result.statusCode! < 600))
      .length;

  // Total semua permintaan yang tercatat
  double successRate = (successfulRequests / _sniffedData.length) * 100;

  int totalResponseTime = 0;
  int totalDataReceived = 0;

  // Kalkulasi waktu respons rata-rata dan total data yang diterima
  for (var result in _sniffedData) {
    if (result.startTime != null && result.endTime != null) {
      int responseTime =
          result.endTime!.difference(result.startTime!).inMilliseconds;
      totalResponseTime += responseTime;
    }
    totalDataReceived += result.responseBodyBytes ?? 0;
  }

  int avgResponseTime = _sniffedData.isNotEmpty
      ? (totalResponseTime / _sniffedData.length).round()
      : 0;

  // Tampilkan hasil metrik di popup
  showModalBottomSheet(
    context: context,
    backgroundColor: Colors.transparent,
    builder: (BuildContext context) {
      return NetworkStatusPopup(
        successRate: successRate,
        avgResponseTime: avgResponseTime,
        totalDataReceived:
            (totalDataReceived / 1024).round(), // Konversi ke KB
        blockedPackets: blockedPackets,
        successfulPackets: successfulRequests, // Pass this value
      );
    },
  );
}
```

```
  void _toggleStart() {
    setState(() {
      isStarted = !isStarted;
      _client.setEnableLogging(isStarted);

      if (isStarted) {
        print("Sniffing started: $isStarted");

        // Kosongkan data di _sniffedData dan inspectorNotifierList
        _sniffedData.clear();

             //  Atur  inspectorNotifierList.value  menjadi  list  kosong,  memicu
notifyListeners secara otomatis
        FNICLient.inspectorNotifierList.value = [];

        _startRequestLoop();
      } else {
        print("Sniffing stopped: $isStarted");
        _timer?.cancel();

        // Tambahkan delay sebelum memproses metrik
        Future.delayed(const Duration(seconds: 2), () {
          _calculateNetworkMetrics(); // Hitung dan tampilkan metrik
        });
      }
    });
  }

  void _startRequestLoop() {
    // Mengatur timer untuk mengirimkan permintaan setiap 0.5 detik
    _timer = Timer.periodic(const Duration(milliseconds: 500), (timer) async {
      await _makeRequest();
    });
  }

  Future<void> _makeRequest() async {
    if (!isStarted) return; // Pastikan sniffing aktif

    // Pengecekan koneksi internet menggunakan fungsi checkInternetConnection
    bool hasInternet = await NetworkCheck.isInternetAvailable();
    if (!hasInternet) {
      print("Tidak ada koneksi internet.");

      // Mengatur hasil sniffing untuk koneksi yang buruk
      InspectorResult result = InspectorResult(
        url: Uri.parse("https://jsonplaceholder.typicode.com"),
        startTime: DateTime.now(),
        statusCode: null,
        reasonPhrase: 'No Internet Connection',
        responseBodyBytes: 0,
        endTime: DateTime.now(),
      );

      // Tambahkan hasil ke inspectorNotifierList untuk memicu notifyListeners
      final updatedList =
          List<InspectorResult>.from(FNICLient.inspectorNotifierList.value);
      updatedList.add(result);
      FNICLient.inspectorNotifierList.value = updatedList;

      return;
    }

    try {
      // Daftar endpoint yang bisa dipilih secara acak
      List<String> endpoints = [
```

```
      '/posts',
      '/posts/1',
      '/posts/1/comments',
      '/comments?postId=1',
      '/posts', // Untuk POST
      '/posts/1', // Untuk PUT dan PATCH
      '/posts/1', // Untuk DELETE
    ];

    // Pilih endpoint secara acak
    String selectedEndpoint = endpoints[Random().nextInt(endpoints.length)];

    // Tentukan metode HTTP berdasarkan endpoint yang dipilih
    String requestMethod;
    if (selectedEndpoint == '/posts') {
      requestMethod = 'POST';
    } else if (selectedEndpoint == '/posts/1') {
      requestMethod = Random().nextBool() ? 'PUT' : 'PATCH';
    } else if (selectedEndpoint == '/posts/1') {
      requestMethod = 'DELETE';
    } else {
      requestMethod = 'GET';
    }

    print(
       "Sending request to https://jsonplaceholder.typicode.com$selectedEndpoint");

    // Kirim permintaan sesuai metode yang ditentukan
    http.Response response;
    if (requestMethod == 'POST') {
      response = await _client.post(
        Uri.parse('https://jsonplaceholder.typicode.com$selectedEndpoint'),
        body: {'title': 'New Post', 'body': 'This is a new post.'},
      );
    } else if (requestMethod == 'PUT' || requestMethod == 'PATCH') {
      response = await _client.put(
        Uri.parse('https://jsonplaceholder.typicode.com$selectedEndpoint'),
        body: {
          'title': 'Updated Post',
          'body': 'This post has been updated.'
        },
      );
    } else if (requestMethod == 'DELETE') {
      response = await _client.delete(
        Uri.parse('https://jsonplaceholder.typicode.com$selectedEndpoint'),
      );
    } else {
      response = await _client.get(
        Uri.parse('https://jsonplaceholder.typicode.com$selectedEndpoint'),
      );
    }

    if (response.statusCode == 200) {
      print("Response status: ${response.statusCode}");
      print("Response body: ${response.body}");
    } else {
      print("Error: ${response.statusCode} ${response.reasonPhrase}");
    }
  } catch (e) {
    print("Connection error: $e");

    // Mengatur hasil sniffing untuk koneksi yang buruk
    InspectorResult result = InspectorResult(
      url: Uri.parse("https://jsonplaceholder.typicode.com"),
      startTime: DateTime.now(),
      statusCode: null,
```

```
      reasonPhrase: 'Connection Error',
      responseBodyBytes: 0,
      endTime: DateTime.now(),
    );

    // Tambahkan hasil ke inspectorNotifierList untuk memicu notifyListeners
    final updatedList =
        List<InspectorResult>.from(FNICLient.inspectorNotifierList.value);
    updatedList.add(result);
    FNICLient.inspectorNotifierList.value = updatedList;
  }
}

void _saveSniffedData() async {
  try {
    // Mendapatkan direktori penyimpanan eksternal untuk folder Downloads
    final directory = await getExternalStorageDirectory();
    final downloadDirectory = Directory('${directory?.path}/Download');
    // Membuat folder Download jika belum ada
    if (!await downloadDirectory.exists()) {
      await downloadDirectory.create(recursive: true);
    }

    final filePath = '${downloadDirectory.path}/sniffed_data.txt';

    // Membuat string dari data yang akan disimpan
    final dataToSave = _sniffedData.map((result) {
      return '''
    Start Time: ${formatDateTime(result.startTime)}
    Status Code: ${result.statusCode ?? '-'}
    Protocol: ${result.url?.scheme.toUpperCase() ?? '-'}
    Host: ${result.url?.host ?? '-'}
    Path: ${result.url?.path ?? '-'}
    Response Size: ${(result.responseBodyBytes ?? 0) / 1024} KB
    Message: ${result.reasonPhrase ?? '-'}
    ''';
    }).join('\n---\n');

    // Menyimpan data ke dalam file teks
    final file = File(filePath);
    await file.writeAsString(dataToSave);

    // Menampilkan notifikasi bahwa data berhasil disimpan
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Data berhasil disimpan di $filePath')),
    );
  } catch (e) {
    // Menampilkan notifikasi jika terjadi error
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Gagal menyimpan data: $e')),
    );
  }
}

void _shareData() async {
  try {
    // Mendapatkan direktori penyimpanan eksternal untuk folder Downloads
    final directory = await getExternalStorageDirectory();
    final downloadDirectory = Directory('${directory?.path}/Download');
    final filePath = '${downloadDirectory.path}/sniffed_data.txt';

    // Memeriksa apakah file ada
    final file = File(filePath);
    if (await file.exists()) {
      // Create an XFile from the file path
      final xFile = XFile(file.path);
```

```
      // Menggunakan Share untuk membagikan file
      await Share.shareXFiles([xFile], text: 'Data hasil sniffing Anda');
    } else {
      // Menampilkan notifikasi jika file tidak ditemukan
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('File tidak ditemukan: $filePath')),
      );
    }
  } catch (e) {
    // Menampilkan notifikasi jika terjadi error
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Gagal membagikan data: $e')),
    );
  }
}

@override
Widget build(BuildContext context) {
  var mediaQuery = MediaQuery.of(context);
  double screenWidth = mediaQuery.size.width;

  double headerHeight = 28;
  double rowHeight = 22;
  double noWidth = screenWidth * 0.09;
  double timeWidth = screenWidth * 0.11;
  double sourceWidth = screenWidth * 0.19;
  double destinationWidth = screenWidth * 0.19;
  double protoWidth = screenWidth * 0.13;
  double lengthWidth = screenWidth * 0.13;
  double infoWidth = screenWidth * 0.15;

  return Scaffold(
    body: Column(
      children: [
        const SizedBox(height: 50),
        Container(
          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
          child: Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              const Text(
                'Packet Capture',
                style: TextStyle(
                  fontFamily: 'Poppins',
                  fontSize: 18,
                  color: Colors.white,
                  fontWeight: FontWeight.bold,
                ),
              ),
              Row(
                children: [
                  GestureDetector(
                    onTap: _toggleStart,
                    child: SvgPicture.asset(
                      'assets/icons/start.svg',
                      color: isStarted ? const Color(0xFF666666) : null,
                      height: 24,
                      width: 24,
                    ),
                  ),
                  const SizedBox(width: 8),
                  GestureDetector(
                    onTap: isStarted ? _toggleStart : null,
                    child: SvgPicture.asset(
                      'assets/icons/stop.svg',
```

```
                color: isStarted ? null : const Color(0xFF666666),
                height: 24,
                width: 24,
              ),
            ),
            const SizedBox(width: 8),
            GestureDetector(
              onTap: _saveSniffedData,
              child: SvgPicture.asset(
                'assets/icons/save.svg',
                height: 24,
                width: 24,
              ),
            ),
            const SizedBox(width: 8),
            GestureDetector(
              onTap: _shareData,
              child: SvgPicture.asset(
                'assets/icons/share.svg',
                height: 24,
                width: 24,
              ),
            ),
          ],
        )
      ],
    ),
  ),
  Container(
    color: const Color(0xFF14172A),
    height: headerHeight,
    child: Row(
      children: [
        _buildTableHeader('No.', noWidth),
        _buildTableHeader('Time', timeWidth),
        _buildTableHeader('Source', sourceWidth),
        _buildTableHeader('Path', destinationWidth),
        _buildTableHeader('Proto.', protoWidth),
        _buildTableHeader('Length', lengthWidth),
        _buildTableHeader('Info', infoWidth),
      ],
    ),
  ),
  Expanded(
    child: SingleChildScrollView(
      child: Column(
        children: _sniffedData.asMap().entries.map((entry) {
          int index = entry.key + 1;
          InspectorResult data = entry.value;

          // Mendapatkan informasi protokol
          String protocol = data.url?.scheme.toUpperCase() ?? '-';

          // Menyusun informasi untuk kolom Info
          String info =
              '${data.reasonPhrase} ${data.statusCode ?? '-'}';

          // Membungkus baris dengan GestureDetector untuk interaksi klik
          return GestureDetector(
            onTap: () {
              Navigator.pushNamed(
                context,
                '/details',
                arguments:
                    data, // Mengirimkan objek data ke DetailsScreen
              );
```

```
                    },
                    child: Row(
                      children: [
                        _buildTableCell(index.toString(), noWidth, rowHeight),
                        _buildTableCell(
                            data.startTime?.toIso8601String() ?? '-',
                            timeWidth,
                            rowHeight),
                        _buildTableCell(
                            data.url?.host ?? '-', sourceWidth, rowHeight),
                        _buildTableCell(
                            data.url?.path ?? '-', destinationWidth, rowHeight),
                        _buildTableCell(protocol, protoWidth, rowHeight),
                        _buildTableCell(
                            data.responseBodyBytes?.toString() ?? '-',
                            lengthWidth,
                            rowHeight),
                        _buildTableCell(info, infoWidth, rowHeight),
                      ],
                    ),
                  );
                }).toList(),
              ),
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildTableHeader(String title, double width) {
    return Container(
      width: width,
      height: 28,
      alignment: Alignment.centerLeft,
      color: const Color(0xFF14172A),
      padding: const EdgeInsets.symmetric(horizontal: 8),
      child: Text(
        title,
        style: const TextStyle(
            fontFamily: 'Poppins', fontSize: 10, color: Colors.white),
      ),
    );
  }

  Widget _buildTableCell(String content, double width, double height) {
    return Container(
      width: width,
      height: height,
      alignment: Alignment.centerLeft,
      decoration: const BoxDecoration(
        border: Border(bottom: BorderSide(color: Colors.white)),
      ),
      padding: const EdgeInsets.symmetric(horizontal: 8),
      child: Text(
        content,
        style: const TextStyle(fontFamily: 'Poppins', fontSize: 10),
        overflow: TextOverflow.ellipsis,
      ),
    );
  }
}

String formatDateTime(DateTime? dateTime) {
  if (dateTime == null) return '-';
  return '${dateTime.hour}:${dateTime.minute}:${dateTime.second}';
```

```
}
```

- Terminal_page.dart

File yang berisi tampilan utama, logika, serta fungsi dari fitur terminal.

```
import 'package:flutter/material.dart';
import 'package:dart_ping/dart_ping.dart';
import 'package:flutter/services.dart';
import '../utils/ssh_connection.dart';
import '../styles/app_color.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../styles/app_styles.dart';

class TerminalPage extends StatefulWidget {
  const TerminalPage({super.key});

  @override
  _TerminalPageState createState() => _TerminalPageState();
}

class _TerminalPageState extends State<TerminalPage> {
  final _sshConnection = SSHConnection();
  final TextEditingController _hostController = TextEditingController();
  final TextEditingController _usernameController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  List<Map<String, String>> savedHosts = [];

  @override
  void initState() {
    super.initState();
    _loadHosts();
  }

  // Load the saved hosts from SharedPreferences
  Future<void> _loadHosts() async {
    final prefs = await SharedPreferences.getInstance();
    List<String>? hostList = prefs.getStringList('savedHosts');

    if (hostList != null) {
      setState(() {
        savedHosts = hostList.map((e) {
          final parts = e.split('|');
          return {
            'host': parts[0],
            'username': parts[1],
            'password': parts.length > 2 ? parts[2] : "",
          };
        }).toList();
      });
      print('Loaded hosts: $savedHosts'); // Debugging
    }
  }

  // Save a new host to SharedPreferences, ensuring no duplicates
  Future<void> _saveHost(String host, String username, String password,
      {bool isEdit = false, int? index}) async {
    final prefs = await SharedPreferences.getInstance();
    List<String> hostList = prefs.getStringList('savedHosts') ?? [];
    if (isEdit && index != null) {
      hostList[index] = '$host|$username|$password';
      savedHosts[index] = {
        'host': host,
```

```
      'username': username,
      'password': password, // Pastikan password diperbarui di savedHosts
    };
  } else {
    bool exists = hostList.any((item) {
      final parts = item.split('|');
      return parts[0] == host && parts[1] == username;
    });

    if (!exists) {
      hostList.add('$host|$username|$password');
      savedHosts.add({
        'host': host,
        'username': username,
        'password': password,
      });
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('This configuration already exists!')),
      );
      return;
    }
  }

  await prefs.setStringList('savedHosts', hostList);
  _loadHosts(); // Refresh UI
}

Future<void> _deleteHost(int index) async {
  final prefs = await SharedPreferences.getInstance();
  List<String> hostList = prefs.getStringList('savedHosts') ?? [];

  // Hapus host berdasarkan index
  hostList.removeAt(index);

  // Simpan kembali daftar yang telah diperbarui
  await prefs.setStringList('savedHosts', hostList);

  // Muat ulang daftar host
  _loadHosts();
}

@override
Widget build(BuildContext context) {
  // Daftar perintah dengan kategori yang dikelompokkan
  final List<Map<String, dynamic>> commands = [
    {
      'category': 'SSH Commands',
      'commands': [
        {
          'command': 'ssh [user@]host[:port]',
          'example': 'ssh user@192.168.1.1',
        },
        {
          'command': 'scp [file] [user@]host:[path]',
          'example': 'scp myfile.txt user@192.168.1.1:/path/to/directory',
        },
        {
          'command': 'ssh-keygen -t rsa',
          'example': 'Generate SSH key pair',
        },
      ],
    },
    {
      'category': 'Windows Commands',
      'commands': [
```

```
          {
            'command': 'ipconfig',
            'example': 'Display IP configuration',
          },
          {
            'command': 'ping',
            'example': 'Ping a host to check network connectivity',
          },
          {
            'command': 'dir',
            'example': 'List directory contents',
          },
          {
            'command': 'netstat',
            'example': 'Display network connections and listening ports',
          },
        ],
      },
      {
        'category': 'Linux Commands',
        'commands': [
          {
            'command': 'ls',
            'example': 'List directory contents',
          },
          {
            'command': 'top',
            'example': 'Display running processes',
          },
          {
            'command': 'chmod',
            'example': 'Change file permissions',
          },
          {
            'command': 'df -h',
            'example': 'Display disk space usage',
          },
        ],
      },
      {
        'category': 'Cisco Commands',
        'commands': [
          {
            'command': 'show ip interface brief',
            'example': 'Show IP interface status',
          },
          {
            'command': 'show running-config',
            'example': 'Display the current running configuration',
          },
          {
            'command': 'ping',
            'example': 'Ping a remote device to check connectivity',
          },
          {
            'command': 'show version',
            'example': 'Display system hardware and software version',
          },
        ],
      },
    ];

    // Membuat list untuk semua commands yang digabungkan dengan kategori sebagai
judul
    final List<Map<String, String>> allCommands = [];
    for (var category in commands) {
```

```
    // Menambahkan kategori sebagai item pertama
    allCommands.add({
      'command': category['category']!,
      'example': '', // Kategori tidak memiliki contoh
    });
    // Menambahkan semua commands dalam kategori
    for (var cmd in category['commands']) {
      allCommands.add({
        'command': cmd['command']!,
        'example': cmd['example']!,
      });
    }
  }

  return Scaffold(
    body: LayoutBuilder(
      builder: (context, constraints) {
        double width = constraints.maxWidth;
        double baseFontSize = width > 1200
            ? 24
            : width > 600
                ? 20
                : 16;

        return Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const SizedBox(height: 50),
              Container(
                decoration: BoxDecoration(
                  color: AppColors.darkGray,
                  borderRadius: BorderRadius.circular(1000),
                ),
                padding:
                    const EdgeInsets.symmetric(vertical: 8, horizontal: 16),
                child: Row(
                  children: [
                    const Icon(Icons.search,
                        color: AppColors.white, size: 20),
                    const SizedBox(width: 10),
                    Expanded(
                      child: TextField(
                        style: const TextStyle(color: AppColors.white),
                        decoration: InputDecoration(
                          hintText: 'Type a command, ssh...',
                          hintStyle: TextStyle(
                            color: AppColors.mediumGray,
                            fontSize: baseFontSize * 0.9,
                          ),
                          border: InputBorder.none,
                        ),
                        onSubmitted: (command) {
                          if (command.startsWith("ssh")) {
                            _showPasswordBottomSheet(context, command);
                          } else {
                            // Tampilkan snackbar jika perintah tidak dikenal
                            ScaffoldMessenger.of(context).showSnackBar(
                              const SnackBar(
                                content: Text('Command tidak ditemukan'),
                                backgroundColor:
                                    Color.fromARGB(255, 255, 255, 255),
                              ),
                            );
                          }
```

```
          },
        ),
      ),
    ],
  ),
),
const SizedBox(height: 20),
Text(
  'Suggestions',
  style: TextStyle(
    fontSize: baseFontSize,
    color: AppColors.white,
    fontFamily: 'Poppins',
  ),
),
const SizedBox(height: 20),
// Gunakan ListView.builder untuk daftar command items
Expanded(
  child: ListView.builder(
    itemCount: allCommands.length,
    itemBuilder: (context, index) {
      final command = allCommands[index];
      final commandText = command['command']!;
      final example = command['example']!;

     // Jika command adalah kategori, tampilkan sebagai teks kategori
      if (example.isEmpty) {
        return Padding(
          padding: const EdgeInsets.symmetric(vertical: 8.0),
          child: Text(
            commandText, // Kategori
            style: TextStyle(
              fontSize: baseFontSize - 2,
              color: AppColors.white,
              fontFamily: 'Poppins',
              fontWeight: FontWeight.bold,
            ),
          ),
        );
      } else {
       // Jika command adalah perintah, tampilkan sebagai CommandItem
        return CommandItem(
          command: commandText,
          example: example,
          baseFontSize: baseFontSize,
        );
      }
    },
  ),
),
const SizedBox(height: 20),
Text(
  'Saved Hosts',
  style: TextStyle(
    fontSize: baseFontSize,
    color: AppColors.white,
    fontFamily: 'Poppins',
  ),
),
const SizedBox(height: 10),
// Display saved hosts in ListView
Expanded(
  child: savedHosts.isEmpty
      ? Center(
          child: Text(
            'Belum ada konfigurasi host',
```

```
                              style: TextStyle(
                                color: const Color.fromARGB(255, 255, 255, 255),
                                fontFamily: 'Poppins',
                              ),
                            ),
                          )
                        : ListView.builder(
                            itemCount: savedHosts.length,
                            itemBuilder: (context, index) {
                              final host = savedHosts[index];
                              return ListTile(
                                title: Text(
                                  host['host'] ?? '',
                                  style: TextStyle(
                                      color: AppColors.white,
                                      fontSize: baseFontSize),
                                ),
                                subtitle: Text(
                                  host['username'] ??
                                      '', // Hanya tampilkan username
                                  style: TextStyle(
                                      color: AppColors.mediumGray,
                                      fontSize: baseFontSize * 0.9),
                                ),
                                onTap: () async {
                                  String selectedHost = host['host'] ?? '';
                                  String selectedUsername =
                                      host['username'] ?? '';
                                  String? savedPassword = host['password'];

                                  print(
                                    'Host: $selectedHost, Username: $selectedUsername,
Password: $savedPassword');

                                  if (selectedHost.isNotEmpty &&
                                      selectedUsername.isNotEmpty) {
                                    await _sshConnection.connect(
                                      selectedHost,
                                      selectedUsername,
                                      savedPassword!,
                                    );
                                    _showTerminalScreen(context);
                                  }
                                },
                                onLongPress: () {
                                  // Panggil modal untuk mengedit host
                                  _showEditHostBottomSheet(context, index, host);
                                },
                              );
                            },
                          ),
                        ),
                      ],
                    ),
                  );
                },
              ),
              floatingActionButton: _buildAddNewHost(context), // FloatingActionButton
            );
          }

          Widget _buildAddNewHost(BuildContext context) {
            return FloatingActionButton(
              onPressed: () {
                _showPasswordBottomSheet(context, ""); // Trigger modal to add new host
              },
```

```
    backgroundColor: Colors.green,
    child: const Icon(Icons.add, size: 40),
  );
}

Future<void> _showPasswordBottomSheet(
    BuildContext context, String command) async {
  // Load initial values
  String host = "";
  String username = "";
  String password = "";

  if (command.startsWith("ssh")) {
    final regex = RegExp(r"ssh\s+([^\@]+)\@([^\s]+)");
    final match = regex.firstMatch(command);

    if (match != null) {
      username = match.group(1) ?? '';
      host = match.group(2) ?? '';
    }
  }

  // Set initial values for controllers
  _hostController.text = host;
  _usernameController.text = username;
  _passwordController.text = password;

  // Show the modal bottom sheet
  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    builder: (BuildContext context) {
      return Padding(
        padding:
            EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
        child: Container(
          padding: const EdgeInsets.all(16),
          decoration: const BoxDecoration(
            color: AppColors.darkGray,
            borderRadius: BorderRadius.vertical(top: Radius.circular(25)),
          ),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  const Text(
                    'New Host',
                    style: TextStyle(
                      fontFamily: 'Poppins',
                      fontWeight: FontWeight.bold,
                      color: AppColors.white,
                    ),
                  ),
                  GestureDetector(
                    onTap: () {
                      Navigator.pop(context);
                    },
                    child: const Text(
                      'Cancel',
                      style: TextStyle(
                        fontFamily: 'Poppins',
                        fontWeight: FontWeight.normal,
                        color: AppColors.green,
                      ),
```

```
      ),
    ),
  ],
),
const SizedBox(height: 20),
TextField(
  controller: _hostController,
  decoration: const InputDecoration(
    labelText: 'Host Address',
    labelStyle: TextStyle(
      fontFamily: 'Poppins',
      fontWeight: FontWeight.w500,
      color: Color(0xFF8C91A5),
    ),
    filled: true,
    fillColor: Color(0xFF15181F),
    enabledBorder: UnderlineInputBorder(
      borderSide: BorderSide(color: Color(0xFF242834)),
    ),
    focusedBorder: UnderlineInputBorder(
      borderSide: BorderSide(color: Color(0xFF242834)),
    ),
  ),
  style: const TextStyle(color: Colors.white),
),
const SizedBox(height: 10),
TextField(
  controller: _usernameController,
  decoration: const InputDecoration(
    labelText: 'Username',
    labelStyle: TextStyle(
      fontFamily: 'Poppins',
      fontWeight: FontWeight.w500,
      color: Color(0xFF8C91A5),
    ),
    filled: true,
    fillColor: Color(0xFF15181F),
    enabledBorder: UnderlineInputBorder(
      borderSide: BorderSide(color: Color(0xFF242834)),
    ),
    focusedBorder: UnderlineInputBorder(
      borderSide: BorderSide(color: Color(0xFF242834)),
    ),
  ),
  style: const TextStyle(color: Colors.white),
),
const SizedBox(height: 10),

// Gunakan PasswordField untuk input password

PasswordField(
  controller:
      _passwordController, // Langsung menggunakan controller
),

const SizedBox(height: 20),
SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: () async {
      // Grab the values from controllers
      String host = _hostController.text;
      String username = _usernameController.text;
      String password = _passwordController.text;

      // Save the host configuration
```

```
                    await _saveHost(host, username, password);

                    Navigator.pop(context); // Close the modal
                  },
                  style: ElevatedButton.styleFrom(
                    backgroundColor: const Color(0xFF343746),
                    padding: const EdgeInsets.symmetric(vertical: 12),
                  ),
                  child: const Text(
                    "Continue",
                    style: TextStyle(
                      fontFamily: 'Poppins',
                      fontWeight: FontWeight.w600,
                      color: Color(0xFFFFFFFF),
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      );
    },
  );
}

Future<bool> _checkHostAvailability(String host) async {
  final ping = Ping(host, count: 3, timeout: 1);
  bool isReachable = false;

  await for (final pingData in ping.stream) {
    if (pingData.response != null) {
      isReachable = true;
      break;
    }
  }

  return isReachable;
}

void _showEditHostBottomSheet(
    BuildContext context, int index, Map<String, String> host) {
  // Deklarasi TextEditingController untuk setiap field
  final TextEditingController hostController =
      TextEditingController(text: host['host']);
  final TextEditingController usernameController =
      TextEditingController(text: host['username']);
  final TextEditingController passwordController =
      TextEditingController(text: host['password']); // Tambahkan di sini

  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    builder: (BuildContext context) {
      return Padding(
        padding:
            EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
        child: Container(
          padding: const EdgeInsets.all(16),
          decoration: const BoxDecoration(
            color: AppColors.darkGray,
            borderRadius: BorderRadius.vertical(top: Radius.circular(25)),
          ),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
```

```
Row(
  mainAxisAlignment: MainAxisAlignment.start,
  children: [
    const Text(
      'Edit Host',
      style: TextStyle(
        fontFamily: 'Poppins',
        fontWeight: FontWeight.bold,
        color: AppColors.white,
      ),
    ),
    const Spacer(),
    GestureDetector(
      onTap: () async {
        // Fungsi hapus
        await _deleteHost(index);
        Navigator.pop(context);
      },
      child: const Text(
        'Delete',
        style: TextStyle(
          fontFamily: 'Poppins',
          fontWeight: FontWeight.normal,
          color: Colors.red,
        ),
      ),
    ),
    const SizedBox(width: 10),
    GestureDetector(
      onTap: () {
        Navigator.pop(context);
      },
      child: const Text(
        'Cancel',
        style: TextStyle(
          fontFamily: 'Poppins',
          fontWeight: FontWeight.normal,
          color: AppColors.green,
        ),
      ),
    ),
  ],
),
const SizedBox(height: 20),
TextField(
  controller: hostController,
  decoration: const InputDecoration(
    labelText: 'Host Address',
    labelStyle: TextStyle(color: Color(0xFF8C91A5)),
    filled: true,
    fillColor: Color(0xFF15181F),
    enabledBorder: UnderlineInputBorder(
      borderSide: BorderSide(color: Color(0xFF242834)),
    ),
    focusedBorder: UnderlineInputBorder(
      borderSide: BorderSide(color: Color(0xFF242834)),
    ),
  ),
  style: const TextStyle(color: Colors.white),
),
const SizedBox(height: 10),
TextField(
  controller: usernameController,
  decoration: const InputDecoration(
    labelText: 'Username',
    labelStyle: TextStyle(color: Color(0xFF8C91A5)),
```

```
                    filled: true,
                    fillColor: Color(0xFF15181F),
                    enabledBorder: UnderlineInputBorder(
                      borderSide: BorderSide(color: Color(0xFF242834)),
                    ),
                    focusedBorder: UnderlineInputBorder(
                      borderSide: BorderSide(color: Color(0xFF242834)),
                    ),
                  ),
                  style: const TextStyle(color: Colors.white),
                ),
                const SizedBox(height: 10),
                // Gunakan PasswordField untuk input password
                PasswordField(
                  controller:
                      passwordController, // Pastikan controller dihubungkan
                ),
                const SizedBox(height: 20),
                SizedBox(
                  width: double.infinity,
                  child: ElevatedButton(
                    onPressed: () async {
                      String updatedHost = hostController.text;
                      String updatedUsername = usernameController.text;
                      String updatedPassword = passwordController.text;

                      await _saveHost(
                        updatedHost,
                        updatedUsername,
                        updatedPassword,
                        isEdit: true,
                        index: index,
                      );

                      Navigator.pop(context); // Tutup modal
                    },
                    style: ElevatedButton.styleFrom(
                      backgroundColor: const Color(0xFF343746),
                      padding: const EdgeInsets.symmetric(vertical: 12),
                    ),
                    child: const Text(
                      "Save",
                      style: TextStyle(
                        fontFamily: 'Poppins',
                        fontWeight: FontWeight.w600,
                        color: Color(0xFFFFFFFF),
                      ),
                    ),
                  ),
                ),
              ],
            ),
          ),
        );
      },
    );
  }

  void _showTerminalScreen(BuildContext context) async {
    bool isConnected = _sshConnection.isConnected;

    if (isConnected) {
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => TerminalScreen(sshConnection: _sshConnection),
```

```
        ),
      );
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text("Koneksi SSH gagal. Tidak dapat membuka terminal."),
        ),
      );
    }
  }
}

Future<void> _saveConfiguration(
    String host, String username, String password) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('host', host);
  await prefs.setString('username', username);
  await prefs.setString('password', password);
}

Future<Map<String, String?>> _loadConfiguration() async {
  final prefs = await SharedPreferences.getInstance();
  String? host = prefs.getString('host');
  String? username = prefs.getString('username');
  String? password = prefs.getString('password');

  return {'host': host, 'username': username, 'password': password};
}

class TerminalScreen extends StatefulWidget {
  final SSHConnection sshConnection;

  const TerminalScreen({super.key, required this.sshConnection});

  @override
  _TerminalScreenState createState() => _TerminalScreenState();
}

class _TerminalScreenState extends State<TerminalScreen> {
  final TextEditingController _commandController = TextEditingController();
  final FocusNode _focusNode = FocusNode();
  final ScrollController _scrollController = ScrollController();
  String _output = "";
  double _fontSize = 14;

  final List<String> _commandHistory = [];
  int _historyIndex = -1;

  Future<void> _sendCommand(String command) async {
    if (command.isEmpty) return;

    final result = await widget.sshConnection.executeCommand(command);
    setState(() {
      _output += '\n\$ $command\n$result';
      _commandHistory.add(command);
      _historyIndex = _commandHistory.length;
    });

    _scrollToBottom();
    _commandController.clear();
  }

  void _scrollToBottom() {
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (_scrollController.hasClients) {
        _scrollController.animateTo(
```

```
        _scrollController.position.maxScrollExtent,
        duration: const Duration(milliseconds: 300),
        curve: Curves.easeOut,
      );
    }
  });
}

void _toggleKeyboard() {
  if (_focusNode.hasFocus) {
    _focusNode.unfocus();
  } else {
    FocusScope.of(context).requestFocus(_focusNode);
  }
}

void _zoomIn() {
  setState(() {
    _fontSize += 2;
  });
}

void _zoomOut() {
  setState(() {
    _fontSize = (_fontSize - 2).clamp(10, 30);
  });
}

void _handleArrowUp() {
  setState(() {
    if (_historyIndex > 0) {
      _historyIndex--;
      _commandController.text = _commandHistory[_historyIndex];
      _commandController.selection = TextSelection.fromPosition(
          TextPosition(offset: _commandController.text.length));
    }
  });
}

void _handleArrowDown() {
  setState(() {
    if (_historyIndex < _commandHistory.length - 1) {
      _historyIndex++;
      _commandController.text = _commandHistory[_historyIndex];
    } else {
      _historyIndex = _commandHistory.length;
      _commandController.clear();
    }
    _commandController.selection = TextSelection.fromPosition(
        TextPosition(offset: _commandController.text.length));
  });
}

@override
void dispose() {
  widget.sshConnection.close();
  _focusNode.dispose();
  _scrollController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text(
```

```
      "SSH Terminal",
      style: TextStyle(fontFamily: 'Poppins', fontWeight: FontWeight.w600),
    ),
    backgroundColor: Colors.black,
    foregroundColor: Colors.white,
  ),
  body: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      children: [
        Expanded(
          child: SingleChildScrollView(
            controller: _scrollController,
            child: SelectableText(
              _output,
              style: TextStyle(
                fontSize: _fontSize,
                color: Colors.green,
                fontFamily: 'Consolas',
              ),
            ),
          ),
        ),
        Padding(
          padding: const EdgeInsets.only(bottom: 1),
          child: Column(
            children: [
              Row(
                children: [
                  const Text(
                    '\$ ',
                    style: TextStyle(
                      color: Colors.green,
                      fontFamily: 'Consolas',
                    ),
                  ),
                  Expanded(
                    child: TextField(
                      controller: _commandController,
                      focusNode: _focusNode,
                      style: TextStyle(
                        color: Colors.white,
                        fontFamily: 'Consolas',
                        fontSize: _fontSize,
                      ),
                      decoration: const InputDecoration(
                        hintText: 'Enter command',
                        border: InputBorder.none,
                        hintStyle: TextStyle(color: Colors.grey),
                      ),
                      onSubmitted: (command) {
                        _sendCommand(command);
                      },
                    ),
                  ),
                ],
              ),
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                children: [
                  IconButton(
                    icon:
                        const Icon(Icons.arrow_upward, color: Colors.white),
                    onPressed: _handleArrowUp,
                  ),
                  IconButton(
```

```
                        icon: const Icon(Icons.arrow_downward,
                            color: Colors.white),
                        onPressed: _handleArrowDown,
                      ),
                      IconButton(
                        icon: const Icon(Icons.keyboard, color: Colors.white),
                        onPressed: _toggleKeyboard,
                      ),
                      IconButton(
                        icon: const Icon(Icons.zoom_out, color: Colors.white),
                        onPressed: _zoomOut,
                      ),
                      IconButton(
                        icon: const Icon(Icons.zoom_in, color: Colors.white),
                        onPressed: _zoomIn,
                      ),
                    ],
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }

  // Widget _buildShortcutText(String label, VoidCallback onTap) {
  //   return GestureDetector(
  //     onTap: onTap,
  //     child: Text(
  //       label,
  //       style: TextStyle(
  //         fontSize: 16,
  //         fontFamily: 'Arial Rounded MT Bold',
  //         color: _isCtrlActive && label == 'Ctrl' ? Colors.blue : Colors.white,
  //       ),
  //     ),
  //   );
  // }
}

// Widget untuk input password dengan show/hide
class PasswordField extends StatefulWidget {
  final TextEditingController controller;

  const PasswordField({
    super.key,
    required this.controller,
  });

  @override
  _PasswordFieldState createState() => _PasswordFieldState();
}

class _PasswordFieldState extends State<PasswordField> {
  bool _isObscured = true;

  @override
  Widget build(BuildContext context) {
    return TextField(
      controller: widget.controller, // Gunakan controller dari parent
      obscureText: _isObscured, // Tampilkan/hide password
      style: const TextStyle(color: Colors.white),
      decoration: InputDecoration(
        labelText: 'Password',
```

```
        labelStyle: const TextStyle(color: Colors.white),
        suffixIcon: IconButton(
          icon: Icon(
            _isObscured ? Icons.visibility : Icons.visibility_off,
            color: Colors.white,
          ),
          onPressed: () {
            setState(() {
              _isObscured = !_isObscured; // Toggle password visibility
            });
          },
        ),
        filled: true,
        fillColor: const Color(0xFF15181F),
        enabledBorder: const UnderlineInputBorder(
          borderSide: BorderSide(color: Color(0xFF242834)),
        ),
        focusedBorder: const UnderlineInputBorder(
          borderSide: BorderSide(color: Color(0xFF242834)),
        ),
      ),
    );
  }
}

class CommandItem extends StatelessWidget {
  final String command;
  final String example;
  final double baseFontSize;

  const CommandItem({
    super.key,
    required this.command,
    required this.example,
    required this.baseFontSize,
  });

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            command,
            style: TextStyle(
              fontSize: baseFontSize * 0.9,
              color: const Color(0xFF479F79),
              fontFamily: 'CONSOLA',
            ),
          ),
          Text(
            example,
            style: TextStyle(
              fontSize: baseFontSize * 0.8,
              color: Colors.white,
              fontStyle: FontStyle.italic,
              fontFamily: 'consolai',
            ),
          ),
        ],
      ),
    );
  }
}
```

- Vpn_page.dart

File yang berisi tampilan utama, logika, serta fungsi dari fitur VPN.

```
import 'dart:convert';
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter1/pages/timer.dart';
import 'package:get/get.dart';
import 'package:http/http.dart';
import 'location_vpn.dart';
import 'timer.dart';
import 'package:flutter1/APIS/vpn_control.dart';
import 'location_vpn.dart';

class VPNPage extends StatefulWidget {
  const VPNPage({super.key});

  @override
  _VPNPageState createState() => _VPNPageState();
}

class _VPNPageState extends State<VPNPage> {
  bool isConnected = false;
  bool isConnecting = false;
  final _state = Get.put(Vpncontrol());
  final _vpnLocation = Get.put(LocationVpn());

  // Fungsi untuk mengatur status toggle
  void _toggleVPN() {
    setState(() {
      if (!isConnected && !isConnecting) {
        // Ketika tombol di-klik saat disconnected, akan memulai proses connecting
        isConnecting = true;
        Future.delayed(const Duration(seconds: 3), () {
          setState(() {
            isConnected = true;
            isConnecting = false; // Setelah 3 detik, set to connected
          });
        });
      } else if (isConnected) {
        // Jika sudah terhubung, klik akan memutuskan koneksi
        isConnected = false;
        isConnecting = false;
      }
    });
  }

  @override
  Widget build(BuildContext context) {
    // Menggunakan ukuran layar untuk membuat responsive
    double screenWidth = MediaQuery.of(context).size.width;
    double toggleWidth = screenWidth * 0.5; // 50% dari lebar layar
    double toggleHeight = toggleWidth * (252 / 444); // Rasio 444x252 px
    double circleSize = toggleHeight * 0.63; // Ukuran lingkaran dalam toggle

    // Ukuran font untuk detail
    double detailFontSize = 18.0; // Ukuran font detail

    return Scaffold(
      backgroundColor: Colors.black, // Latar belakang hitam
      body: Column(
        children: [
```

```
_changeLocation(),
Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment
        .center, // Mengatur agar toggle tidak ke tengah
    crossAxisAlignment: CrossAxisAlignment.center, // Rata tengah
    children: [
      const SizedBox(
          height: 100), // Jarak 50 piksel antara atas app dan toggle
      // Toggle Switch
      GestureDetector(
        onTap: _toggleVPN, // Fungsi saat toggle ditekan
        child: AnimatedContainer(
          duration:
              const Duration(milliseconds: 300), // Animasi smooth
          width: toggleWidth,
          height: toggleHeight,
          padding: const EdgeInsets.symmetric(horizontal: 16),
          decoration: BoxDecoration(
            gradient: LinearGradient(
              colors: isConnected || isConnecting
                  ? const [
                      Color(0xFFFF69B4),
                      Color(0xFFFF74B6),
                      Color(0xFFFFB6C1),
                    ]
                  : const [
                      Color(0xFF888888),
                      Color(0xFF575757),
                      Color(0xFF373737),
                    ],
              begin: Alignment.centerLeft,
              end: Alignment.centerRight,
            ),
            borderRadius: BorderRadius.circular(
                120), // Bentuk lonjong sesuai ukuran baru
          ),
          child: Stack(
            children: [
              // Lingkaran putih di dalam toggle
              AnimatedAlign(
                duration: const Duration(milliseconds: 300),
                alignment: isConnected || isConnecting
                    ? Alignment.centerRight
                    : Alignment.centerLeft, // Posisi lingkaran
                child: Container(
                  width: circleSize,
                  height: circleSize,
                  decoration: const BoxDecoration(
                    color: Colors.white,
                    shape: BoxShape.circle,
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
      const SizedBox(height: 20), // Jarak antara toggle dan teks
      // Teks status berdasarkan koneksi
      // Text(
      //   isConnecting
      //       ? 'Connecting...'
      //       : isConnected
      //           ? 'Connected'
      //           : 'Disconnected',
```

```
                //   style: TextStyle(
                //     fontFamily: 'Poppins',
                //     fontSize: screenWidth *
                //         0.07, // Ukuran teks status (10% dari lebar layar)
                //     fontWeight: FontWeight.normal, // Regular
                //     color: Colors.white,
                //   ),
                // ),
                isConnecting
                    ? Text(
                        'Connecting',
                        style: TextStyle(fontSize: 30),
                      )
                    : isConnected
                        ? TimerWidget(startTimer: true)
                        : Text('Disconnect', style: TextStyle(fontSize: 30)),
                // Jika statusnya "Connected", tampilkan detail di bawahnya
                if (isConnected) ...[
                  const SizedBox(height: 30), // Jarak antar teks lebih kecil
                  Container(
                    padding: const EdgeInsets.symmetric(horizontal: 32),
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        _buildDetailRow(
                            'DNS protocol status', 'DoH', detailFontSize),
                        const SizedBox(height: 10),
                        _buildDetailRow(
                            'Colocation center', 'New York', detailFontSize),
                        const SizedBox(height: 10),
                        _buildDetailRow(
                            'Connection type', 'Wi-Fi', detailFontSize),
                        const SizedBox(height: 10),
                        _buildDetailRow(
                            'Public IP', '23.45.67.89', detailFontSize,
                            isBold: true),
                        const SizedBox(height: 10),
                        _buildDetailRow('Device info', 'Infinix Zero 5G 2023',
                            detailFontSize,
                            isBold: true),
                      ],
                    ),
                  ),
                ],
                const SizedBox(height: 10),
              ],
            ),
          ),
        ],
      ),
    );
  }

  Widget _changeLocation() {
    return GestureDetector(
      onTap: () {
        // Navigasi ke halaman `LocationVpn`
        Navigator.push(
          context,
          MaterialPageRoute(builder: (context) => const LocationVpn()),
        );
      },
      child: SafeArea(
        child: Container(
          padding: const EdgeInsets.all(10.0),
          color: Colors.white,
```

```
        height: 60,
        child: Row(
          children: [
            const Icon(
              CupertinoIcons.globe,
              color: Colors.black,
              size: 25,
            ),
            const SizedBox(
              width: 16,
            ),
            const Text(
              'Change Location',
              style: TextStyle(
                color: Colors.black,
                fontSize: 18,
                fontWeight: FontWeight.w600,
              ),
            ),
            const Spacer(),
            const CircleAvatar(
              backgroundColor: Color.fromARGB(255, 192, 192, 192),
              child: Icon(
                Icons.keyboard_arrow_right_rounded,
                color: Colors.black,
                size: 25,
              ),
            ),
          ],
        ),
      ),
    ),
  );
}

// Fungsi untuk membangun baris detail dengan label dan hasil
Widget _buildDetailRow(String label, String value, double fontSize,
    {bool isBold = false}) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.start, // Rata kiri
    children: [
      // Label
      Expanded(
        flex: 5,
        child: Text(
          '$label: ',
          style: TextStyle(
            fontFamily: 'Poppins',
            fontSize: fontSize, // Ukuran font yang diatur
            fontWeight: FontWeight.w200, // ExtraLight
            color: Colors.white,
          ),
        ),
      ),
      // Hasil
      Expanded(
        flex: 4,
        child: Text(
          value,
          style: TextStyle(
            fontFamily: 'Poppins',
            fontSize: fontSize, // Ukuran font yang diatur
            fontWeight: isBold
                ? FontWeight.w600
                : FontWeight.w300, // Semibold atau Light
            color: Colors.white,
```

```
            ),
          ),
        ),
      ],
    );
  }
}
```

- Files_page.dart

  File yang berisi tampilan utama, logika, serta fungsi dari fitur File transfer via SFTP.

```
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:flutter1/styles/app_color.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../utils/ssh_connection.dart';
import 'dir_screen.dart';

List<Map<String, String>> hostServers = [];

class FileTransferPage extends StatefulWidget {
  const FileTransferPage({super.key});

  @override
  _FileTransferPageState createState() => _FileTransferPageState();
}

class _FileTransferPageState extends State<FileTransferPage>
    with SingleTickerProviderStateMixin {
  late TabController _tabController;
  @override
  void initState() {
    super.initState();
    _tabController = TabController(length: 2, vsync: this);
    _loadHostsFromPreferences();
  }

  @override
  void dispose() {
    _tabController.dispose();
    super.dispose();
  }

  Future<void> _loadHostsFromPreferences() async {
    SharedPreferences prefs = await SharedPreferences.getInstance();
    List<String>? savedHosts = prefs.getStringList('hostServers');
    if (savedHosts != null) {
      setState(() {
        hostServers = savedHosts
            .map((hostJson) => Map<String, String>.from(jsonDecode(hostJson)))
            .toList();
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: [
          const SizedBox(height: 50),
          Container(
```

```
            color: Colors.black,
            child: TabBar(
              controller: _tabController,
              indicatorColor: Colors.white,
              labelColor: Colors.white,
              unselectedLabelColor: Colors.white70,
              tabs: const [
                Tab(text: 'Host 1'),
                Tab(text: 'Host 2'),
              ],
            ),
          ),
          Expanded(
            child: TabBarView(
              controller: _tabController,
              children: [
                HostPage(tabController: _tabController),
                HostPage(tabController: _tabController),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

class HostPage extends StatefulWidget {
  final TabController tabController;

  const HostPage({super.key, required this.tabController});

  @override
  _HostPageState createState() => _HostPageState();
}

class _HostPageState extends State<HostPage> {
  late List<Map<String, String>> servers;
  SSHConnection? _sshConnection; // Ubah dari late menjadi nullable
  bool _showDirScreen = false;

  @override
  void initState() {
    super.initState();
    servers = hostServers;
    _loadHostsFromPreferences();
  }

  Future<void> refreshHosts() async {
    await _loadHostsFromPreferences();
    setState(() {
      servers = hostServers;
    });
  }

  Future<void> _loadHostsFromPreferences() async {
    SharedPreferences prefs = await SharedPreferences.getInstance();
    List<String>? savedHosts = prefs.getStringList('hostServers');

    if (savedHosts != null) {
      setState(() {
        hostServers = savedHosts
            .map((hostJson) => Map<String, String>.from(jsonDecode(hostJson)))
            .toList();
      });
    }
```

```
}

Future<void> _removeHost(int index) async {
  setState(() {
    hostServers.removeAt(index);
  });

  SharedPreferences prefs = await SharedPreferences.getInstance();
  List<String> updatedHostList =
      hostServers.map((host) => jsonEncode(host)).toList();
  await prefs.setStringList('hostServers', updatedHostList);
}

@override
Widget build(BuildContext context) {
  // Tampilkan DirScreen jika _showDirScreen true, atau daftar server jika false
  return _showDirScreen && _sshConnection != null
      ? DirScreen(
          sshConnection: _sshConnection!,
          onClose: () {
            setState(() {
              _showDirScreen = false;
            });
            _sshConnection?.close(); // Tutup koneksi jika tidak null
          },
        )
      : Scaffold(
          body: Padding(
            padding: const EdgeInsets.all(16.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const SizedBox(height: 20),
                const Text(
                  "Pilih server",
                  style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                ),
                const SizedBox(height: 20),
                Expanded(
                  child: RefreshIndicator(
                    onRefresh: refreshHosts,
                    child: ListView.builder(
                      itemCount: servers.isEmpty ? 1 : servers.length,
                      itemBuilder: (context, index) {
                        if (servers.isEmpty) {
                          return const Center(
                              child: Text("Belum ada server"));
                        } else {
                          return _buildHostTile(servers[index], index);
                        }
                      },
                    ),
                  ),
                ),
              ],
            ),
          ),
          floatingActionButton: _buildAddNewHost(context),
        );
}

Widget _buildAddNewHost(BuildContext context) {
  return FloatingActionButton(
    onPressed: () {
      addHostBottomSheet(context, refreshHosts);
    },
```

```
    backgroundColor: Colors.green,
    child: const Icon(Icons.add, size: 40),
  );
}

Widget _buildHostTile(Map<String, String> server, int index) {
  return Card(
    margin: const EdgeInsets.only(bottom: 16),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    child: ListTile(
      leading: const Icon(Icons.cloud, color: Colors.blue),
      title: Text(server['name'] ?? 'Unknown'),
      subtitle: Text('${server['username']}, ${server['hostname']}'),
      onLongPress: () {
        modifyHostBottomSheet(context, server, index, refreshHosts);
      },
      onTap: () async {
        String host = server['hostname'] ?? '';
        String username = server['username'] ?? '';
        String password = server['password'] ?? '';

        _sshConnection = SSHConnection(); // Inisialisasi SSH Connection

        try {
          await _sshConnection!.connect(host, username, password);
          if (_sshConnection!.isConnected) {
            print("Successfully connected to $host via SFTP");

            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('Connected to $host via SFTP')),
            );

            // Tampilkan DirScreen dengan koneksi SSH yang terhubung
            setState(() {
              _showDirScreen = true;
            });
          } else {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('Failed to connect to $host')),
            );
          }
        } catch (e) {
          print("Error: $e");
          ScaffoldMessenger.of(context)
              .showSnackBar(SnackBar(content: Text('Error: $e')));
        }
      },
    ),
  );
}

Widget buildHostList(BuildContext context) {
  return FutureBuilder(
    future: _loadHostsFromPreferences(), // Memuat data awal
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        // Menampilkan loading spinner saat data sedang dimuat
        return const Center(child: CircularProgressIndicator());
      }

      if (snapshot.hasError) {
        return Center(child: Text('Error: ${snapshot.error}'));
      }

      // Memastikan data sudah tersedia
      if (!snapshot.hasData || hostServers.isEmpty) {
```

```
          return const Center(child: Text("Belum ada host tersedia"));
        }

        return ListView.builder(
          itemCount: hostServers.length,
          itemBuilder: (context, index) {
            final host = hostServers[index];
            return Card(
              margin: const EdgeInsets.symmetric(vertical: 8, horizontal: 16),
              shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12)),
              child: ListTile(
                leading: const Icon(Icons.cloud, color: Colors.blue),
                title: Text(host['name'] ?? 'Unknown'),
                subtitle: Text('${host['username']}, ${host['hostname']}'),
                onTap: () => showDialog(
                  context: context,
                  builder: (BuildContext context) {
                    return AlertDialog(
                      title: const Text("Detail Host"),
                      content: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          Text("Name: ${host['name']}"),
                          Text("Hostname: ${host['hostname']}"),
                          Text("Port: ${host['port']}"),
                          Text("Username: ${host['username']}"),
                          Text("Password: ${host['password']}"),
                        ],
                      ),
                      actions: [
                        TextButton(
                            onPressed: () => Navigator.pop(context),
                            child: const Text("Tutup")),
                      ],
                    );
                  },
                ),
                onLongPress: () {
                  modifyHostBottomSheet(context, host, index, refreshHosts);
                },
              ),
            );
          },
        );
      },
    );
  }
}

// Fungsi untuk menyimpan host ke SharedPreferences
Future<void> _saveHostsToPreferences() async {
  SharedPreferences prefs = await SharedPreferences.getInstance();
  List<String> hostsJsonList =
      hostServers.map((host) => jsonEncode(host)).toList();
  await prefs.setStringList('hostServers', hostsJsonList);
}

// Fungsi untuk memuat host dari SharedPreferences
Future<void> _loadHostsFromPreferences() async {
  SharedPreferences prefs = await SharedPreferences.getInstance();
  List<String>? savedHosts = prefs.getStringList('hostServers');
  if (savedHosts != null) {
    hostServers = savedHosts
        .map((hostJson) {
```

```
          try {
            final decoded = jsonDecode(hostJson);
            return decoded is Map<String, String> ? decoded : null;
          } catch (e) {
            print("Error decoding host: $e");
            return null;
          }
        })
        .whereType<Map<String, String>>()
        .toList();
  }
}

// Fungsi untuk menampilkan bottom sheet untuk menambahkan host baru
// Fungsi untuk menampilkan bottom sheet untuk menambahkan host baru
void addHostBottomSheet(BuildContext context, Function refreshHosts) {
  String name = '';
  String hostname = '';
  String port = '';
  String username = '';
  String password = '';

  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    builder: (BuildContext context) {
      return Padding(
        padding:
            EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
        child: Container(
          padding: const EdgeInsets.all(16),
          decoration: const BoxDecoration(
            color: AppColors
                .darkGray, // Sesuaikan dengan latar belakang terminal_page
            borderRadius: BorderRadius.vertical(top: Radius.circular(25)),
          ),
          child: StatefulBuilder(
            builder: (context, setState) {
              return SingleChildScrollView(
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Row(
                      mainAxisAlignment: MainAxisAlignment.spaceBetween,
                      children: [
                        const Text(
                          "Add Host",
                          style: TextStyle(
                            fontSize: 20,
                            color: AppColors
                                .white, // Sama dengan desain terminal_page
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                        GestureDetector(
                          onTap: () {
                            Navigator.pop(context); // Menutup modal jika Cancel
                          },
                          child: const Text(
                            'Cancel',
                            style: TextStyle(
                              fontFamily: 'Poppins',
                              fontWeight: FontWeight.normal,
                              color: AppColors
                                  .green, // Sama dengan desain terminal_page
```

```
        ),
      ),
    ),
  ],
),
const SizedBox(height: 20),
// Kolom untuk Name
_buildTextField(
  label: 'Name',
  onChanged: (value) => name = value,
  initialValue: name,
),
const SizedBox(height: 12),
// Kolom untuk Hostname/IP Address
_buildTextField(
  label: 'Hostname / IP Address',
  onChanged: (value) => hostname = value,
  initialValue: hostname,
),
const SizedBox(height: 12),
// Kolom untuk Port
_buildTextField(
  label: 'Port',
  onChanged: (value) => port = value,
  initialValue: port,
),
const SizedBox(height: 12),
// Kolom untuk Username
_buildTextField(
  label: 'Username',
  onChanged: (value) => username = value,
  initialValue: username,
),
const SizedBox(height: 12),
// Kolom untuk Password
PasswordField(
  onChanged: (value) => password = value,
  initialValue: password,
),
const SizedBox(height: 20),
// Tombol untuk menyimpan host
ElevatedButton(
  onPressed: () async {
    final newHost = {
      "name": name,
      "hostname": hostname,
      "port": port,
      "username": username,
      "password": password,
    };

    // Menambahkan host baru ke dalam list
    setState(() {
      hostServers.add(newHost);
    });

    // Menyimpan data ke SharedPreferences
    await _saveHostsToPreferences();

    // Memanggil fungsi untuk refresh data
    await refreshHosts();

    // Menutup bottom sheet
    Navigator.pop(context);
  },
  style: ElevatedButton.styleFrom(
```

```
                        minimumSize: const Size(double.infinity, 50),
                        backgroundColor: const Color(
                            0xFF343746), // Sesuaikan dengan desain terminal_page
                      ),
                      child: const Text(
                        "Continue",
                        style: TextStyle(
                          fontSize: 18,
                          color: Colors.white,
                        ),
                      ),
                    ),
                  ],
                ),
              );
            },
          ),
        ),
      );
    },
  );
}

void modifyHostBottomSheet(BuildContext context, Map<String, String> host,
    int index, Function refreshHosts) {
  // Cek apakah hostServers memiliki index yang valid
  if (index < 0 || index >= hostServers.length) {
    // Keluarkan pesan error atau return jika index tidak valid
    print('Index is out of range for hostServers');
    return;
  }

  String name = host['name'] ?? '';
  String hostname = host['hostname'] ?? '';
  String port = host['port'] ?? '';
  String username = host['username'] ?? '';
  String password = host['password'] ?? '';

  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    builder: (BuildContext context) {
      return Padding(
        padding:
            EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
        child: Container(
          padding: const EdgeInsets.all(16),
          decoration: const BoxDecoration(
            color: Color(0xFF15181F),
            borderRadius: BorderRadius.vertical(top: Radius.circular(25)),
          ),
          child: StatefulBuilder(
            builder: (context, setState) {
              return SingleChildScrollView(
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Row(
                      mainAxisAlignment: MainAxisAlignment.spaceBetween,
                      children: [
                        const Text(
                          "Edit Host",
                          style: TextStyle(fontSize: 16, color: Colors.white),
                        ),
                        Row(
```

```
      children: [
        GestureDetector(
          onTap: () async {
            setState(() {
              hostServers.removeAt(index); // Menghapus host
            });
            await _saveHostsToPreferences();
            await refreshHosts(); // Memanggil refreshHosts
            Navigator.pop(context); // Menutup bottom sheet
          },
          child: const Text(
            "Hapus",
            style:
                TextStyle(fontSize: 16, color: Colors.red),
          ),
        ),
        const SizedBox(width: 8),
        GestureDetector(
          onTap: () => Navigator.pop(
              context), // Menutup bottom sheet
          child: const Text(
            "Batal",
            style: TextStyle(
                fontSize: 16, color: Colors.green),
          ),
        ),
      ],
    ),
  ],
),
const SizedBox(height: 20),
_buildTextField(
    label: 'Name',
    onChanged: (value) => name = value,
    initialValue: name),
const SizedBox(height: 10),
_buildTextField(
    label: 'Hostname / IP Address',
    onChanged: (value) => hostname = value,
    initialValue: hostname),
const SizedBox(height: 10),
_buildTextField(
    label: 'Port',
    onChanged: (value) => port = value,
    initialValue: port),
const SizedBox(height: 10),
_buildTextField(
    label: 'Username',
    onChanged: (value) => username = value,
    initialValue: username),
const SizedBox(height: 12),
PasswordField(
    onChanged: (value) => password = value,
    initialValue: password),
const SizedBox(height: 20),
ElevatedButton(
  onPressed: () async {
    final updatedHost = {
      "name": name,
      "hostname": hostname,
      "port": port,
      "username": username,
      "password": password,
    };
    setState(() {
      hostServers[index] = updatedHost; // Update host data
```

```
                        });
                        await _saveHostsToPreferences();
                        await refreshHosts(); // Memanggil refreshHosts untuk update
UI
                        Navigator.pop(context); // Menutup bottom sheet
                      },
                      style: ElevatedButton.styleFrom(
                        minimumSize: const Size(double.infinity, 50),
                        backgroundColor: const Color(0xFF343746),
                      ),
                      child: const Text("Save",
                          style: TextStyle(fontSize: 18, color: Colors.white)),
                    ),
                  ],
                ),
              );
            },
          ),
        ),
      );
    },
  );
}

// Fungsi untuk membuat text field dengan nilai awal
Widget _buildTextField({
  required String label,
  required Function(String) onChanged,
  required String initialValue,
}) {
  return TextField(
    controller: TextEditingController(text: initialValue),
    decoration: InputDecoration(
      labelText: label,
      labelStyle: const TextStyle(
        fontFamily: 'Poppins',
        fontWeight: FontWeight.w500,
        color: Color(0xFF8C91A5), // Warna label
      ),
      filled: true,
      fillColor: const Color(0xFF15181F), // Warna latar belakang
      enabledBorder: const UnderlineInputBorder(
        borderSide: BorderSide(
            color: Color(0xFF242834)), // Garis bawah saat tidak fokus
      ),
      focusedBorder: const UnderlineInputBorder(
        borderSide:
            BorderSide(color: Color(0xFF242834)), // Garis bawah saat fokus
      ),
    ),
    style: const TextStyle(color: Colors.white), // Warna teks
    onChanged: onChanged, // Fungsi untuk menangani perubahan input
  );
}

class PasswordField extends StatefulWidget {
  final Function(String) onChanged;
  final String initialValue; // Menambahkan parameter initialValue

  const PasswordField({
    super.key,
    required this.onChanged,
    this.initialValue =
        '', // Default menjadi string kosong jika tidak diberikan
  });
```

```
  @override
  _PasswordFieldState createState() => _PasswordFieldState();
}

class _PasswordFieldState extends State<PasswordField> {
  bool _isObscured = true;
  late TextEditingController _controller;

  @override
  void initState() {
    super.initState();
    _controller = TextEditingController(
        text:
            widget.initialValue); // Inisialisasi controller dengan initialValue
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return TextField(
      controller: _controller, // Menggunakan controller di sini
      obscureText: _isObscured,
      onChanged: widget.onChanged,
      style: const TextStyle(color: Colors.white),
      decoration: InputDecoration(
        labelText: 'Password',
        suffixIcon: IconButton(
          icon: Icon(
            _isObscured ? Icons.visibility : Icons.visibility_off,
            color: Colors.white,
          ),
          onPressed: () {
            setState(() {
              _isObscured = !_isObscured;
            });
          },
        ),
        filled: true,
        fillColor: const Color(0xFF15181F),
        // Mengganti OutlineInputBorder dengan UnderlineInputBorder
        enabledBorder: const UnderlineInputBorder(
          borderSide: BorderSide(
              color: Color(0xFF242834)), // Garis bawah saat tidak fokus
        ),
        focusedBorder: const UnderlineInputBorder(
          borderSide:
              BorderSide(color: Color(0xFF242834)), // Garis bawah saat fokus
        ),
      ),
    );
  }
}
```

- Details_screen.dart

File ini berisi informasi detail yang ditangkap pada fitur sniffing. Ketika salah satu informasi sniffing ditekan, maka akan berpindah dan menampilkan detail informasi yang sesuai dari packet sniffing tersebut di details_screen.

```
import 'package:flutter/material.dart';
import 'package:flutter_network_inspector/models/inspector_result.dart';
```

```
class DetailsScreen extends StatelessWidget {
  const DetailsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // Mengambil data InspectorResult dari argument di ModalRoute
    final data = ModalRoute.of(context)!.settings.arguments as InspectorResult;
    final success =
        (data.statusCode ?? 0) >= 200 && (data.statusCode ?? 0) < 300;

    return Scaffold(
      appBar: AppBar(
        elevation: 3,
        centerTitle: true,
        backgroundColor: const Color(0xFF14172A), // Background warna #14172A
        title: const Text(
          'Details',
          style: TextStyle(
            color: Colors.white, // Mengubah warna teks menjadi putih #FFFFFF
          ),
        ),
      ),
      body: SafeArea(
        child: ListView(
          padding: const EdgeInsets.fromLTRB(20, 5, 20, 20),
          children: [
            _buildRow('Start Time', formatDateTime(data.startTime)),
            _buildRow('Status Code', data.statusCode?.toString() ?? '-'),
            _buildRow('Protocol', data.url?.scheme.toUpperCase() ?? '-'),
            _buildRow('Host', data.url?.host ?? '-'),
            _buildRow('Path', data.url?.path ?? '-'),
            _buildRow(
              'Response Size',
              '${(data.responseBodyBytes ?? 0) / 1024} KB',
            ),
            _buildRow(
              success ? 'Reason Phrase' : 'Error Message',
              data.reasonPhrase ?? '-',
            ),
          ],
        ),
      ),
    );
  }

  Widget _buildRow(String title, String value) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 4),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text(title, style: const TextStyle(fontWeight: FontWeight.bold)),
          Text(value),
        ],
      ),
    );
  }
}

String formatDateTime(DateTime? dateTime) {
  if (dateTime == null) return '-';
  return '${dateTime.hour}:${dateTime.minute}:${dateTime.second}';
}
```

- Dir_screen.dart

  File ini berisi tampilan directory screen, pada fitur transfer file via sftp. Ketika sftp berhasil, maka aplikasi akan berpindah dan menampilkan directory screen sesuai tampilan directory pada server sftp tujuan.

```
import 'dart:io'; // Import dart:io untuk File
import 'package:flutter/material.dart';
import 'package:dartssh2/dartssh2.dart'; // Paket SFTP
import 'package:path/path.dart' as path; // Untuk mendapatkan ekstensi file
import 'package:file_picker/file_picker.dart'; // Untuk memilih file
import '../utils/ssh_connection.dart'; // Koneksi SSH

class DirScreen extends StatefulWidget {
  final SSHConnection sshConnection;
  final VoidCallback onClose;

  const DirScreen(
      {super.key, required this.sshConnection, required this.onClose});

  @override
  _DirScreenState createState() => _DirScreenState();
}

class _DirScreenState extends State<DirScreen> {
  List<SftpName> _directoryItems = [];
  String _currentPath = "/";
  List<String> _pathStack = [];
  bool _isLoading = true;

  List<String> _selectedFoldersToDelete = [];
  bool _isDeleteMode = false;

  @override
  void initState() {
    super.initState();
    _loadDirectory(_currentPath);
  }

  Future<void> _loadDirectory(String path) async {
    setState(() {
      _isLoading = true;
    });
    try {
      List<SftpName> items = await widget.sshConnection.listDirectory(path);
      setState(() {
        _directoryItems = items;
        _currentPath = path;
        _isLoading = false;
      });
    } catch (e) {
      setState(() {
        _isLoading = false;
      });
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Failed to load directory: $e')),
      );
    }
  }

  Future<void> _goBack() async {
    if (_pathStack.isNotEmpty) {
      String previousPath = _pathStack.removeLast();
      await _loadDirectory(previousPath);
    } else {
```

```
    widget.onClose();
  }
}

// Fungsi untuk menampilkan SnackBar dengan durasi 1 detik
void showCustomSnackBar(String message) {
  ScaffoldMessenger.of(context).clearSnackBars();
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(message),
      duration: const Duration(seconds: 1),
    ),
  );
}

Future<void> _uploadFile() async {
  FilePickerResult? result = await FilePicker.platform.pickFiles();

  if (result != null) {
    PlatformFile file = result.files.first;

    showCustomSnackBar('Uploading file...'); // Menggunakan showCustomSnackBar

    try {
      File fileToUpload = File(file.path!);

      await widget.sshConnection.uploadFile(
        '$_currentPath/${file.name}',
        fileToUpload,
      );

      showCustomSnackBar('File uploaded successfully');
      await _loadDirectory(_currentPath);
    } catch (e) {
      showCustomSnackBar('Failed to upload file: $e');
    }
  } else {
    showCustomSnackBar('No file selected');
  }
}

Future<void> _deleteSelectedFolders() async {
  try {
    for (String folderPath in _selectedFoldersToDelete) {
      try {
        final items = await widget.sshConnection.listDirectory(folderPath);
        if (items.isEmpty) {
          await widget.sshConnection.removeDirectory(folderPath);
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(
                content: Text('Successfully deleted folder: $folderPath')),
          );
        } else {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('Folder is not empty: $folderPath')),
          );
        }
      } catch (e) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Folder not found: $folderPath - $e')),
        );
      }
    }

    setState(() {
      _selectedFoldersToDelete.clear();
```

```
      _isDeleteMode = false;
    });

    await _loadDirectory(_currentPath);
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error: $e')),
    );
  }
}

Future<void> _createDirectory() async {
  TextEditingController controller = TextEditingController();

  showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: const Text("Create New Directory"),
        content: TextField(
          controller: controller,
          decoration: const InputDecoration(hintText: "Enter directory name"),
        ),
        actions: [
          TextButton(
            onPressed: () async {
              String dirName = controller.text;
              String newDirPath = '$_currentPath/$dirName';

              try {
                await widget.sshConnection.createDirectory(newDirPath);
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(
                      content: Text('Directory created successfully')),
                );
                await _loadDirectory(_currentPath);
              } catch (e) {
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text('Failed to create directory: $e')),
                );
              }
              Navigator.pop(context);
            },
            child: const Text("Create"),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: const Text("Cancel"),
          ),
        ],
      );
    },
  );
}

Widget _buildDeleteCheckbox(SftpName item) {
  return Checkbox(
    value: _selectedFoldersToDelete.contains(item.filename),
    onChanged: (bool? selected) {
      setState(() {
        if (selected == true) {
          _selectedFoldersToDelete.add(item.filename);
        } else {
          _selectedFoldersToDelete.remove(item.filename);
        }
      });
```

```
      },
    );
  }

  Future<void> _showProperties(SftpName item) async {
    final filePath = '$_currentPath/${item.filename}';
    final fileAttributes = item.attr;

    showDialog(
      context: context,
      builder: (context) {
        return AlertDialog(
          title: Text(item.filename),
          content: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            mainAxisSize: MainAxisSize.min,
            children: [
              Text("Path: $filePath"),
              Text("Type: ${fileAttributes.isDirectory ? 'Folder' : 'File'}"),
              if (!fileAttributes.isDirectory)
                Text("Size: ${fileAttributes.size ?? 'Unknown'} bytes"),
              if (fileAttributes.modifyTime != null)
                Text(
                                                                "Last      Modified:
${DateTime.fromMillisecondsSinceEpoch(fileAttributes.modifyTime! * 1000)}",
                ),
            ],
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text("Close"),
            ),
          ],
        );
      },
    );
  }

// New function to handle file reading and editing
  Future<void> _readFile(SftpName item) async {
    final filePath = '$_currentPath/${item.filename}';
    final fileExtension =
        path.extension(filePath).toLowerCase(); // Get file extension

    // Check if the file is a .txt file
    if (fileExtension != '.txt') {
      showCustomSnackBar('Only .txt files can be opened.');
      return; // Exit the function if the file isn't a .txt file
    }

    try {
      // Read file content using your readFile function
      String content = await widget.sshConnection.readFile(filePath);

      // Controller to manage the TextField for editing
      TextEditingController controller = TextEditingController(text: content);

      showDialog(
        context: context,
        builder: (context) {
          return AlertDialog(
            title: Text(item.filename),
            content: Container(
              width: MediaQuery.of(context).size.width *
                  0.9, // 90% of screen width
```

```
            child: SingleChildScrollView(
              child: TextField(
                controller: controller,
                maxLines:
                    20, // Restrict the height of the TextField by limiting lines
                style: const TextStyle(fontFamily: 'monospace'),
                decoration:
                    const InputDecoration(hintText: "Edit file content"),
              ),
            ),
          ),
          actions: [
            TextButton(
              onPressed: () async {
                // When the user saves the file
                String updatedContent = controller.text;

                try {
                  // Write the updated content to the file
                  await widget.sshConnection
                      .writeFile(filePath, updatedContent);

                  ScaffoldMessenger.of(context).showSnackBar(
                    const SnackBar(
                        content: Text('File updated successfully')),
                  );
                  Navigator.pop(context);
                } catch (e) {
                  ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(content: Text('Failed to update file: $e')),
                  );
                }
              },
              child: const Text(""),
            ),
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text("Close"),
            ),
          ],
        );
      },
    );
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Failed to read file: $e')),
    );
  }
}

@override
Widget build(BuildContext context) {
  return WillPopScope(
    onWillPop: () async {
      if (_pathStack.isNotEmpty) {
        await _goBack();
        return false;
      }
      return true;
    },
    child: Scaffold(
      backgroundColor: Colors.black87,
      appBar: AppBar(
        backgroundColor: Colors.black87,
        title: Text(_currentPath),
        leading: _pathStack.isNotEmpty
```

```
        ? IconButton(
            icon: const Icon(Icons.arrow_back),
            onPressed: _goBack,
          )
        : null,
    actions: [
      IconButton(
        icon: const Icon(Icons.close),
        onPressed: widget.onClose,
      ),
      IconButton(
        icon: const Icon(Icons.upload_file),
        onPressed: _uploadFile,
      ),
      IconButton(
        icon: const Icon(Icons.create_new_folder),
        onPressed: _createDirectory,
      ),
      // IconButton(
      //   icon: Icon(
      //     _isDeleteMode ? Icons.delete_forever : Icons.delete,
      //     color: _isDeleteMode ? Colors.red : null,
      //   ),
      //   onPressed: () {
      //     setState(() {
      //       if (_isDeleteMode) {
      //         if (_selectedFoldersToDelete.isNotEmpty) {
      //           _deleteSelectedFolders();
      //         } else {
      //           _isDeleteMode = false;
      //         }
      //       } else {
      //         _isDeleteMode = true;
      //       }
      //     });
      //   },
      // ),
    ],
  ),
  body: _isLoading
      ? const Center(child: CircularProgressIndicator())
      : _directoryItems.isEmpty
          ? const Center(
              child: Text(
                "Empty directory",
                style: TextStyle(color: Colors.white, fontSize: 16),
              ),
            )
          : RefreshIndicator(
              onRefresh: () async {
                await _loadDirectory(_currentPath);
              },
              child: ListView.builder(
                itemCount: _directoryItems.length,
                itemBuilder: (context, index) {
                  final item = _directoryItems[index];
                  final isDirectory = item.attr.isDirectory;

                  return ListTile(
                    leading: Icon(
                      isDirectory
                          ? Icons.folder
                          : Icons.insert_drive_file,
                      color: isDirectory ? Colors.yellow : Colors.grey,
                    ),
                    title: Text(
```

```
                            item.filename,
                            style: const TextStyle(color: Colors.white),
                          ),
                          onTap: () {
                            if (isDirectory) {
                              _pathStack.add(_currentPath);
                              _loadDirectory('$_currentPath/${item.filename}');
                            } else {
                              // Read the file if it's not a directory
                              _readFile(item);
                            }
                          },
                          onLongPress: () => _showProperties(item),
                        );
                      },
                    ),
                  ),
        ),
      );
    }
  }
```

- Location_vpn.dart

  File ini berisi implementasi halaman LocationVpn untuk mendukung fitur VPN dengan menampilkan daftar lokasi VPN, menangani proses pemuatan data VPN, dan navigasi ke halaman pengaturan koneksi VPN menggunakan controller dari vpn_control.

```
import 'package:flutter/material.dart';
import 'package:flutter1/APIS/vpn_control.dart';
import 'package:flutter1/pages/vpn_page.dart';
import 'package:get/get.dart';
import 'package:lottie/lottie.dart';
import 'vpn_card.dart';

class LocationVpn extends StatefulWidget {
  const LocationVpn({super.key});

  @override
  State<LocationVpn> createState() => _LocationVpnState();
}

class _LocationVpnState extends State<LocationVpn> {
  final _controller = Get.put(Vpncontrol());

  @override
  void initState() {
    _controller.toogle.value = false;
    super.initState();
    // Menunda pemanggilan getVPNdata sampai frame selesai
    WidgetsBinding.instance.addPostFrameCallback((_) {
      _controller.getVPNdata();
    });
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
```

```
  return Scaffold(
    appBar: AppBar(
      centerTitle: true,
      title: Text("VPN LOCATION"),
      actions: [
        IconButton(
          padding: EdgeInsets.only(right: 20),
          onPressed: () {
            // Panggil getVPNdata hanya saat refresh ditekan
            _controller.getVPNdata();
          },
          icon: Icon(
            Icons.refresh_rounded,
            size: 40,
          ),
        )
      ],
    ),
    body: Obx(() {
      if (_controller.isLoading.value) {
        return _loadingWidget();
      } else if (_controller.isFound.isNotEmpty) {
        return _noVpnFount();
      } else {
        return _vpnData();
      }
    }),
  );
}

_vpnData() => Container(
      child: ListView.builder(
        itemCount: _controller.vpnlist.length,
        itemBuilder: (ctx, i) => Padding(
          padding: const EdgeInsets.all(1.0),
          child: GestureDetector(
            onTap: () {
              _controller.isConnecting.value ^= true;
              // Setelah itu, navigasi ke VPNPage
              Navigator.push(
                ctx,
                MaterialPageRoute(
                  builder: (_) => VPNPage(), // Pindah ke halaman VPNPage
                ),
              );
            },
            child: VpnCard(vpn: _controller.vpnlist[i]),
          ),
        ),
      ),
    );

_loadingWidget() => SizedBox(
      width: double.infinity,
      height: double.infinity,
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          LottieBuilder.asset(
            'assets/loading/loadingAnimation2.json',
            width: MediaQuery.of(context).size.width * 7,
          ),
          Text(
            'LOADING....',
            style: TextStyle(fontSize: 22),
          ),
```

```
          ],
        ),
      );

  _noVpnFount() => Center(
        child: Text(
          'NO VPN FOUND',
          style: TextStyle(fontSize: 30),
        ),
      );
}
```

- Timer.dart

  File ini berisi widget yang menampilkan timer berjalan untuk menghitung durasi dalam format HH:mm:ss dan memperbarui waktu setiap detik sesuai pengaturan awal.

```
import 'package:flutter/material.dart';
import 'dart:async';

class TimerWidget extends StatefulWidget {
  final bool startTimer;

  const TimerWidget({super.key, required this.startTimer});

  @override
  State<TimerWidget> createState() => _TimerWidgetState();
}

class _TimerWidgetState extends State<TimerWidget> {
  Duration _duration = Duration();
  Timer? _timer;

  /// Memulai timer untuk memperbarui durasi setiap detik
  void _startTimer() {
    _timer = Timer.periodic(Duration(seconds: 1), (timer) {
      setState(() {
        _duration += Duration(seconds: 1);
      });
    });
  }

  @override
  void initState() {
    super.initState();
    if (widget.startTimer) {
      _startTimer();
    }
  }

  @override
  void dispose() {
    _timer?.cancel(); // Menghentikan timer saat widget dihapus
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    String formattedTime = _formatDuration(_duration);

    return Text(
      formattedTime,
      style: TextStyle(fontSize: 30),
    );
```

```
  }

  /// Format durasi menjadi `HH:mm:ss`
  String _formatDuration(Duration duration) {
    String twoDigits(int n) => n.toString().padLeft(2, '0');
    String hours = twoDigits(duration.inHours);
    String minutes = twoDigits(duration.inMinutes.remainder(60));
    String seconds = twoDigits(duration.inSeconds.remainder(60));
    return '$hours:$minutes:$seconds';
  }
}
```

- Vpn_card.dart

File ini berisi desain kartu yang menampilkan detail lokasi VPN, seperti nama negara, kecepatan, dan jumlah sesi pengguna, untuk mendukung daftar lokasi VPN.

```
import 'package:flutter/material.dart';
import 'package:flutter1/APIS/vpn.dart';
import 'package:flutter1/styles/app_color.dart';

class VpnCard extends StatelessWidget {
  final Vpn vpn;
  const VpnCard({super.key, required this.vpn});

  @override
  Widget build(BuildContext context) {
    return Card(
        color: AppColors.gray,
        elevation: 0,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(13)),
        child: ListTile(
          shape:
              RoundedRectangleBorder(borderRadius: BorderRadius.circular(13)),
          leading:
              Image.asset('assets/flags/${vpn.countryShort.toLowerCase()}.png'),
          title: Text(
            '${vpn.countryLong} (${vpn.countryShort})',
            style: TextStyle(fontSize: 17),
          ),
          subtitle: Row(
            children: [
              Icon(
                Icons.speed_rounded,
                size: 17,
              ),
              SizedBox(
                width: 5,
              ),
              Text(vpn.speed.toString()),
            ],
          ),
          trailing: Row(
            mainAxisSize: MainAxisSize.min,
            children: [
              Text(
                vpn.numVpnSessions.toString(),
                style: TextStyle(fontSize: 17),
              ),
              SizedBox(
```

```
                width: 4,
              ),
              Icon(
                Icons.people_alt_rounded,
                size: 17,
              ),
            ],
          ),
        ));
  }
}
```

- Utils

  Folder ini berisi utilitas yang menyediakan fungsi pendukung seperti pemeriksaan jaringan, sniffing HTTP, dan koneksi SSH untuk mendukung fitur aplikasi.

➢ network_check.dart

File ini berisi fungsi untuk memeriksa konektivitas perangkat ke jaringan dan akses internet, serta memantau statusnya secara real-time.

```
import 'dart:async';
import 'package:connectivity_plus/connectivity_plus.dart';
import 'package:http/http.dart' as http;

class NetworkCheck {
  // Mengecek apakah perangkat terhubung ke jaringan (WiFi atau data seluler)
  static Future<bool> isNetworkAvailable() async {
    final connectivityResult = await Connectivity().checkConnectivity();
    return connectivityResult != ConnectivityResult.none;
  }

  // Mengecek apakah perangkat memiliki akses internet dengan ping ke server Google
  static Future<bool> isInternetAvailable() async {
    try {
      final response = await http
          .get(Uri.parse('https://www.google.com'))
          .timeout(const Duration(seconds: 5));
      return response.statusCode == 200;
    } catch (_) {
      return false;
    }
  }

  // Memantau perubahan status jaringan dan internet secara real-time
  static Stream<bool> get internetStatusStream async* {
    yield await isInternetAvailable();
    // ignore: unused_local_variable
    await for (var status in Connectivity().onConnectivityChanged) {
      yield await isInternetAvailable();
    }
  }
}
```

➢ sniffing_helper.dart

File ini mendukung fitur sniffing dengan mencatat detail permintaan HTTP (GET, POST, PUT, PATCH, DELETE) dan menyimpan hasilnya dalam bentuk log yang dapat dimonitor.

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_network_inspector/models/inspector_result.dart';
import 'package:http/http.dart' as http;

class FNICLient {
  static final ValueNotifier<List<InspectorResult>> inspectorNotifierList =
      ValueNotifier([]);

  Future<http.Response> get(Uri url, {Map<String, String>? headers}) async {
    InspectorResult result =
        InspectorResult(url: url, startTime: DateTime.now());

    // Buat salinan sementara dan tambahkan `result`
    final updatedList = List<InspectorResult>.from(inspectorNotifierList.value);
    updatedList.add(result);
    inspectorNotifierList.value = updatedList; // Memicu notifyListeners

    final response = await http.get(url, headers: headers);
    result.endTime = DateTime.now();
    result.statusCode = response.statusCode;
    result.reasonPhrase = response.reasonPhrase;
    result.responseBodyBytes = response.bodyBytes.length;

    // Memperbarui kembali value untuk memicu notifyListeners
    inspectorNotifierList.value =
        List<InspectorResult>.from(inspectorNotifierList.value);

    return response;
  }

  Future<http.Response> post(Uri url,
      {Map<String, String>? headers, Object? body}) async {
    InspectorResult result =
        InspectorResult(url: url, startTime: DateTime.now());

    final updatedList = List<InspectorResult>.from(inspectorNotifierList.value);
    updatedList.add(result);
    inspectorNotifierList.value = updatedList; // Memicu notifyListeners

    final response = await http.post(url, headers: headers, body: body);
    result.endTime = DateTime.now();
    result.statusCode = response.statusCode;
    result.reasonPhrase = response.reasonPhrase;
    result.responseBodyBytes = response.bodyBytes.length;

    inspectorNotifierList.value =
        List<InspectorResult>.from(inspectorNotifierList.value);

    return response;
  }

  Future<http.Response> put(Uri url,
      {Map<String, String>? headers, Object? body}) async {
    InspectorResult result =
        InspectorResult(url: url, startTime: DateTime.now());
```

```
  final updatedList = List<InspectorResult>.from(inspectorNotifierList.value);
  updatedList.add(result);
  inspectorNotifierList.value = updatedList; // Memicu notifyListeners

  final response = await http.put(url, headers: headers, body: body);
  result.endTime = DateTime.now();
  result.statusCode = response.statusCode;
  result.reasonPhrase = response.reasonPhrase;
  result.responseBodyBytes = response.bodyBytes.length;

  inspectorNotifierList.value =
      List<InspectorResult>.from(inspectorNotifierList.value);

  return response;
}

Future<http.Response> patch(Uri url,
    {Map<String, String>? headers, Object? body}) async {
  InspectorResult result =
      InspectorResult(url: url, startTime: DateTime.now());

  final updatedList = List<InspectorResult>.from(inspectorNotifierList.value);
  updatedList.add(result);
  inspectorNotifierList.value = updatedList; // Memicu notifyListeners

  final response = await http.patch(url, headers: headers, body: body);
  result.endTime = DateTime.now();
  result.statusCode = response.statusCode;
  result.reasonPhrase = response.reasonPhrase;
  result.responseBodyBytes = response.bodyBytes.length;

  inspectorNotifierList.value =
      List<InspectorResult>.from(inspectorNotifierList.value);

  return response;
}

Future<http.Response> delete(Uri url, {Map<String, String>? headers}) async {
  InspectorResult result =
      InspectorResult(url: url, startTime: DateTime.now());

  final updatedList = List<InspectorResult>.from(inspectorNotifierList.value);
  updatedList.add(result);
  inspectorNotifierList.value = updatedList; // Memicu notifyListeners

  final response = await http.delete(url, headers: headers);
  result.endTime = DateTime.now();
  result.statusCode = response.statusCode;
  result.reasonPhrase = response.reasonPhrase;
  result.responseBodyBytes = response.bodyBytes.length;

  inspectorNotifierList.value =
      List<InspectorResult>.from(inspectorNotifierList.value);

  return response;
}

void setEnableLogging(bool enable) {
  // Implementasi untuk mengatur logging
}
```

```
  void close() {
    // Implementasi untuk menutup koneksi jika diperlukan
  }
}
```

- ssh_connection.dart

  File ini menyediakan fungsi untuk koneksi SSH, menjalankan perintah terminal, dan transfer file menggunakan SFTP, termasuk operasi file dan direktori di server jarak jauh.

```
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:dartssh2/dartssh2.dart';

class SSHConnection {
  SSHClient? _client;
  bool _isConnected = false;

  // Menambahkan getter untuk memeriksa status koneksi
  bool get isConnected => _isConnected;

  // Fungsi untuk koneksi SSH
  Future<void> connect(String host, String username, String password) async {
    try {
      final socket = await SSHSocket.connect(host, 22);

      _client = SSHClient(
        socket,
        username: username,
        onPasswordRequest: () => password,
      );

      await _client!.authenticated;
      _isConnected = true;
      print("Connected to $host");
    } catch (e) {
      _isConnected = false;
      print("Connection failed: $e");
    }
  }

  // Menutup koneksi
  void close() {
    _client?.close();
    _isConnected = false;
  }

  // Menjalankan perintah SSH
  Future<String> executeCommand(String command) async {
    if (_client != null) {
      final result = await _client!.run(command);
      return utf8.decode(result);
    } else {
      return "Not connected to SSH server.";
    }
  }
```

```
// Fungsi-fungsi SFTP

// 1. List Directory
Future<List<SftpName>> listDirectory(String path) async {
  if (_client != null) {
    final sftp = await _client!.sftp();
    final items = await sftp.listdir(path);
    return items;
  } else {
    throw Exception("Not connected to SSH server.");
  }
}

// 2. Read Remote File
Future<String> readFile(String filePath) async {
  if (_client != null) {
    final sftp = await _client!.sftp();
    final file = await sftp.open(filePath);
    final content = await file.readBytes();
    return latin1.decode(content); // Mengubah byte ke teks
  } else {
    throw Exception("Not connected to SSH server.");
  }
}

// 3. Write Remote File
Future<void> writeFile(String filePath, String content) async {
  if (_client != null) {
    final sftp = await _client!.sftp();
    final file = await sftp.open(filePath, mode: SftpFileOpenMode.write);
    await file.writeBytes(utf8.encode(content));
  } else {
    throw Exception("Not connected to SSH server.");
  }
}

// 4. File Upload
Future<void> uploadFile(String remotePath, File localFile) async {
  if (_client != null) {
    final sftp = await _client!.sftp();
    final file = await sftp.open(remotePath,
        mode: SftpFileOpenMode.create | SftpFileOpenMode.write);
    await file.write(localFile.openRead().cast());
  } else {
    throw Exception("Not connected to SSH server.");
  }
}

// 5. Directory Operations
Future<void> createDirectory(String path) async {
  if (_client != null) {
    final sftp = await _client!.sftp();
    await sftp.mkdir(path);
  } else {
    throw Exception("Not connected to SSH server.");
  }
}

Future<void> removeDirectory(String path) async {
  if (_client != null) {
```

```
      final sftp = await _client!.sftp();
      await sftp.rmdir(path);
    } else {
      throw Exception("Not connected to SSH server.");
    }
  }

  // 6. Get/Set Attributes
  Future<SftpFileAttrs> getFileAttributes(String path) async {
    if (_client != null) {
      final sftp = await _client!.sftp();
      return await sftp.stat(path);
    } else {
      throw Exception("Not connected to SSH server.");
    }
  }

  Future<void> setFileAttributes(String path, SftpFileAttrs attrs) async {
    if (_client != null) {
      final sftp = await _client!.sftp();
      await sftp.setStat(path, attrs);
    } else {
      throw Exception("Not connected to SSH server.");
    }
  }

  // 7. Check Free Space
  Future<Map<String, int>> checkFreeSpace(String path) async {
    if (_client != null) {
      final sftp = await _client!.sftp();
      final statvfs = await sftp.statvfs(path);
      final total = statvfs.blockSize * statvfs.totalBlocks;
      final free = statvfs.blockSize * statvfs.freeBlocks;
      return {'total': total, 'free': free};
    } else {
      throw Exception("Not connected to SSH server.");
    }
  }

  deleteFile(String s) {}

  exec(String s) {}
}
```

- Styles

  Folder ini berisi elemen gaya seperti warna dan teks yang digunakan secara konsisten di seluruh aplikasi untuk menciptakan identitas visual yang seragam.

➢ App_color.dart

File ini mendefinisikan palet warna aplikasi, termasuk warna dasar, utama, dan aksen, yang digunakan untuk tampilan antarmuka.

```
import 'package:flutter/material.dart';

class AppColors {
  // Basic Colors
  static const Color white = Color(0xFFFFFFFF);
  static const Color black = Color(0xFF000000);
```

```
  // Primary Colors
  static const Color darkNavy = Color(0xFF0A0C1B);
  static const Color navy = Color(0xFF14172A);
  static const Color darkGray = Color(0xFF15181F);
  static const Color slateGray = Color(0xFF343746);
  static const Color gray = Color(0xFF373737);
  static const Color mediumGray = Color(0xFF575757);
  static const Color lightGray = Color(0xFF888888);

  // Accent Colors
  static const Color hotPink = Color(0xFFFF69B4);
  static const Color softPink = Color(0xFFFF74B6);
  static const Color lightPink = Color(0xFFFFB6C1);
  static const Color mintGreen = Color(0xFF8FEA7E);
  static const Color brightOrange = Color(0xFFFFAF00);
  static const Color deepRed = Color(0xFFD8275A);
  static const Color slateBlue = Color(0xFF5B6289);
  static const Color brightGreen = Color(0xFF1CB167);
  static const Color green = Color(0xFF29B06C);
}
```

- App_styles.dart

File ini berisi definisi gaya teks, mencakup berbagai jenis font, berat huruf, dan format seperti miring, yang digunakan untuk elemen teks di aplikasi.

```
import 'package:flutter/material.dart';

class AppStyles {
  // Poppins Font Styles
  static const TextStyle poppinsThin = TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w100,
  );
  static const TextStyle poppinsExtraLight = TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w200,
  );
  static const TextStyle poppinsLight = TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w300,
  );
  static const TextStyle poppinsRegular = TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w400,
  );
  static const TextStyle poppinsMedium = TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w500,
  );
  static const TextStyle poppinsSemiBold = TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w600,
  );
  static const TextStyle poppinsBold = TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w700,
  );
```

```
static const TextStyle poppinsExtraBold = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w800,
);
static const TextStyle poppinsBlack = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w900,
);

// Poppins Italic Styles
static const TextStyle poppinsThinItalic = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w100,
  fontStyle: FontStyle.italic,
);
static const TextStyle poppinsExtraLightItalic = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w200,
  fontStyle: FontStyle.italic,
);
static const TextStyle poppinsLightItalic = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w300,
  fontStyle: FontStyle.italic,
);
static const TextStyle poppinsItalic = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w400,
  fontStyle: FontStyle.italic,
);
static const TextStyle poppinsMediumItalic = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w500,
  fontStyle: FontStyle.italic,
);
static const TextStyle poppinsSemiBoldItalic = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w600,
  fontStyle: FontStyle.italic,
);
static const TextStyle poppinsBoldItalic = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w700,
  fontStyle: FontStyle.italic,
);
static const TextStyle poppinsExtraBoldItalic = TextStyle(
  fontFamily: 'Poppins',
  fontWeight: FontWeight.w800,
  fontStyle: FontStyle.italic,
);

// Product Sans Font Styles
static const TextStyle productSansRegular = TextStyle(
  fontFamily: 'ProductSans',
  fontWeight: FontWeight.w400,
);
static const TextStyle productSansBold = TextStyle(
  fontFamily: 'ProductSans',
  fontWeight: FontWeight.w700,
);
static const TextStyle productSansItalic = TextStyle(
```

```
    fontFamily: 'ProductSans',
    fontWeight: FontWeight.w400,
    fontStyle: FontStyle.italic,
  );
  static const TextStyle productSansBoldItalic = TextStyle(
    fontFamily: 'ProductSans',
    fontWeight: FontWeight.w700,
    fontStyle: FontStyle.italic,
  );

  // Consolas Font Styles
  static const TextStyle consolasRegular = TextStyle(
    fontFamily: 'Consolas',
    fontWeight: FontWeight.w400,
  );
  static const TextStyle consolasBold = TextStyle(
    fontFamily: 'Consolas',
    fontWeight: FontWeight.w700,
  );
  static const TextStyle consolasItalic = TextStyle(
    fontFamily: 'Consolas',
    fontWeight: FontWeight.w400,
    fontStyle: FontStyle.italic,
  );
  static const TextStyle consolasBoldItalic = TextStyle(
    fontFamily: 'Consolas',
    fontWeight: FontWeight.w700,
    fontStyle: FontStyle.italic,
  );
}
```

VI. Penerapan Frontend dalam Pengembangan Fitur Aplikasi

1. Speedtest

Speedtest bertujuan untuk mengukur kecepatan unduhan dan pengunggahan jaringan. Penerapan frontend dari fitur ini melibatkan beberapa elemen UI yang interaktif, animasi, serta logika yang terhubung dengan pengujian kecepatan jaringan.

a. Tombol interaktif start

Pada halaman Speedtest, pengguna dapat memulai pengujian kecepatan jaringan dengan menekan tombol Start. Tombol ini diatur agar interaktif, artinya akan mengubah tampilan dan perilakunya berdasarkan status koneksi dan status pengujian. Ketika aplikasi pertama kali dijalankan, tombol Start berada dalam kondisi aktif.

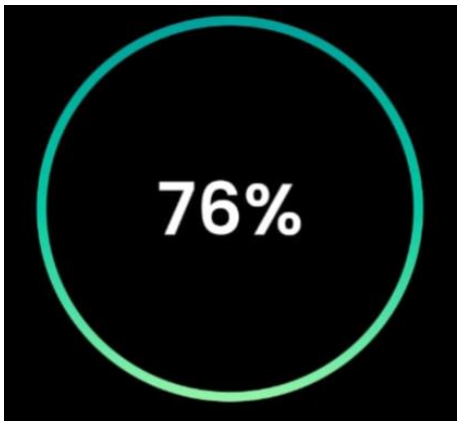
Berikut adalah bagian kode yang mengatur tombol Start:

```
GestureDetector(
  onTap: isTesting ? null : _startSpeedTest,
  child: Container(
    width: circleSize,
    height: circleSize,
    decoration: BoxDecoration(
      gradient: isConnected
          ? const LinearGradient(
              colors: [
                Color(0xFF00A19C),
                Color(0xFF03CAA4),
                Color(0xFF96F5A9),
              ],
              begin: Alignment.topCenter,
              end: Alignment.bottomCenter,
            )
          : const LinearGradient(
              colors: [
                Color(0xFF7A1B34),
                Color(0xFFFF2D6A),
                Color(0xFFFFA376),
              ],
              begin: Alignment.topCenter,
              end: Alignment.bottomCenter,
            ),
      shape: BoxShape.circle,
    ),
    child: Center(
      child: Container(
        width: circleSize - (2 * borderWidth),
        height: circleSize - (2 * borderWidth),
        decoration: const BoxDecoration(
          shape: BoxShape.circle,
          color: Colors.black,
        ),
        child: Center(
          child: Text(
            isTesting ? '$progressPercent%' : 'START',
            style: TextStyle(
              fontFamily: 'Poppins',
              fontWeight: FontWeight.w600,
              fontSize: startTextSize,
              color: Colors.white,
            ),
          ),
        ),
      ),
    ),
  ),
)
```

Pada kode ini, kita menggunakan GestureDetector untuk menangani interaksi sentuhan pada tombol. Jika status pengujian (isTesting) sudah aktif, tombol Start akan dinonaktifkan (tidak dapat ditekan). Sebaliknya, ketika pengujian belum dimulai, tombol ini bisa ditekan untuk memulai tes. isConnected memeriksa konektivitas, di mana warna tombol akan berubah sesuai dengan status koneksi.

b. Animasi progress

Ketika tombol Start ditekan, animasi progres dimulai. Animasi ini menunjukkan progres pengujian secara real-time, mulai dari 0% hingga 100%. Proses ini dilakukan dengan menggunakan Timer untuk memperbarui persentase progres setiap 100ms.



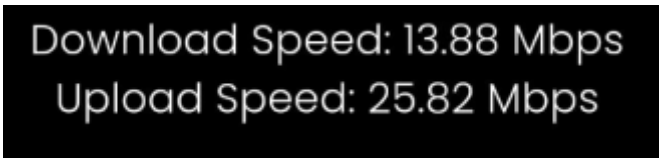
Berikut adalah bagian kode yang mengatur animasi progres:

```
void _startProgressAnimation() {
  _progressTimer = Timer.periodic(const Duration(milliseconds: 100), (timer) {
    if (isTesting && progressPercent < 98) {
      setState(() {
        progressPercent += 2; // Tambah progres 2% setiap 100ms hingga 89%
      });
    } else if (!isTesting && progressPercent < 100) {
      setState(() {
        progressPercent += 1; // Lanjutkan progres ke 100 setelah tes selesai
      });
    } else {
      _progressTimer?.cancel(); // Berhenti jika sudah 100% atau tes selesai
    }
  });
}
```

Pada kode ini, Timer.periodic digunakan untuk memperbarui nilai progressPercent setiap 100ms. Ketika pengujian selesai, animasi progres dihentikan, dan nilai progres diubah menjadi 100%.

c. Tampilan hasil pengujian

Setelah pengujian selesai, hasil kecepatan unduh dan unggah ditampilkan pada layar. Ini menggunakan data yang diperoleh dari fungsi pengujian kecepatan dan ditampilkan dalam format yang dapat dibaca pengguna. Proses ini dimulai dengan menampilkan status "Testing in progress..." ketika pengujian sedang berlangsung, dan berlanjut dengan menampilkan hasil pengujian setelah selesai. Berikut adalah bagian kode yang mengatur tampilan hasil pengujian:



Kode yang bertanggung jawab untuk menampilkan hasil pengujian adalah:

```
Text(
  isTesting
      ? 'Testing in progress...'
      : downloadSpeed != null
          ? 'Download Speed: ${downloadSpeed!.toStringAsFixed(2)} Mbps\nUpload Speed:
${uploadSpeed == null ? 'failed' : uploadSpeed!.toStringAsFixed(2)} Mbps'
          : isConnected
              ? 'Connected to the internet'
              : 'Check connection, \ninternet not connected',
  textAlign: TextAlign.center,
  style: TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w300,
    fontSize: statusTextSize,
    color: Colors.white,
  ),
)
```

Pada kode ini, selama pengujian berlangsung (isTesting == true), teks yang ditampilkan adalah "Testing in progress...". Setelah pengujian selesai, kecepatan unduh dan unggah ditampilkan dalam format yang mudah dibaca, dengan dua angka desimal. Jika pengujian gagal atau tidak ada koneksi, status akan menunjukkan bahwa tidak ada koneksi internet atau pengujian gagal.

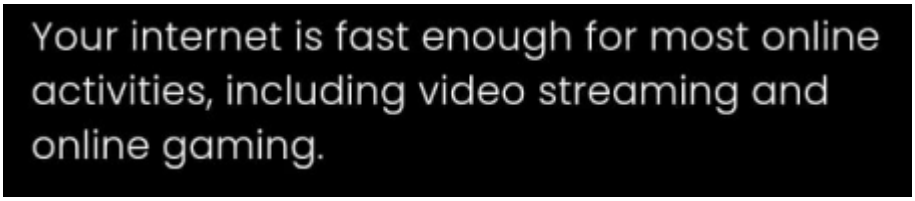
Setelah hasil pengujian, aplikasi menampilkan kategori kecepatan internet berdasarkan rata-rata kecepatan unduh dan unggah yang diperoleh. Fungsi getSpeedCategory menghitung rata-rata kecepatan unduh dan unggah, kemudian mengklasifikasikannya menjadi kategori seperti "Internet Anda sangat cepat!", "Internet Anda cepat!", atau "Internet Anda lambat!".

Kode untuk menampilkan kategori kecepatan adalah sebagai berikut:

```
Text(
  getSpeedCategory(downloadSpeed!, uploadSpeed!),
  textAlign: TextAlign.center,
  style: TextStyle(
    fontFamily: 'Poppins',
    fontWeight: FontWeight.w600,
    fontSize: statusTextSize,
    color: Colors.white,
  ),
)
```

Fungsi getSpeedCategory ini menentukan kategori kecepatan berdasarkan nilai rata-rata dari kecepatan unduh dan unggah yang diperoleh, memberikan pengguna gambaran jelas mengenai kualitas koneksi mereka.

Selanjutnya, aplikasi memberikan penjelasan tambahan tentang kecepatan internet yang diperoleh. Penjelasan ini bertujuan untuk membantu pengguna memahami apakah koneksi mereka cukup cepat untuk berbagai aktivitas, seperti streaming video HD, bermain game online, atau mengunduh file besar.



Berikut adalah bagian kode yang menampilkan penjelasan kecepatan internet:

```
Padding(
  padding: EdgeInsets.only(left: screenWidth * 0.06),
  child: Text(
    getSpeedExplanation(downloadSpeed!),
    textAlign: TextAlign.left,
    style: TextStyle(
      fontFamily: 'Poppins',
      fontWeight: FontWeight.w300,
      fontSize: statusTextSize * 0.7,
      color: Colors.white,
    ),
  ),
)
```

Penjelasan ini disesuaikan berdasarkan hasil tes kecepatan unduh dan memberikan saran terkait aktivitas yang sesuai dengan kategori kecepatan yang diperoleh. Sebagai contoh, jika kecepatan unduh lebih dari 50 Mbps, aplikasi akan menyarankan bahwa koneksi tersebut cukup untuk streaming video HD dan bermain game online secara bersamaan tanpa gangguan.

d. Kondisi koneksi tidak terhubung

Jika aplikasi tidak mendeteksi koneksi internet saat memulai tes, tombol Start akan berubah dan status pengujian tidak dapat dimulai.

Kode untuk memeriksa konektivitas internet dan memperbarui status UI adalah:

```
_internetSubscription = NetworkCheck.internetStatusStream.listen((status) {
  setState(() {
    isConnected = status; // Memperbarui status koneksi
  });
});
```

Jika koneksi tidak ada, tombol Start akan menggunakan warna yang berbeda untuk menandakan status koneksi buruk. Jika koneksi tersedia, tombol kembali normal dan dapat digunakan untuk memulai pengujian.

2. Sniffing

Sniffing bertujuan untuk menangkap dan memonitor paket data yang melewati jaringan. Fitur ini sangat berguna untuk mengidentifikasi lalu lintas jaringan yang mencurigakan, memahami lebih dalam tentang komunikasi data dalam jaringan, dan mendapatkan wawasan tentang koneksi internet mereka. Pada sisi frontend, ada beberapa elemen interaktif yang digunakan untuk memberikan visualisasi yang jelas dan mudah dipahami oleh pengguna, seperti tampilan icon, tabel untuk menampilkan informasi paket, metrik jaringan, serta tampilan detail paket saat ditekan.

a. Tampilan icon

Ikon interaktif di bagian atas halaman mengatur status sniffing, dengan ikon yang berubah saat proses dimulai atau dihentikan. Tombol ini untuk memulai dan menghentikan proses sniffing, tak hanya itu terdapat juga tombol save dan share

Berikut adalah kode yang menangani tampilan tombol Start dan Stop:

```
GestureDetector(
  onTap: _toggleStart,
  child: SvgPicture.asset(
    'assets/icons/start.svg',
    color: isStarted ? const Color(0xFF666666) : null,
    height: 24,
    width: 24,
  ),
),

GestureDetector(
  onTap: isStarted ? _toggleStart : null,
  child: SvgPicture.asset(
    'assets/icons/stop.svg',
    color: isStarted ? null : const Color(0xFF666666),
    height: 24,
    width: 24,
  ),
),
```

Pada kode ini, ikon Start akan berubah menjadi warna abu-abu ketika sniffing sudah dimulai (karena isStarted bernilai true), dan ikon Stop akan menjadi aktif hanya ketika sniffing telah dimulai. Interaksi ini memudahkan pengguna untuk memulai dan menghentikan sniffing secara visual.

b. Tampilan table paket data

Setelah sniffing dimulai, paket data yang berhasil ditangkap akan ditampilkan dalam bentuk tabel. Setiap paket memiliki informasi seperti nomor urut, waktu penangkapan,

alamat IP sumber, alamat tujuan, protokol yang digunakan, panjang paket, dan informasi tambahan seperti status respons dan payload.

No.	Time	Source	Path	Proto.	Length	Info
1	202...	jsonplace...	/posts/1	HTTPS	81	OK 200
2	202...	jsonplace...	/posts/1	HTTPS	81	OK 200
3	202...	jsonplace...	/posts/1	HTTPS	81	OK 200
4	202...	jsonplace...	/posts/1/c...	HTTPS	1510	OK 200
5	202...	jsonplace...	/posts	HTTPS	71	Create...
6	202...	jsonplace...	/commen...	HTTPS	1510	OK 200
7	202...	jsonplace...	/posts	HTTPS	71	Create...
8	202...	jsonplace...	/posts/1	HTTPS	81	OK 200
9	202...	jsonplace...	/posts	HTTPS	71	Create...
10	202...	jsonplace...	/posts	HTTPS	71	Create...

Berikut adalah kode yang menangani pembuatan tabel ini:

```
Container(
  color: const Color(0xFF14172A),
  height: headerHeight,
  child: Row(
    children: [
      _buildTableHeader('No.', noWidth),
      _buildTableHeader('Time', timeWidth),
      _buildTableHeader('Source', sourceWidth),
      _buildTableHeader('Path', destinationWidth),
      _buildTableHeader('Proto.', protoWidth),
      _buildTableHeader('Length', lengthWidth),
      _buildTableHeader('Info', infoWidth),
    ],
  ),
),
```

Fungsi _buildTableHeader ini membuat header tabel untuk setiap kolom, seperti No., Time, Source, Path, Proto., Length, dan Info. Masing-masing kolom memiliki lebar yang disesuaikan dengan data yang ditampilkan di dalamnya. Kode ini menciptakan struktur tabel yang rapi untuk menampilkan informasi paket yang ditangkap. Berikut adalah kode untuk menampilkan data setiap paket dalam tabel:

```
Expanded(
  child: SingleChildScrollView(
    child: Column(
      children: _sniffedData.asMap().entries.map((entry) {
        int index = entry.key + 1;
        InspectorResult data = entry.value;

        // Mendapatkan informasi protokol
        String protocol = data.url?.scheme.toUpperCase() ?? '-';

        // Menyusun informasi untuk kolom Info
        String info = '${data.reasonPhrase} ${data.statusCode ?? '-'}';
```

```
        return GestureDetector(
          onTap: () {
            Navigator.pushNamed(
              context,
              '/details',
              arguments: data, // Mengirimkan objek data ke DetailsScreen
            );
          },
          child: Row(
            children: [
              _buildTableCell(index.toString(), noWidth, rowHeight),
              _buildTableCell(data.startTime?.toIso8601String()  ??  '-',  timeWidth,
rowHeight),
              _buildTableCell(data.url?.host ?? '-', sourceWidth, rowHeight),
              _buildTableCell(data.url?.path ?? '-', destinationWidth, rowHeight),
              _buildTableCell(protocol, protoWidth, rowHeight),
              _buildTableCell(data.responseBodyBytes?.toString()       ??       '-',
lengthWidth, rowHeight),
              _buildTableCell(info, infoWidth, rowHeight),
            ],
          ),
        );
      }).toList(),
    ),
  ),
)
```

Dalam kode ini, Row digunakan untuk menampilkan setiap baris data dalam tabel. Masing-masing kolom, seperti nomor paket, waktu, sumber, protokol, panjang data, dan informasi lainnya, dibangun menggunakan _buildTableCell. Setiap baris juga di-wrap dalam GestureDetector, memungkinkan pengguna untuk menekan paket tertentu untuk melihat rincian lebih lanjut.

c. Metrik dan konklusi akhir

Setelah proses sniffing dihentikan, aplikasi menghitung dan menampilkan metrik jaringan seperti tingkat keberhasilan paket yang diterima, waktu respons rata-rata, data yang diterima, dan jumlah paket yang diblokir. Metode _calculateNetworkMetrics menghitung metrik-metrik ini, dan hasilnya ditampilkan melalui popup di bagian bawah layar.

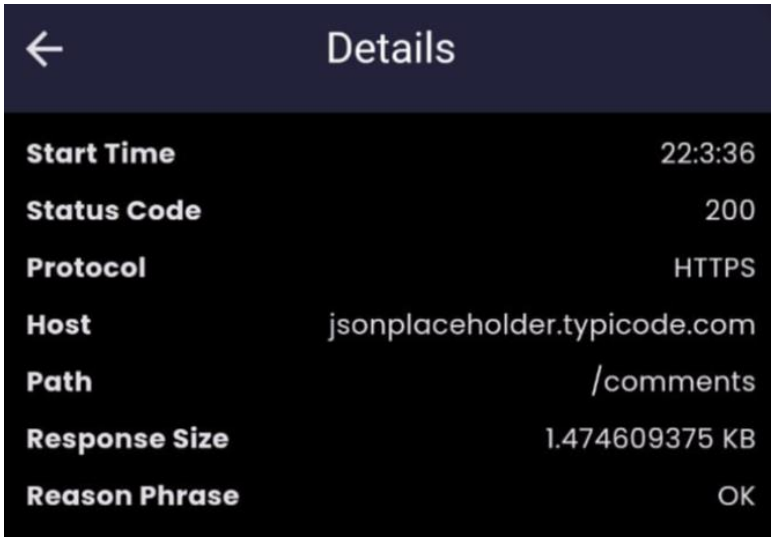
Berikut adalah kode yang menangani tampilan diatas:

```
void _calculateNetworkMetrics() {
  if (_sniffedData.isEmpty) return;

  int successfulRequests = _sniffedData
      .where((result) => result.statusCode != null && result.statusCode! >= 200 &&
result.statusCode! < 300)
      .length;

  int blockedPackets = _sniffedData
      .where((result) => result.statusCode != null && (result.statusCode! >= 400 &&
result.statusCode! < 600))
      .length;

  double successRate = (successfulRequests / _sniffedData.length) * 100;

  int totalResponseTime = 0;
  int totalDataReceived = 0;

  for (var result in _sniffedData) {
    if (result.startTime != null && result.endTime != null) {
      int                              responseTime                              =
result.endTime!.difference(result.startTime!).inMilliseconds;
      totalResponseTime += responseTime;
    }
    totalDataReceived += result.responseBodyBytes ?? 0;
  }

  int   avgResponseTime   =   _sniffedData.isNotEmpty   ?   (totalResponseTime   /
_sniffedData.length).round() : 0;

  showModalBottomSheet(
    context: context,
    backgroundColor: Colors.transparent,
    builder: (BuildContext context) {
      return NetworkStatusPopup(
        successRate: successRate,
        avgResponseTime: avgResponseTime,
        totalDataReceived: (totalDataReceived / 1024).round(), // Konversi ke KB
        blockedPackets: blockedPackets,
        successfulPackets: successfulRequests,
      );
    },
  );
}
```

Setelah menghentikan sniffing, hasil perhitungan metrik ditampilkan dalam bentuk popup yang menampilkan tingkat keberhasilan, waktu respons rata-rata, data yang diterima, dan jumlah paket yang diblokir.

d. Rincian paket data

Selain menampilkan daftar paket yang ditangkap, aplikasi juga memungkinkan pengguna untuk melihat rincian lebih lanjut dari paket tertentu yang ditekan. Ketika pengguna menekan baris paket, aplikasi akan mengarahkan pengguna ke halaman rincian yang menampilkan detail lebih lanjut mengenai paket tersebut. Rincian ini mencakup waktu mulai dan selesai, kode status HTTP, protokol, panjang data, dan informasi lainnya yang relevan.



Berikut adalah kode yang menangani tampilan diatas:

```
Navigator.pushNamed(
  context,
  '/details',
  arguments: data, // Mengirimkan objek data ke DetailsScreen
);


//Pada details_screen
child: ListView(
          padding: const EdgeInsets.fromLTRB(20, 5, 20, 20),
          children: [
            _buildRow('Start Time', formatDateTime(data.startTime)),
            _buildRow('Status Code', data.statusCode?.toString() ?? '-'),
            _buildRow('Protocol', data.url?.scheme.toUpperCase() ?? '-'),
            _buildRow('Host', data.url?.host ?? '-'),
            _buildRow('Path', data.url?.path ?? '-'),
            _buildRow(
              'Response Size',
              '${(data.responseBodyBytes ?? 0) / 1024} KB',
            ),
            _buildRow(
              success ? 'Reason Phrase' : 'Error Message',
              data.reasonPhrase ?? '-',
            ),
          ],
        ),
```

Pada kode ini, pengguna dapat melihat informasi lebih rinci mengenai paket tertentu yang dipilih. Halaman rincian ini memungkinkan pengguna untuk memahami lebih dalam tentang paket yang ditangkap, termasuk protokol yang digunakan dan data tambahan terkait komunikasi jaringan.

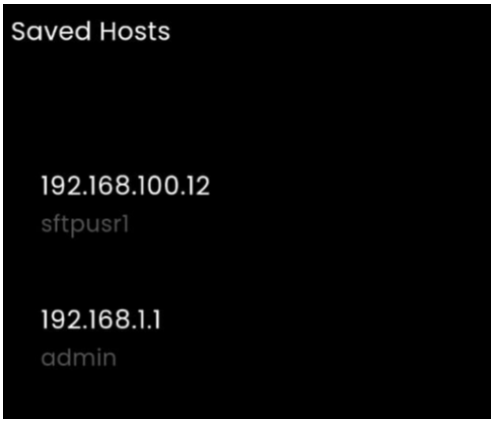
3. Terminal

Terminal berfungsi untuk menghubungkan ke server menggunakan protokol SSH dan menjalankan perintah terminal langsung dari antarmuka aplikasi. Pada sisi frontend, fitur ini melibatkan tampilan dinamis yang memudahkan pengguna untuk berinteraksi dengan

server melalui terminal virtual, memilih host yang akan diakses, serta menjalankan perintah-perintah yang diperlukan. Dalam penerapan frontend ini, terdapat beberapa komponen utama, yaitu List View Daftar Host, Bottom Sheet untuk Pilihan Koneksi, dan Tampilan Terminal SSH yang dirancang untuk memberikan pengalaman pengguna yang lancar dan interaktif.

a. List view daftar host

Untuk memilih host yang akan diakses melalui SSH, aplikasi menyediakan tampilan List View yang menampilkan daftar host yang tersedia. Setiap item dalam daftar ini berisi informasi terkait host seperti nama, alamat IP, dan status koneksi. Berikut adalah implementasi tampilan daftar host menggunakan ListView:



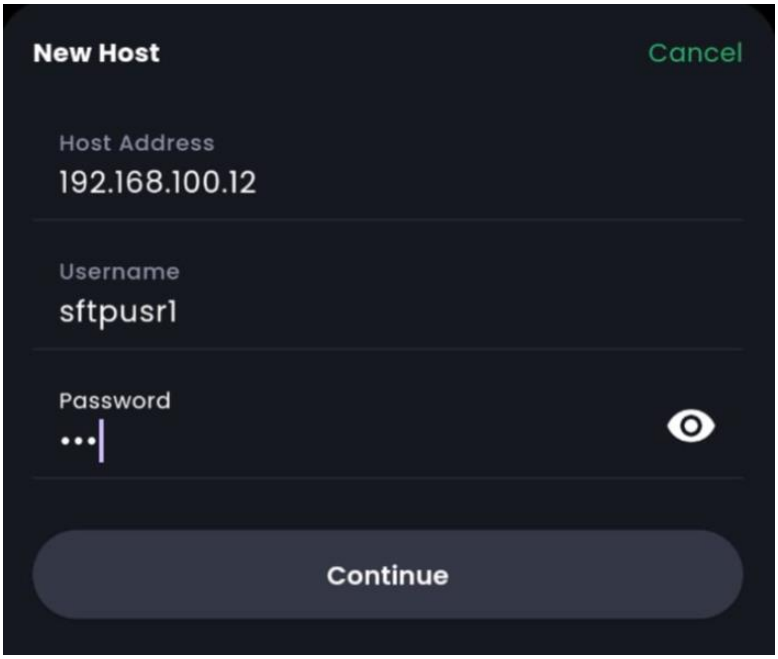
Berikut adalah kode yang menangani tampilan diatas:

```
ListView.builder(
  itemCount: hostList.length,
  itemBuilder: (context, index) {
    final host = hostList[index];
    return ListTile(
      title: Text(host.name),
      subtitle: Text('IP: ${host.ip}'),
      onTap: () {
        _connectToHost(host);
      },
    );
  },
)
```

Pada kode di atas, ListView.builder digunakan untuk menampilkan daftar host yang tersedia. Setiap item memiliki informasi nama host dan alamat IP yang ditampilkan sebagai Text di dalam ListTile. Ketika pengguna menekan salah satu host, aplikasi akan mencoba untuk melakukan koneksi SSH ke host tersebut dengan memanggil fungsi _connectToHost(host).

b. Bottom sheet add/modify host

Saat pengguna memilih host dari daftar, aplikasi menampilkan Bottom Sheet yang berfungsi untuk mengonfirmasi koneksi atau menyediakan opsi tambahan, seperti memilih username dan password untuk otentikasi. Host juga dapat di modifikasi, tampilan pada modify mirip seperti ketika membuat baru. Berikut adalah contoh implementasi Bottom Sheet:



Berikut adalah kode yang menangani tampilan diatas:

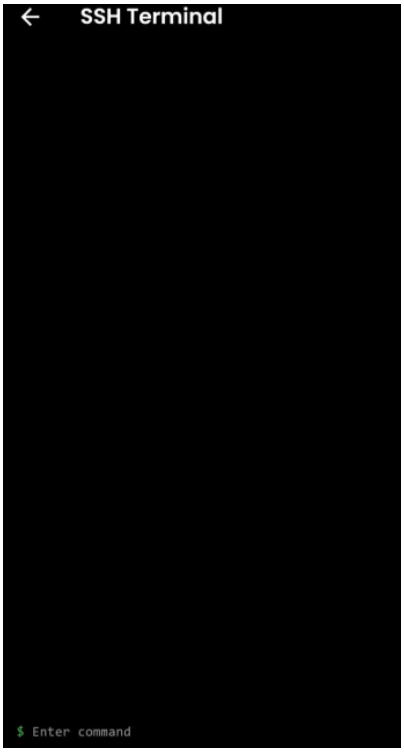
```
showModalBottomSheet(
  context: context,
  builder: (BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text('Connect to: ${selectedHost.name}', style: TextStyle(fontSize: 18)),
          SizedBox(height: 10),
          TextField(
            decoration: InputDecoration(labelText: 'Username'),
            controller: _usernameController,
          ),
          TextField(
            decoration: InputDecoration(labelText: 'Password'),
            controller: _passwordController,
            obscureText: true,
          ),
          SizedBox(height: 20),
          ElevatedButton(
            onPressed: () {
              _connectSSH(_usernameController.text, _passwordController.text);
            },
            child: Text('Connect'),
          ),
        ],
      ),
    );
  },
```

```
);
```

Pada kode ini, Bottom Sheet ditampilkan dengan menggunakan showModalBottomSheet, yang memuat dua kolom input untuk memasukkan username dan password, serta tombol Connect yang akan mencoba menyambungkan ke host SSH dengan kredensial yang diberikan. Fungsi _connectSSH akan digunakan untuk memulai koneksi setelah pengguna menekan tombol Connect.

c. Tampilan terminal

Setelah koneksi SSH berhasil, aplikasi menampilkan tampilan Terminal SSH yang memungkinkan pengguna untuk mengetik perintah dan melihat hasilnya secara langsung. Tampilan terminal ini menyerupai antarmuka terminal konvensional, dengan area input di bagian bawah dan area keluaran di bagian atas yang menampilkan hasil perintah yang dijalankan. Berikut adalah implementasi frontend untuk tampilan terminal:



Berikut adalah kode yang menangani tampilan diatas:

```
Column(
  children: [
    Expanded(
      child: SingleChildScrollView(
        controller: _scrollController,
        child: Text(
          terminalOutput,
          style:    TextStyle(fontFamily:    'Consolas',    fontSize:    14,    color:
Colors.white),
        ),
      ),
    ),
    Padding(
```

```
        padding: const EdgeInsets.all(8.0),
        child: TextField(
          controller: _commandController,
          onSubmitted: (command) {
            _sendCommand(command);
          },
          style: TextStyle(fontFamily: 'Consolas', fontSize: 14, color: Colors.white),
          decoration: InputDecoration(
            hintText: 'Enter command...',
            hintStyle:   TextStyle(fontFamily:   'Consolas',   fontSize:   14,   color:
Colors.grey),
            filled: true,
            fillColor: Colors.black,
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(8),
            ),
          ),
        ),
      ),
    ],
  )
```

Tampilan terminal dibagi menjadi dua bagian: bagian pertama adalah area keluaran terminal yang menampilkan hasil dari perintah yang dijalankan. Area ini menggunakan widget SingleChildScrollView agar pengguna dapat menggulir hasil perintah jika hasilnya terlalu panjang. Di bagian bawah terminal terdapat TextField yang memungkinkan pengguna untuk mengetik perintah yang ingin dijalankan. Setiap perintah yang dimasukkan akan dikirimkan menggunakan fungsi _sendCommand(command).

d. Icon pada tampilan SSH terminal

Di bagian atas terminal SSH, terdapat beberapa ikon interaktif yang memberikan kontrol tambahan bagi pengguna, seperti ikon untuk memutuskan koneksi SSH atau menyimpan log perintah yang telah dijalankan. Berikut adalah contoh kode untuk menampilkan ikon-ikon ini:

Berikut adalah kode yang menangani tampilan diatas:

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    GestureDetector(
      onTap: _disconnectSSH,
      child: Icon(Icons.exit_to_app, color: Colors.white, size: 24),
    ),
    GestureDetector(
      onTap: _saveLog,
      child: Icon(Icons.save_alt, color: Colors.white, size: 24),
    ),
    GestureDetector(
      onTap: _clearScreen,
      child: Icon(Icons.refresh, color: Colors.white, size: 24),
    ),
  ],
```

```
)
```

Pada kode di atas, terdapat tiga ikon: Exit untuk memutuskan koneksi SSH, Save untuk menyimpan log terminal, dan Clear untuk membersihkan layar terminal. Setiap ikon terhubung dengan fungsi terkait seperti _disconnectSSH, _saveLog, dan _clearScreen, yang akan memicu tindakan sesuai dengan ikon yang dipilih.

4. VPN

Pada sisi frontend, aplikasi menyediakan antarmuka pengguna yang memungkinkan pengguna untuk mengelola koneksi VPN dengan mudah. Fitur utama yang dihadirkan adalah tombol toggle untuk menghubungkan atau memutuskan koneksi VPN, yang berfungsi dengan animasi visual yang halus. Ketika pengguna menekan tombol toggle, status koneksi berubah menjadi "Connecting" dan kemudian menjadi "Connected" setelah beberapa detik. Setelah koneksi berhasil, aplikasi menampilkan informasi rinci tentang koneksi VPN, seperti DNS protocol, colocation center, jenis koneksi, dan informasi perangkat. Kode di bawah ini mengatur tampilan tombol toggle dan status koneksi VPN, serta memuat informasi lebih lanjut setelah koneksi terjalin:

```
GestureDetector(
  onTap: _toggleVPN, // Fungsi saat toggle ditekan
  child: AnimatedContainer(
    duration: const Duration(milliseconds: 300), // Animasi smooth
    width: toggleWidth,
    height: toggleHeight,
    padding: const EdgeInsets.symmetric(horizontal: 16),
    decoration: BoxDecoration(
      gradient: LinearGradient(
        colors: isConnected || isConnecting
            ? const [Color(0xFFFF69B4), Color(0xFFFF74B6), Color(0xFFFFB6C1)]
            : const [Color(0xFF888888), Color(0xFF575757), Color(0xFF373737)],
        begin: Alignment.centerLeft,
        end: Alignment.centerRight,
      ),
      borderRadius: BorderRadius.circular(120), // Bentuk lonjong
    ),
    child: Stack(
      children: [
        AnimatedAlign(
          duration: const Duration(milliseconds: 300),
          alignment: isConnected || isConnecting
              ? Alignment.centerRight
              : Alignment.centerLeft, // Posisi lingkaran
          child: Container(
            width: circleSize,
            height: circleSize,
            decoration: const BoxDecoration(
              color: Colors.white,
              shape: BoxShape.circle,
            ),
          ),
        ),
      ],
    ),
  ),
),
```

5. File Transfer

File transfer memungkinkan pengguna untuk mengirim dan menerima file antara perangkat mereka dengan server jarak jauh menggunakan protokol SFTP. Fitur ini memiliki beberapa elemen antarmuka pengguna yang penting, seperti List View Daftar Host/Server, Bottom Sheet untuk Pilihan Koneksi, Tampilan Directory List (dir_screen), dan ikon interaktif di dalam layar direktori.

a. Daftar server

Daftar ini menunjukkan nama host dan alamat IP untuk memudahkan pengguna dalam memilih server yang ingin digunakan untuk mentransfer file. Berikut adalah implementasi tampilan daftar host menggunakan ListView:

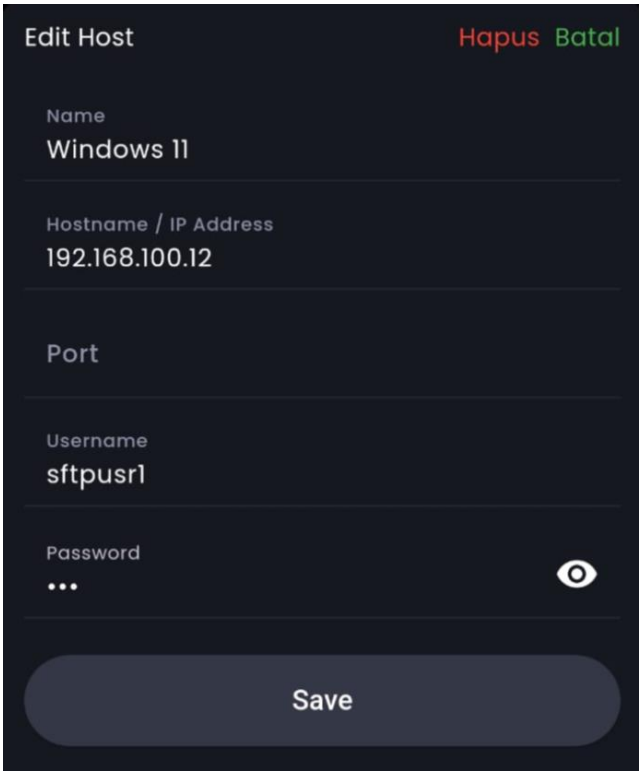
Berikut adalah kode yang menangani tampilan diatas:

```
ListView.builder(
  itemCount: serverList.length,
  itemBuilder: (context, index) {
    final server = serverList[index];
    return ListTile(
      title: Text(server.name),
      subtitle: Text('IP: ${server.ip}'),
      onTap: () {
        _connectToServer(server);
      },
    );
  },
)
```

Pada kode ini, ListView.builder digunakan untuk membuat daftar dinamis dari server yang terhubung, dengan masing-masing item menampilkan nama server dan alamat IP. Setiap item dalam daftar dapat diketuk oleh pengguna untuk memulai koneksi ke server yang dipilih, memanggil fungsi _connectToServer untuk melanjutkan ke langkah selanjutnya dalam proses transfer file.

b. Bottom sheet

Setelah memilih server dari daftar, aplikasi menampilkan Bottom Sheet untuk meminta kredensial pengguna (seperti username dan password) untuk terhubung ke server yang dipilih. Bottom Sheet ini memberikan cara yang mudah dan bersih untuk menangani input data. Berikut adalah kode untuk menampilkan Bottom Sheet yang berisi input form untuk username dan password:



Berikut adalah kode yang menangani tampilan diatas:

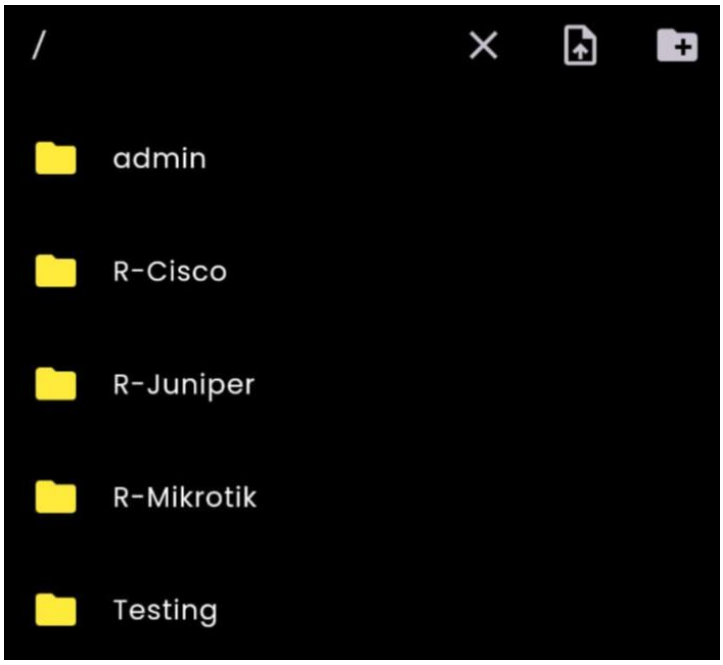
```
showModalBottomSheet(
  context: context,
  builder: (BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text('Connect to: ${selectedServer.name}', style: TextStyle(fontSize: 18)),
          SizedBox(height: 10),
          TextField(
            decoration: InputDecoration(labelText: 'Username'),
            controller: _usernameController,
          ),
          TextField(
            decoration: InputDecoration(labelText: 'Password'),
            controller: _passwordController,
            obscureText: true,
          ),
          SizedBox(height: 20),
          ElevatedButton(
            onPressed: () {
              _connectSFTP(_usernameController.text, _passwordController.text);
```

```
            },
            child: Text('Connect'),
          ),
        ],
      ),
    );
  },
);
```

Pada kode ini, showModalBottomSheet digunakan untuk menampilkan form input yang berisi dua TextField untuk mengisi username dan password. Setelah pengguna mengisi kredensial mereka, tombol Connect akan mencoba untuk terhubung ke server menggunakan kredensial yang diberikan. Fungsi _connectSFTP akan dipanggil untuk melakukan koneksi setelah tombol ditekan.

c. Tampilan directory server tujuan

Setelah berhasil terhubung ke server, aplikasi menampilkan Tampilan Daftar Direktori (dir_screen) yang menunjukkan struktur direktori di server tersebut. Direktori dan file yang ada pada server ditampilkan dalam bentuk list yang dapat diketuk untuk membuka folder atau memilih file. Berikut adalah implementasi tampilan Directory List menggunakan ListView:



Berikut adalah kode yang menangani tampilan diatas:

```
ListView.builder(
  itemCount: directoryList.length,
  itemBuilder: (context, index) {
    final dirItem = directoryList[index];
    return ListTile(
      title: Text(dirItem.name),
      leading: Icon(
```

```
          dirItem.isDirectory ? Icons.folder : Icons.insert_drive_file,
          color: dirItem.isDirectory ? Colors.blue : Colors.green,
        ),
        onTap: () {
          if (dirItem.isDirectory) {
            _openDirectory(dirItem);
          } else {
            _selectFile(dirItem);
          }
        },
      );
    },
  )
```

Pada kode ini, ListView.builder digunakan untuk menampilkan daftar item dalam direktori. Setiap item dalam daftar dapat berupa folder atau file, yang ditandai dengan ikon folder atau file. Ketika pengguna mengetuk item yang bertipe direktori, fungsi _openDirectory(dirItem) akan dipanggil untuk membuka isi direktori tersebut, sedangkan jika item tersebut adalah file, fungsi _selectFile(dirItem) akan dipanggil untuk memilih file yang akan diunduh atau diunggah.

d. Ikon pada directory screen

Di dalam tampilan Directory List pada layar direktori (dir_screen), ikon sangat penting untuk memberikan indikasi visual tentang tipe item, apakah itu sebuah folder atau file. Di bawah ini adalah kode untuk menampilkan ikon yang sesuai dengan tipe item:

Berikut adalah kode yang menangani tampilan diatas:

```
leading: Icon(
  dirItem.isDirectory ? Icons.folder : Icons.insert_drive_file,
  color: dirItem.isDirectory ? Colors.blue : Colors.green,
),
```

Pada kode ini, setiap item di dalam daftar direktori memiliki ikon yang diatur berdasarkan apakah item tersebut adalah folder atau file. Folder ditandai dengan ikon folder berwarna biru, sementara file menggunakan ikon file berwarna hijau. Ini memberikan petunjuk visual yang jelas kepada pengguna tentang apa yang mereka pilih.

```
Row(
  children: [
    IconButton(
```

```
        icon: Icon(Icons.arrow_back, color: Colors.white),
        onPressed: _goBack,
      ),
      IconButton(
        icon: Icon(Icons.upload_file, color: Colors.white),
        onPressed: _uploadFile,
      ),
    ],
  )
```

Pada kode ini, dua ikon back dan upload ditampilkan di bagian atas layar direktori. Ikon back memungkinkan pengguna untuk kembali ke direktori sebelumnya dengan memanggil fungsi _goBack, sementara ikon upload memungkinkan pengguna untuk memilih dan mengunggah file ke server dengan memanggil fungsi _uploadFile.

VII. Penerapan Backend dalam Pengembangan Fitur Aplikasi

1. Speedtest

Dari sisi backend dalam pengembangan fitur Sniffing, konsep pemrograman yang diterapkan adalah pengujian kecepatan jaringan, yang melibatkan pengunduhan dan pengunggahan file untuk menghitung download dan upload speed secara real-time. Untuk mengimplementasikan hal ini, kami menggunakan HTTP requests untuk mengukur waktu yang dibutuhkan dalam mengunduh dan mengunggah data, dan kemudian menghitung kecepatan berdasarkan jumlah byte yang diterima atau dikirim dalam satuan waktu tertentu. Implementasi backend untuk fitur ini berfokus pada pengambilan data menggunakan HTTP dan penghitungan waktu yang dibutuhkan untuk memperoleh data tersebut, yang memungkinkan perhitungan kecepatan jaringan secara efisien dan akurat.

Pada aplikasi ini, backend dimulai dengan dua URL yaitu downloadUrl dan uploadUrl, yang digunakan untuk menguji kecepatan unduh dan unggah. URL ini adalah sumber file yang digunakan untuk pengujian download dan server untuk pengujian upload. Ketika pengujian dimulai, aplikasi membuat permintaan HTTP ke downloadUrl untuk mengunduh file besar dan menghitung waktu yang diperlukan untuk menerima data tersebut. Setelah itu, kecepatan unduh dihitung dengan rumus: kecepatan = jumlah byte yang diterima / waktu yang diperlukan dalam detik, yang kemudian diubah menjadi megabit per detik (Mbps). Hal ini dilakukan menggunakan fungsi _testDownloadSpeed() seperti yang terlihat pada kode berikut:

```
Future<double?> _testDownloadSpeed() async {
    print("Starting download test..."); // Log mulai pengujian download
    try {
      final stopwatch = Stopwatch()..start();
      final response = await http.get(Uri.parse(downloadUrl));

      if (response.statusCode == 200) {
        stopwatch.stop();
        int bytes = response.contentLength ?? response.bodyBytes.length;
        double seconds = stopwatch.elapsedMilliseconds / 1000;
        double speedMbps = (bytes / seconds) / (1024 * 1024) * 8;
```

```
        String fileSize = convertSize(
            bytes); // Mengonversi ukuran file ke format yang lebih mudah dibaca
        print(
            "Download speed test successful: $speedMbps Mbps, File Size: $fileSize");
// Log hasil pengujian download

        return speedMbps;
      } else {
        print("Failed to download. Status Code: ${response.statusCode}");
      }
    } catch (e) {
      print("Download speed test failed: $e");
    }
    return null;
  }
```

Begitu file berhasil diunduh, fungsi ini menghitung jumlah byte yang diterima dan menghitung kecepatan download dalam satuan megabit per detik. Fungsi ini mengukur durasi download menggunakan Stopwatch dan menggunakan hasilnya untuk menghitung kecepatan transfer data. Jika pengujian berhasil, kecepatan unduh akan ditampilkan dalam aplikasi. Begitu pula untuk pengujian upload, backend menggunakan metode yang serupa, dengan mengunggah file menggunakan HTTP POST request untuk menghitung kecepatan unggah, yang dapat dilihat pada kode berikut:

```
Future<double?> _testUploadSpeed() async {
    print("Starting upload test..."); // Log mulai pengujian upload
    try {
      final stopwatch = Stopwatch()..start();

      var request = http.MultipartRequest('POST', Uri.parse(uploadUrl));
      var file = http.MultipartFile.fromBytes(
        'file',
        List<int>.generate(104857600, (index) => index % 256),
        filename: 'filedummy.txt',
      );
      request.files.add(file);

      var response = await request.send();
      stopwatch.stop();

      int bytesSent = file.length;
      double seconds = stopwatch.elapsedMilliseconds / 1000;
      double speedMbps = (bytesSent / seconds) / (1024 * 1024) * 8;

      if (response.statusCode == 200) {
        print(
            "Upload speed test successful: $speedMbps Mbps, Data Size: 100MB"); //
Log hasil pengujian upload
        return speedMbps;
      } else {
        print("Failed to upload data. Status Code: ${response.statusCode}");
      }
    } catch (e) {
      print("Upload speed test failed: $e");
    }
    return null;
  }
```

Fungsi _testUploadSpeed() memulai pengujian upload dengan mengunggah file dummy ke server menggunakan http.MultipartRequest. Seperti pada pengujian download, waktu yang dibutuhkan untuk mengunggah file diukur menggunakan

Stopwatch, dan kecepatan dihitung berdasarkan ukuran file yang dikirim dan durasi pengiriman. Jika upload berhasil, kecepatan akan dihitung dan ditampilkan di aplikasi. Dalam kedua pengujian ini, Future.wait() digunakan untuk menjalankan kedua pengujian tersebut secara paralel, yang memungkinkan aplikasi mengukur kecepatan unduh dan unggah sekaligus, meningkatkan efisiensi pengujian.

Setelah kedua pengujian selesai, backend melakukan perhitungan kategori kecepatan berdasarkan nilai rata-rata antara kecepatan download dan upload. Kategori ini membantu pengguna memahami kualitas koneksi mereka dengan memberikan pesan yang sesuai, seperti "Your internet is fast!" atau "Your internet is slow!". Fungsi untuk menentukan kategori kecepatan adalah getSpeedCategory() yang menghitung rata-rata kecepatan dan mengembalikan pesan berdasarkan nilai tersebut. Berikut adalah implementasi fungsinya:

```
String getSpeedCategory(double downloadSpeed, double uploadSpeed) {
    // Hitung rata-rata kecepatan unduhan dan unggahan
    double averageSpeed = (downloadSpeed + uploadSpeed) / 2;

    // Menentukan kategori kecepatan berdasarkan rata-rata
    if (averageSpeed >= 50) {
      return "Your internet is very fast!";
    } else if (averageSpeed >= 10) {
      return "Your internet is fast!";
    } else {
      return "Your internet is slow!";
    }
  }
```

Akhirnya, setelah tes selesai, backend memberikan penjelasan tambahan mengenai kecepatan internet berdasarkan hasil tes, yang menjelaskan apakah koneksi cukup cepat untuk melakukan berbagai aktivitas online seperti streaming atau gaming. Penjelasan ini bertujuan untuk memberi konteks lebih lanjut kepada pengguna mengenai kualitas jaringan mereka dan memberikan rekomendasi aktivitas yang dapat dilakukan sesuai dengan kecepatan yang terukur. Sehingga aplikasi dapat mengukur dan menampilkan kecepatan internet secara real-time, memberikan hasil yang akurat kepada pengguna, serta membantu mereka memahami kualitas koneksi mereka dengan kategori dan penjelasan yang mudah dipahami.

2. Sniffing

Dari sisi backend dalam pengembangan fitur Sniffing, penerapan backend dilakukan dengan menggunakan konsep pengujian dan pemantauan paket jaringan yang diterima oleh aplikasi. Backend berfungsi untuk memantau dan mencatat status koneksi jaringan, menangani pengiriman dan penerimaan paket, serta mengelola hasil sniffing yang dilakukan pada jaringan. Pada dasarnya, backend berperan dalam mengumpulkan dan memproses data terkait paket-paket yang ditangkap dalam lalu lintas jaringan, yang kemudian akan ditampilkan pada frontend sebagai informasi yang berguna bagi pengguna. Dalam pengembangan fitur ini, backend menggunakan konsep pemrograman jaringan dan HTTP request-response untuk menangani permintaan paket secara dinamis dan memonitor status jaringan secara real-time.

Proses backend dimulai ketika aplikasi diaktifkan untuk melakukan sniffing, yang dipicu oleh tombol Start. Begitu sniffing dimulai, backend akan secara berkala mengirimkan permintaan HTTP ke endpoint tertentu untuk memantau status jaringan dan menangkap paket data. Fungsi _makeRequest() dalam kode ini berperan untuk mengirimkan berbagai jenis permintaan HTTP ke server, termasuk GET, POST, PUT, PATCH, dan DELETE, tergantung pada endpoint yang dipilih secara acak. Pengiriman permintaan ini bertujuan untuk memantau respons server serta memperoleh informasi terkait waktu respons, jumlah data yang diterima, dan status koneksi, yang semuanya penting untuk menganalisis kualitas jaringan secara keseluruhan. Berikut adalah implementasi fungsi pengiriman permintaan secara acak:

```
Future<void> _makeRequest() async {
    if (!isStarted) return; // Pastikan sniffing aktif

    // Pengecekan koneksi internet menggunakan fungsi checkInternetConnection
    bool hasInternet = await NetworkCheck.isInternetAvailable();
    if (!hasInternet) {
      print("Tidak ada koneksi internet.");
      // Mengatur hasil sniffing untuk koneksi yang buruk
      InspectorResult result = InspectorResult(
        url: Uri.parse("https://jsonplaceholder.typicode.com"),
        startTime: DateTime.now(),
        statusCode: null,
        reasonPhrase: 'No Internet Connection',
        responseBodyBytes: 0,
        endTime: DateTime.now(),
      );
      // Tambahkan hasil ke inspectorNotifierList untuk memicu notifyListeners
      final updatedList =
          List<InspectorResult>.from(FNICLient.inspectorNotifierList.value);
      updatedList.add(result);
      FNICLient.inspectorNotifierList.value = updatedList;

      return;
    }

    try {
      List<String> endpoints = [
        '/posts', '/posts/1', '/posts/1/comments', '/comments?postId=1',
      ];

      String selectedEndpoint = endpoints[Random().nextInt(endpoints.length)];
      String requestMethod;
      if (selectedEndpoint == '/posts') {
        requestMethod = 'POST';
      } else if (selectedEndpoint == '/posts/1') {
        requestMethod = Random().nextBool() ? 'PUT' : 'PATCH';
      } else {
        requestMethod = 'GET';
      }

      print("Sending                          request                          to
https://jsonplaceholder.typicode.com$selectedEndpoint");

      http.Response response;
      if (requestMethod == 'POST') {
        response = await _client.post(
          Uri.parse('https://jsonplaceholder.typicode.com$selectedEndpoint'),
          body: {'title': 'New Post', 'body': 'This is a new post.'},
        );
      } else {
```

```
        response                                =                                 await
_client.get(Uri.parse('https://jsonplaceholder.typicode.com$selectedEndpoint'));
      }

      if (response.statusCode == 200) {
        print("Response status: ${response.statusCode}");
        print("Response body: ${response.body}");
      } else {
        print("Error: ${response.statusCode} ${response.reasonPhrase}");
      }
    } catch (e) {
      print("Connection error: $e");
      InspectorResult result = InspectorResult(
        url: Uri.parse("https://jsonplaceholder.typicode.com"),
        startTime: DateTime.now(),
        statusCode: null,
        reasonPhrase: 'Connection Error',
        responseBodyBytes: 0,
        endTime: DateTime.now(),
      );
      final updatedList =
          List<InspectorResult>.from(FNICLient.inspectorNotifierList.value);
      updatedList.add(result);
      FNICLient.inspectorNotifierList.value = updatedList;
    }
  }
```

Fungsi di atas mengirimkan permintaan HTTP ke endpoint tertentu dan memeriksa apakah server merespons dengan status 200 (OK). Jika ada masalah dalam koneksi, hasilnya akan menunjukkan kesalahan koneksi dan menambahkan informasi tersebut ke dalam daftar hasil sniffing. Proses ini memungkinkan backend untuk terus memantau lalu lintas jaringan dan memberikan laporan yang akurat kepada frontend mengenai kondisi jaringan saat itu.

Backend juga bertanggung jawab untuk menghitung dan menampilkan metrik jaringan, seperti tingkat keberhasilan permintaan, waktu respons rata-rata, data yang diterima, dan jumlah paket yang diblokir. Fungsi _calculateNetworkMetrics() menghitung metrik-metrik ini berdasarkan data yang ditangkap oleh sniffing, yang kemudian digunakan untuk menampilkan NetworkStatusPopup di frontend. Fungsi ini menghitung tingkat keberhasilan paket yang diterima, total waktu respons, serta total data yang diterima, dan akhirnya memberikan informasi yang bermanfaat kepada pengguna mengenai kualitas jaringan mereka. Berikut adalah implementasi fungsi perhitungan metrik:

```
void _calculateNetworkMetrics() {
    if (_sniffedData.isEmpty) return;

    int successfulRequests = _sniffedData
        .where((result) =>
            result.statusCode != null &&
            result.statusCode! >= 200 &&
            result.statusCode! < 300)
        .length;

    int blockedPackets = _sniffedData
        .where((result) =>
            result.statusCode != null &&
            (result.statusCode! >= 400 && result.statusCode! < 600))
        .length;
```

```
    double successRate = (successfulRequests / _sniffedData.length) * 100;

    int totalResponseTime = 0;
    int totalDataReceived = 0;

    for (var result in _sniffedData) {
      if (result.startTime != null && result.endTime != null) {
        int responseTime =
            result.endTime!.difference(result.startTime!).inMilliseconds;
        totalResponseTime += responseTime;
      }
      totalDataReceived += result.responseBodyBytes ?? 0;
    }

    int avgResponseTime = _sniffedData.isNotEmpty
        ? (totalResponseTime / _sniffedData.length).round()
        : 0;

    showModalBottomSheet(
      context: context,
      backgroundColor: Colors.transparent,
      builder: (BuildContext context) {
        return NetworkStatusPopup(
          successRate: successRate,
          avgResponseTime: avgResponseTime,
          totalDataReceived:
              (totalDataReceived / 1024).round(), // Konversi ke KB
          blockedPackets: blockedPackets,
          successfulPackets: successfulRequests, // Pass this value
        );
      },
    );
  }
```

Proses backend ini memungkinkan aplikasi untuk menyediakan statistik secara real-time mengenai kinerja jaringan, yang sangat berguna untuk analisis jaringan dan pemecahan masalah. Metrik yang dihitung akan ditampilkan dalam NetworkStatusPopup, memberikan wawasan yang berguna bagi pengguna terkait kecepatan, kualitas, dan status koneksi mereka. Menghasilkan penanganan dari pengujian kecepatan dan konektivitas, serta juga memberikan informasi yang jelas dan bermanfaat kepada pengguna melalui antarmuka yang dinamis.

3. Terminal

Dari sisi backend dalam pengembangan fitur Terminal, dengan mengandalkan penggunaan protokol SSH untuk memungkinkan pengguna terhubung ke server jarak jauh dan menjalankan perintah terminal. Backend dalam hal ini terfokus pada pengelolaan koneksi ke server menggunakan SSH, penyimpanan host yang tersimpan, serta pengelolaan data untuk menghubungkan pengguna dengan host yang telah disimpan. Salah satu bagian penting dari pengembangan backend adalah kemampuan untuk menyimpan dan memuat informasi host (seperti alamat IP, username, dan password) untuk memudahkan koneksi SSH tanpa harus memasukkan kredensial secara manual setiap kali. Backend menggunakan SharedPreferences untuk menyimpan daftar host, dan ketika host dipilih, informasi ini digunakan untuk membangun koneksi SSH menggunakan kredensial yang disimpan. Fungsi utama untuk memuat dan

menyimpan host adalah _loadHosts() dan _saveHost(), yang berperan dalam memanipulasi data yang disimpan di perangkat pengguna, seperti yang terlihat pada kode berikut:

```
Future<void> _loadHosts() async {
    final prefs = await SharedPreferences.getInstance();
    List<String>? hostList = prefs.getStringList('savedHosts');
    if (hostList != null) {
      setState(() {
        savedHosts = hostList.map((e) {
          final parts = e.split('|');
          return {
            'host': parts[0],
            'username': parts[1],
            'password': parts.length > 2 ? parts[2] : "",
          };
        }).toList();
      });
      print('Loaded hosts: $savedHosts'); // Debugging
    }
  }

Future<void> _saveHost(String host, String username, String password,
      {bool isEdit = false, int? index}) async {
    final prefs = await SharedPreferences.getInstance();
    List<String> hostList = prefs.getStringList('savedHosts') ?? [];
    if (isEdit && index != null) {
      hostList[index] = '$host|$username|$password';
      savedHosts[index] = {
        'host': host,
        'username': username,
        'password': password, // Pastikan password diperbarui di savedHosts
      };
    } else {
      bool exists = hostList.any((item) {
        final parts = item.split('|');
        return parts[0] == host && parts[1] == username;
      });

      if (!exists) {
        hostList.add('$host|$username|$password');
        savedHosts.add({
          'host': host,
          'username': username,
          'password': password,
        });
      } else {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('This configuration already exists!')),
        );
        return;
      }
    }

    await prefs.setStringList('savedHosts', hostList);
    _loadHosts(); // Refresh UI
  }
```

Dengan backend yang terintegrasi dengan SharedPreferences, aplikasi dapat memuat daftar host yang telah disimpan sebelumnya dan menampilkannya kepada pengguna dalam antarmuka pengguna. Pengguna dapat memilih host yang diinginkan untuk terhubung dengan server, yang memicu backend untuk memulai koneksi SSH menggunakan kredensial yang dipilih. Koneksi SSH ini menggunakan pustaka

dartssh2, yang menangani semua komunikasi antara aplikasi dan server melalui protokol SSH. Fungsi _showTerminalScreen() digunakan untuk menampilkan antarmuka terminal setelah koneksi berhasil, memungkinkan pengguna untuk menjalankan perintah-perintah di server.

Koneksi SSH di backend diatur dengan kelas SSHConnection, yang memfasilitasi metode untuk menghubungkan ke server menggunakan username dan password yang disimpan, serta mengeksekusi perintah terminal. Koneksi ini penting untuk memungkinkan pengguna menjalankan perintah langsung di server jarak jauh tanpa membuka terminal fisik. Backend menangani proses otentikasi dan eksekusi perintah SSH, sementara frontend bertanggung jawab untuk menampilkan hasil perintah dan memungkinkan pengguna berinteraksi dengan terminal secara langsung. Berikut adalah implementasi koneksi SSH di backend:

```
class SSHConnection {
  SSHClient? _client;
  bool _isConnected = false;

  bool get isConnected => _isConnected;

  Future<void> connect(String host, String username, String password) async {
    try {
      final socket = await SSHSocket.connect(host, 22);
      _client = SSHClient(socket, username: username, onPasswordRequest: () =>
password);
      await _client!.authenticated;
      _isConnected = true;
      print("Connected to $host");
    } catch (e) {
      _isConnected = false;
      print("Connection failed: $e");
    }
  }

  Future<String> executeCommand(String command) async {
    if (_client != null) {
      final result = await _client!.run(command);
      return utf8.decode(result);
    } else {
      return "Not connected to SSH server.";
    }
  }

  void close() {
    _client?.close();
    _isConnected = false;
  }
}
```

Di backend, setelah koneksi berhasil dilakukan, pengguna dapat menjalankan perintah-perintah sistem di server menggunakan metode executeCommand(). Metode ini menerima perintah sebagai parameter dan mengirimkannya ke server yang terhubung melalui protokol SSH. Hasil dari perintah tersebut kemudian dikembalikan sebagai string yang dapat ditampilkan di antarmuka terminal aplikasi. Hal ini memungkinkan pengguna untuk berinteraksi dengan server secara langsung, mengeksekusi perintah dan mendapatkan hasilnya dalam waktu nyata. Backend juga menangani penyimpanan konfigurasi koneksi, di mana pengguna dapat menyimpan beberapa host dengan kredensialnya untuk akses cepat di masa depan. Pengguna dapat menambahkan,

mengedit, atau menghapus konfigurasi host, yang secara otomatis memperbarui daftar host yang disimpan di SharedPreferences. Fungsi-fungsi backend ini memberikan fleksibilitas bagi pengguna untuk mengelola beberapa koneksi ke berbagai server tanpa harus memasukkan kredensial setiap kali. Pengelolaan host ini dijalankan melalui fungsi _saveHost(), _loadHosts(), dan _deleteHost(), yang memastikan bahwa data host tersimpan dan dapat dimodifikasi sesuai kebutuhan.

Penerapan backend dalam fitur Terminal memungkinkan aplikasi untuk mengelola koneksi SSH, menjalankan perintah terminal di server, dan menyimpan konfigurasi host yang digunakan untuk akses cepat. Semua proses ini dilakukan dengan menggunakan pustaka dartssh2 untuk koneksi SSH dan SharedPreferences untuk penyimpanan data host. Dengan backend yang solid ini, aplikasi dapat memberikan pengalaman pengguna yang lancar dalam mengelola dan berinteraksi dengan server jarak jauh, memudahkan administrasi sistem tanpa harus beralih ke terminal fisik.

4. VPN

Di sisi backend, aplikasi mengelola data VPN dengan menghubungkan ke API eksternal untuk mendapatkan daftar server VPN yang tersedia. Data ini diambil menggunakan permintaan HTTP dan diproses untuk membuat objek Vpn yang berisi informasi tentang host, kecepatan, ping, dan lainnya. Backend menangani pengambilan data server VPN melalui API vpnGate, di mana data yang diterima kemudian diubah menjadi objek Vpn dan disimpan dalam daftar. Selain itu, aplikasi memungkinkan pengguna untuk mengonfigurasi dan memilih server VPN yang terhubung, serta mengelola koneksi SSH untuk memungkinkan akses ke server melalui protokol VPN. Berikut adalah contoh kode backend untuk mengambil data server VPN dan membuat objek Vpn:

```
class Apis {
  static Future<List<Vpn>> getVPNservers() async {
    final List<Vpn> vpnList = [];
    HttpClient httpClient = HttpClient()..badCertificateCallback =
        (X509Certificate cert, String host, int port) => true;
    IOClient ioClient = IOClient(httpClient);

    try {
      final                    response                    =                    await
ioClient.get(Uri.parse("https://www.vpngate.net/api/iphone/"));
      if (response.statusCode == 200) {
        String body = response.body;
        List<String> vpnDataList = body.split('\n');
        for (var vpnData in vpnDataList) {
          if (vpnData.isNotEmpty) {
            List<String> columns = vpnData.split(',');
            if (columns.length >= 14) {
              Vpn vpn = Vpn(
                hostname: columns[0],
                ip: columns[1],
                score: int.parse(columns[2]),
                ping: int.tryParse(columns[3]) ?? 0,
```

```
                speed: int.parse(columns[4]),
                countryLong: columns[5],
                countryShort: columns[6],
                numVpnSessions: int.parse(columns[7]),
                upTime: int.parse(columns[8]),
                totalUser: int.parse(columns[9]),
                totalTrafic: int.parse(columns[10]),
                longType: columns[11],
                operator: columns[12],
                message: columns[13],
                base64OpenVPNConfigData: columns.length > 14 ? columns[14] : '',
              );
              vpnList.add(vpn);
            }
          }
        }
      } else {
        print('Failed to load VPN servers');
      }
    } catch (e) {
      print('Error: $e');
    } finally {
      ioClient.close();
    }
    vpnList.shuffle();
    return vpnList;
  }
}
```

5. File Transfer

Dari sisi backend dalam pengembangan fitur File Transfer, backend bertanggung jawab untuk menangani koneksi SSH dan SFTP yang memungkinkan pengguna untuk mentransfer file antara perangkat mereka dan server jarak jauh. Fungsi utama backend dalam fitur ini adalah mengelola koneksi ke server, melakukan operasi file di server seperti membaca, menulis, dan mengunggah file, serta memungkinkan pengguna untuk menjelajahi sistem file server menggunakan protokol SFTP. Backend mengelola status koneksi, menangani perintah untuk mengakses direktori, dan memanipulasi file sesuai dengan permintaan pengguna, baik itu untuk membaca file, mengunggah file, atau memeriksa atribut file.

Backend menggunakan pustaka dartssh2 untuk menyediakan fungsi-fungsi SSH dan SFTP yang diperlukan. Setiap kali pengguna memilih server untuk terhubung, aplikasi memulai koneksi SSH menggunakan kredensial yang disimpan dan menyediakan antarmuka untuk menjalankan operasi file. Misalnya, saat pengguna memilih untuk mengunggah file, backend akan menangani pengiriman file menggunakan SFTP untuk memastikan file ditransfer dengan benar ke server. Fungsi-fungsi seperti uploadFile, listDirectory, dan readFile digunakan untuk memungkinkan berbagai operasi file di server. Berikut adalah kode untuk mengunggah file ke server melalui koneksi SFTP:

```
Future<void> uploadFile(String remotePath, File localFile) async {
    if (_client != null) {
      final sftp = await _client!.sftp();
```

```
      final file = await sftp.open(remotePath,
          mode: SftpFileOpenMode.create | SftpFileOpenMode.write);
      await file.write(localFile.openRead().cast());
    } else {
      throw Exception("Not connected to SSH server.");
    }
  }
```

Fungsi uploadFile ini memungkinkan backend untuk menangani pengunggahan file dari perangkat pengguna ke server yang terhubung. Fungsi ini menerima dua parameter: path tujuan file di server dan file lokal yang ingin diunggah. Backend akan mengonversi file lokal menjadi aliran byte dan mengirimkannya ke server melalui protokol SFTP. Proses pengunggahan dilakukan dengan membuka file di server dalam mode create dan write, yang memastikan bahwa file yang diunggah akan dibuat dan ditulis ke server.

Selain itu, backend juga memungkinkan pengguna untuk menjelajahi direktori di server menggunakan SFTP. Fungsi listDirectory digunakan untuk mengambil daftar file dan folder yang ada di direktori server. Ketika pengguna ingin melihat konten suatu direktori, backend akan mengirimkan permintaan SFTP untuk mendapatkan daftar item di dalam direktori tersebut. Fungsi ini memungkinkan pengguna untuk menavigasi struktur direktori server dan memilih file yang ingin mereka operasikan, baik itu untuk dibaca, diunggah, atau dihapus. Berikut adalah implementasi backend untuk menampilkan daftar direktori:

```
Future<List<SftpName>> listDirectory(String path) async {
    if (_client != null) {
      final sftp = await _client!.sftp();
      final items = await sftp.listdir(path);
      return items;
    } else {
      throw Exception("Not connected to SSH server.");
    }
  }
```

Setelah backend memuat daftar direktori atau file, data tersebut dikirim ke frontend untuk ditampilkan kepada pengguna dalam antarmuka yang mudah digunakan. Backend mengelola koneksi dan operasi file, sementara frontend bertanggung jawab untuk menampilkan hasil dan memungkinkan pengguna berinteraksi dengan file yang ada di server.

Selain pengelolaan file, backend juga menangani penyimpanan dan pengelolaan informasi host yang disimpan oleh pengguna. Informasi host (seperti hostname, username, dan password) disimpan di SharedPreferences, yang memungkinkan pengguna untuk menyimpan beberapa konfigurasi host dan mengaksesnya dengan mudah. Fungsi seperti _loadHostsFromPreferences() dan _saveHostsToPreferences() digunakan untuk memuat dan menyimpan informasi host di backend, memastikan bahwa pengguna dapat mengakses kembali host yang telah mereka simpan tanpa harus memasukkan kredensial setiap kali. Berikut adalah implementasi penyimpanan host:

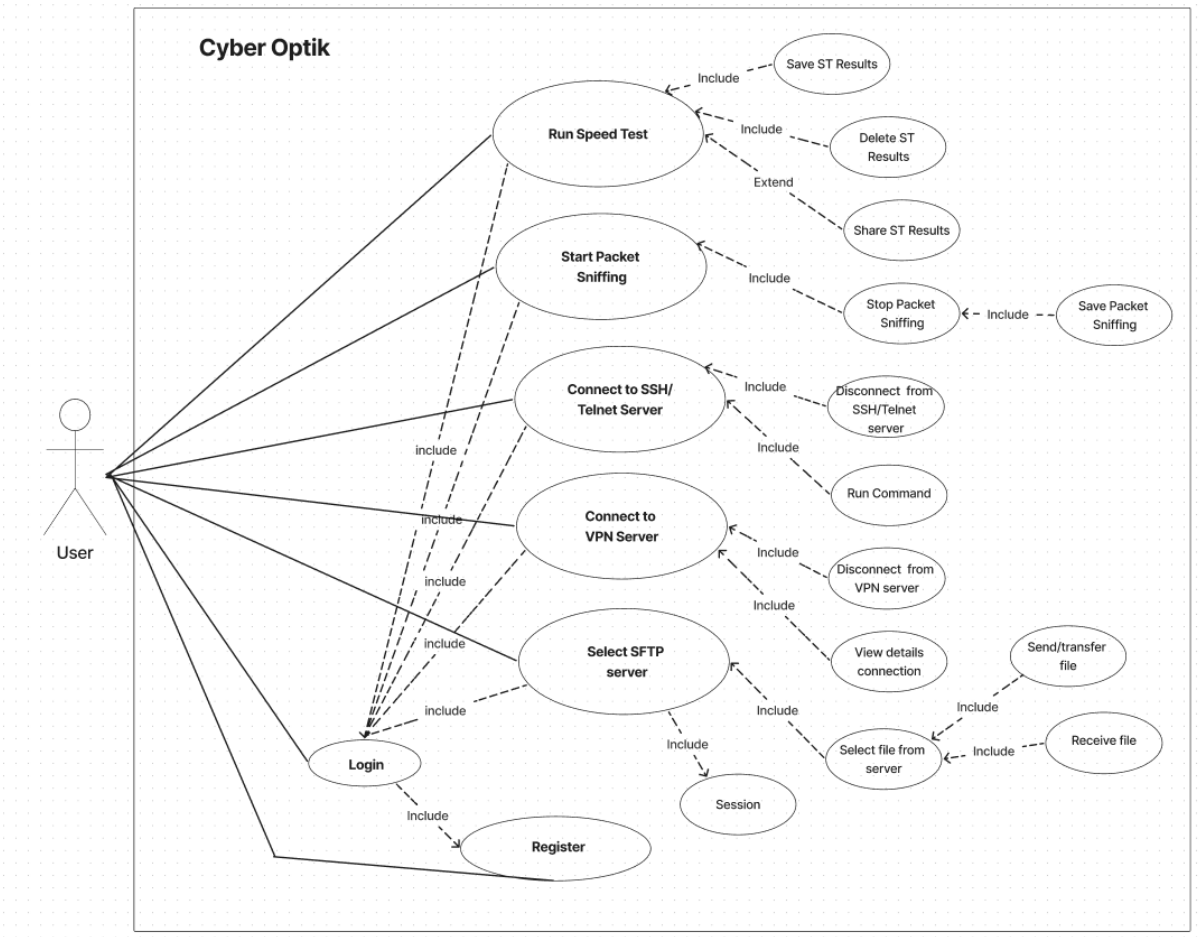
```
Future<void> _saveHostsToPreferences() async {
  SharedPreferences prefs = await SharedPreferences.getInstance();
  List<String> hostsJsonList =
      hostServers.map((host) => jsonEncode(host)).toList();
```

```
  await prefs.setStringList('hostServers', hostsJsonList);
}
```

Fungsi _saveHostsToPreferences() ini menyimpan daftar host yang disimpan pengguna dalam format JSON ke SharedPreferences, sehingga ketika aplikasi dimulai lagi, backend dapat memuat informasi host tersebut dan menampilkannya kepada pengguna. Backend juga memastikan tidak ada duplikasi dalam daftar host yang disimpan, dengan memeriksa apakah host yang akan disimpan sudah ada sebelumnya.

Dengan memanfaatkan pustaka dartssh2, backend dapat menyediakan semua fungsionalitas yang diperlukan untuk mentransfer file dengan lancar dan aman antara perangkat pengguna dan server yang terhubung.

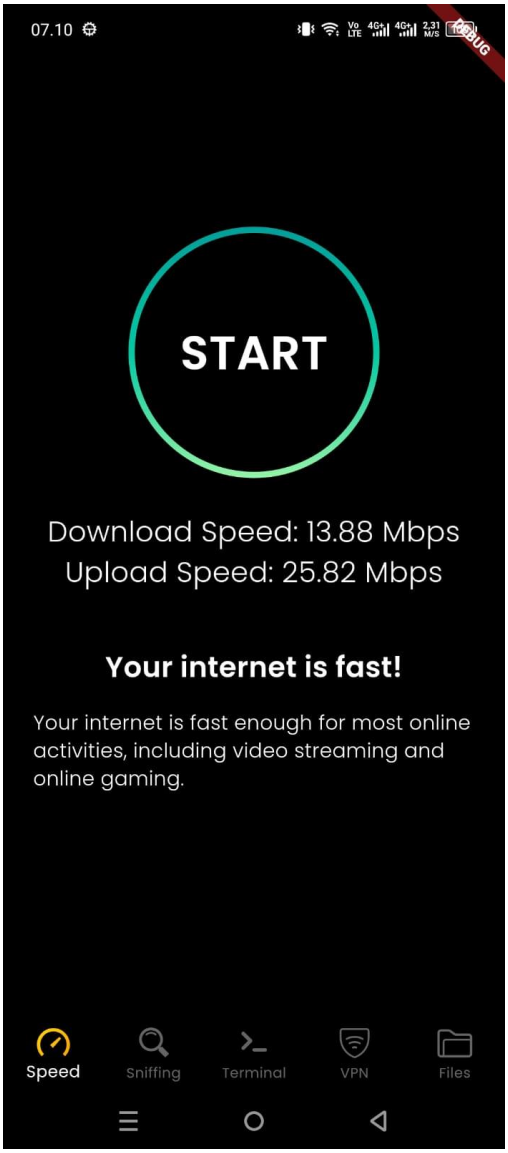
VIII. Use Case Diagram

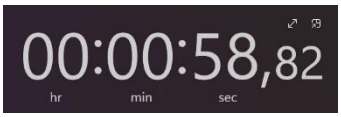


IX. Hasil Pengujian

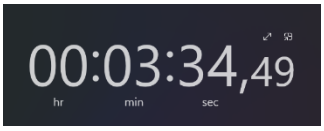
Pengujian pada Speedtest

1. Pengujian di lingkungan rumah (cepat)



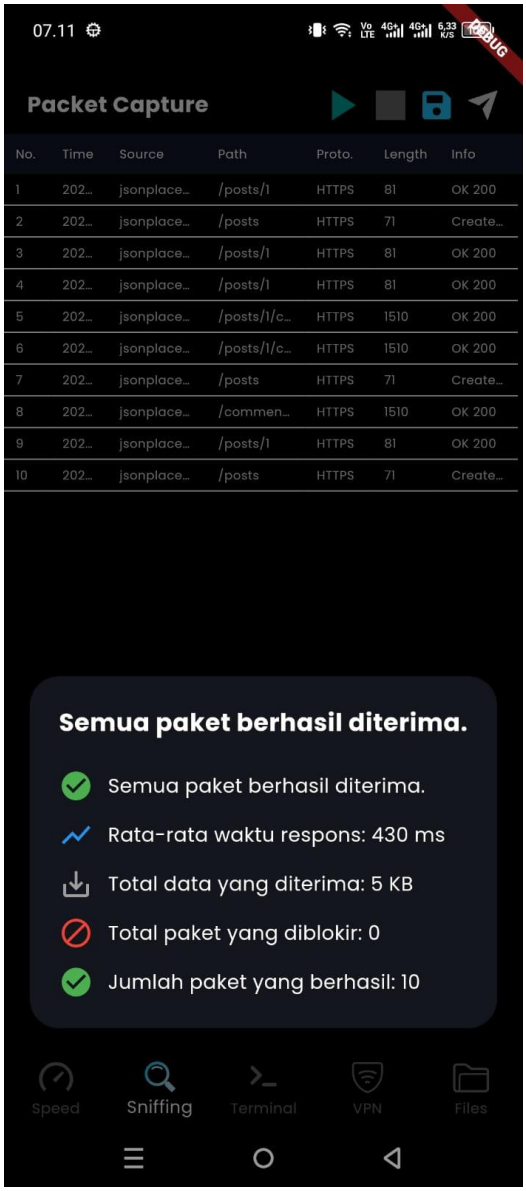


2. Pengujian di lingkungan PENS (lambat)



Pengujian Sniffing

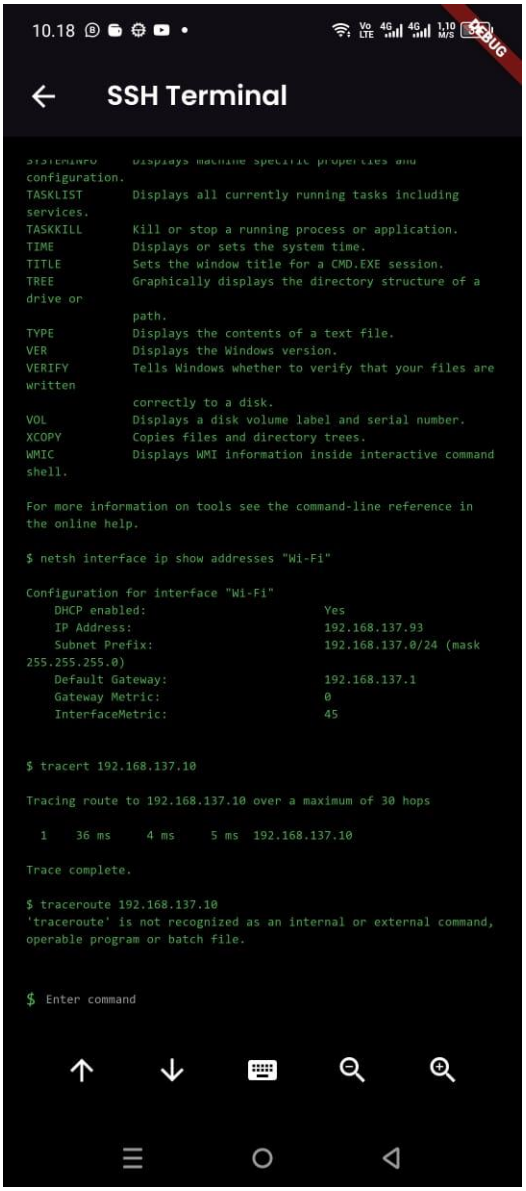
1. Pengujian sniffing hingga 10 paket



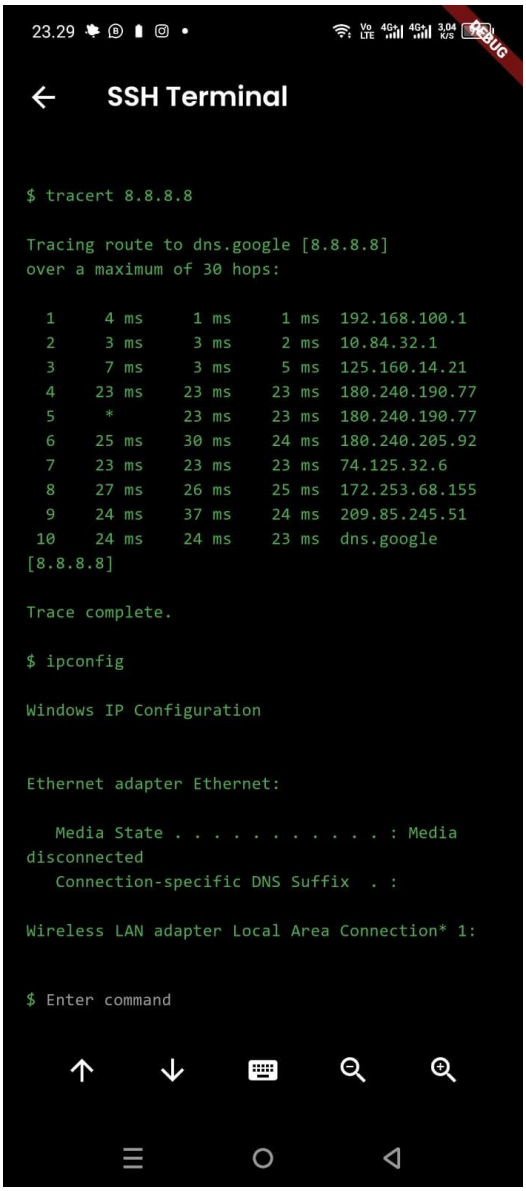
2. Pengujian sniffing hingga 99 paket

Pengujian Terminal

1. Pengujian dengan menggunakan SSH (server Win)

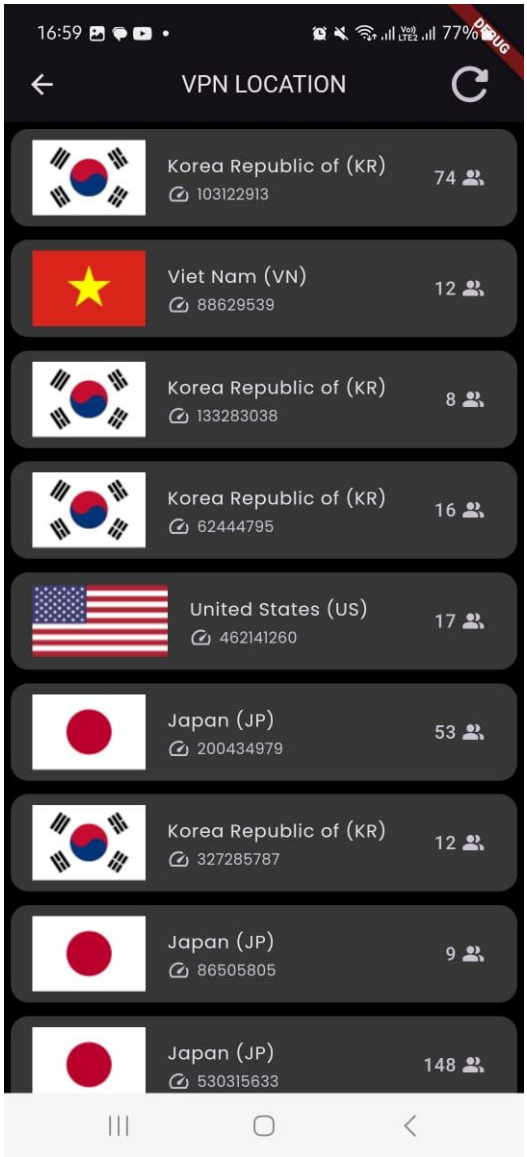


2. Pengujian tracert di terminal

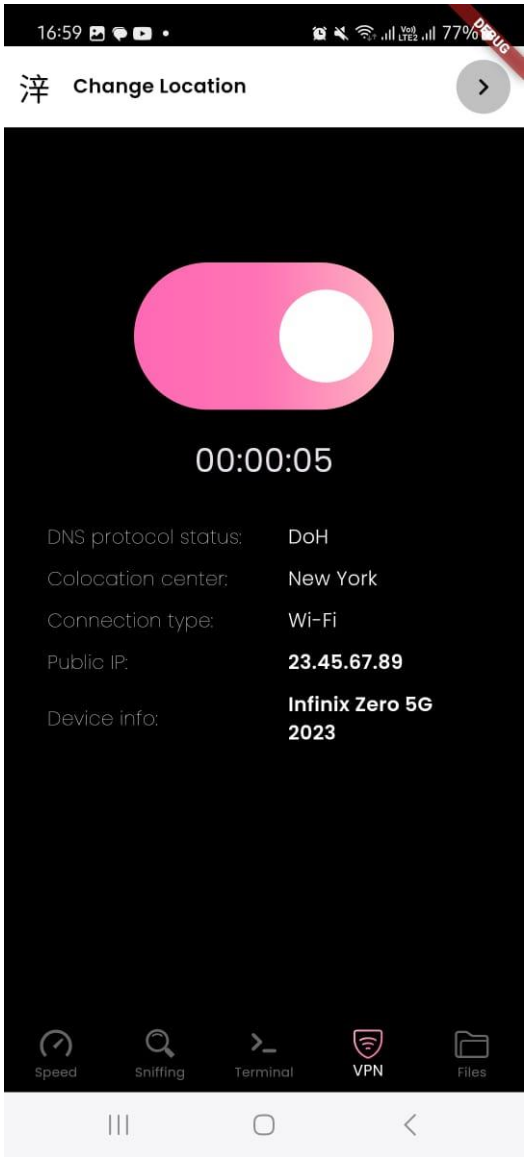


Pengujian VPN

1. Daftar server dari VPN Gate

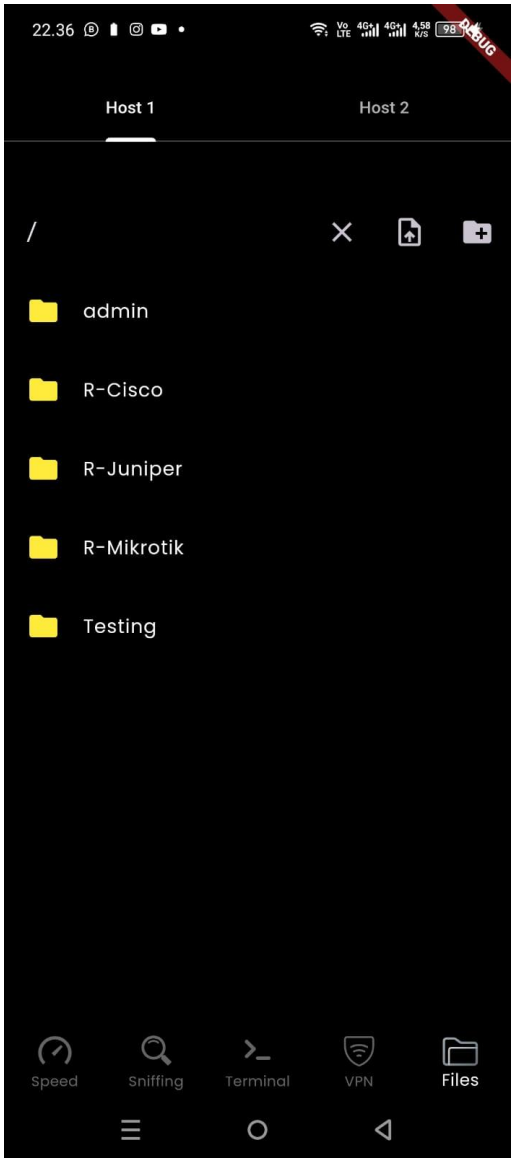


2. Tampilan ketika terhubung



Pengujian File Transfer

1. Tampilan direktori dari server tujuan



X. Review User

Berdasarkan data yang diperoleh dari formulir umpan balik pengguna, aplikasi ini mendapatkan berbagai masukan yang bervariasi mengenai fitur-fitur yang ada.

Respon Pengguna aplikasi Cyber Optik (Jawaban) - Google Spreadsheet

Banyak pengguna mengungkapkan bahwa fitur Speedtest cukup bermanfaat, dengan beberapa pengguna menyarankan untuk menambahkan animasi interaktif untuk meningkatkan pengalaman pengguna. Sebagian besar pengguna merasa fitur Sniffing aman digunakan, meskipun ada beberapa yang memberikan masukan terkait peningkatan fitur VPN yang masih belum berfungsi dengan baik, serta penambahan fitur baru seperti pemecah password Wi-Fi. Selain itu, fitur Files juga mendapat respons

yang positif terkait kemudahan penggunaannya, meskipun beberapa pengguna merasa aplikasi dapat diperbaiki dengan menambah lebih banyak fitur seperti menghubungkan perangkat tanpa menggunakan backend atau database.

Secara keseluruhan, pengguna merasa aplikasi ini membantu aktivitas jaringan mereka, dengan banyak yang menyatakan akan merekomendasikan aplikasi ini kepada orang lain. Namun, mereka juga memberikan beberapa masukan penting, seperti peningkatan performa pada fitur VPN dan penambahan animasi yang lebih menarik untuk fitur Speedtest agar tampak lebih interaktif. Ada juga saran untuk mengembangkan Terminal agar lebih sesuai dengan kebutuhan pengguna yang membutuhkan konektivitas yang lebih stabil dan cepat. Dengan demikian, aplikasi ini dapat terus ditingkatkan dengan mengikuti kebutuhan pengguna yang lebih dinamis dan berkembang.

XI. Kesimpulan

Aplikasi Cyber Optik telah berhasil dikembangkan sebagai solusi multifungsi untuk memantau dan meningkatkan kualitas koneksi jaringan. Dengan fitur seperti pengujian kecepatan internet, sniffing untuk memantau paket data, terminal untuk akses jarak jauh, VPN untuk keamanan, serta transfer file melalui SFTP, aplikasi ini menawarkan kontrol penuh kepada pengguna terhadap koneksi mereka. Setiap fitur dirancang dengan antarmuka yang intuitif, memberikan pengalaman pengguna yang optimal, serta memastikan keamanan data dan privasi. Penggunaan aplikasi ini diharapkan dapat meningkatkan efisiensi aktivitas online, baik untuk kebutuhan personal maupun profesional. Dengan pemantauan kualitas jaringan secara real-time dan kemampuan untuk mengamankan koneksi melalui VPN, pengguna dapat lebih percaya diri dalam mengelola aktivitas digital mereka. Keseluruhan, Cyber Optik menjadi dapat solusi yang relevan di era digital, memberikan manfaat yang signifikan dalam mendukung produktivitas dan keamanan jaringan.